

nanoGPT: PyTorch 호출에서 실제 CUDA 커널까지

실행 결과 PDF · Dropdown/slider의 모든 구간과 단계를 펼쳤습니다. 원본 interactive 출력은 main.ipynb에서 확인할 수 있습니다.

실행하고, 그림으로 보고, 같은 계산을 끝까지 따라가는 **compiler lab**

`nn.Module`과 `loss.backward()`를 사용해 봤다면 시작할 수 있습니다. Python 내부, compiler, CUDA programming은 여기서 작은 예제로 익힙니다.

우리가 답할 질문은 하나입니다. **model(tokens)**의 계산은 어떻게 **graph, loop, buffer**를 거쳐 GPU에서 실행되는가?

- 모든 tensor-compiler-성능 실습의 target은 **NVIDIA CUDA**입니다. 기본값은 작은 입력과 FP32입니다.
- Graph-IR-Triton-profiler 출력은 .py 실험을 새 프로세스로 실행해서 생성합니다. 동봉된 결과나 이전 run을 읽지 않습니다.
- 그림 위에 **실측/capture**와 **개념도/이론 계산**을 구분합니다. Node의 화살표는 보통 값의 의존성이며 시간축이 아닙니다.
- GPU 성능과 생성된 kernel 수는 버전·shape·dtype에 따라 달라집니다. 본문은 결과를 관찰하는 방법을 설명합니다.

원전: nanoGPT: PyTorch 코드에서 실제 GPU 커널까지, compiler_lab의 85쪽 자료. 원전의 CPU 예제는 이 노트북에서 CUDA로 다시 실행합니다. 모델 가중치는 작은 무작위 초기값이며 다운로드가 없습니다.

학습 순서

구간	학습 절
1. 기초와 모델	00 환경·지도 · 01 Tensor-그래프 · 02 GPU 실행 · 03 GPT block · 04 Eager와 GELU
2. Python에서 graph로	05 Bytecode-symbolic 실행 · 06 Dynamo · 07 AOTAutograd
3. Graph에서 kernel로	08 Inductor lowering · 09 Scheduling-memory · 10 Triton-PTX-SASS
4. 전체 연결	11 GPT block 추적 · 12 Fusion-측정 · 13 Megakernel-vocabulary · 14 복습-실행 길잡이

실행 방법: 저장소 루트에서 `uv sync --locked` 후 `uv run --locked jupyter lab`을 실행하고 이 파일을 엽니다. 커널도 이 저장소의 uv Python을 선택합니다. 처음에는 **Restart Kernel and Run All**로 순서대로 실행하세요. Compiler 실습은 실행 진행 메시지 뒤에 그림이 나옵니다.

00. 환경 확인과 전체 지도

원문: §1, §3, §16 · 목차로

아래 설정은 전체 실습의 기준입니다. `cuda_environment()`가 실패하면 CUDA 환경부터 해결합니다. 실험을 실행하는 Python은 노트북 커널의 `sys.executable`과 같습니다.

`RUN_VOCAB_LAB`은 큰 vocabulary 두 개의 비교, `RUN_NCU`는 권한이 필요한 Nsight Compute 추가 실습입니다. 기본 compiler-CUDA-Triton 실습은 항상 실행합니다.

```
from pathlib import Path
import sys

ROOT = next(p for p in [Path.cwd(), *Path.cwd().parents]
             if (p / 'study02' / 'runtime.py').is_file())
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import math
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
import plotly.graph_objects as go
import plotly.io as pio
```

```

from plotly.subplots import make_subplots
from IPython.display import display
from study02 import visuals as viz
from study02.runtime import cuda_environment
from study02.notebook_utils import run_experiment, show_summary, show_source, show_artifact, details
from study02.model import GPT, GPTConfig, Block, LayerNorm
from study02.examples import ManualGELU, ManualLayerNorm, ResidualLayerNorm, tiny, mutate_view

ENV = cuda_environment()
DEVICE, DTYPE_NAME, SEED = 'cuda', 'float32', 42
DTYPE = getattr(torch, DTYPE_NAME)
RUN_VOCAB_LAB, RUN_NCU = True, True
BLOCK_SHAPE = dict(batch=4, sequence=128, channels=256, vocab_size=1024)
MLP_SHAPE = dict(batch=2, sequence=128, channels=256)
pio.renderers.default = 'plotly_mimetype+notebook'
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.set_num_threads(1)
torch.backends.cuda.matmul.allow_tf32 = False
RUNS = []

def experiment(name, **options):
    result = run_experiment(name, seed=SEED, dtype=DTYPE_NAME, **options)
    RUNS.append(result)
    return result

show_summary({k: ENV[k] for k in ['executable', 'python', 'torch', 'cuda', 'driver', 'triton', 'gpu',
'compute_capability']}, '이번 실행 환경')

```

이번 실행 환경

executable	/home/riverfog7/Workspace/mlsys_study/.venv/bin/python3
python	3.9.25
torch	2.8.0+cu128
cuda	12.8
driver	580.173.02
triton	3.4.0
gpu	NVIDIA GeForce RTX 2080
compute_capability	[7, 5]

Compiler는 한 번에 Python을 GPU 명령으로 바꾸지 않습니다. 표현마다 답할 질문이 다르기 때문입니다.

표현	이제 알 수 있는 것	아직 결정되지 않은 것
Python source	사용자가 원하는 계산-제어 흐름	실제 실행 kernel
Dynamo FX + guards	이번 호출에서의 tensor 계산과 가정	최종 kernel 경계
Functional ATen FX	Mutation을 정리한 연산-미분 계산	Index 순회·memory 배치
Inductor IR	Loop/reduction 공간·layout·buffer·외부 호출	최종 scheduling/code
Scheduled groups + wrapper	계산 묶음, allocation/reuse, 호출 순서	GPU instruction의 세부 구현
Triton → IR → PTX/cubin	GPU에서 실행할 구현	실제 실행 시간·병목
Profiler trace	실행된 kernel과 시간	한 번의 측정만으로 다른 환경의 성능을 예측할 수는 없음

```

viz.concept([
    'Python source', 'Dynamo FX + guards', 'Functional ATen / AOT',
    'Inductor loop · buffer IR', 'Scheduling + memory plan',

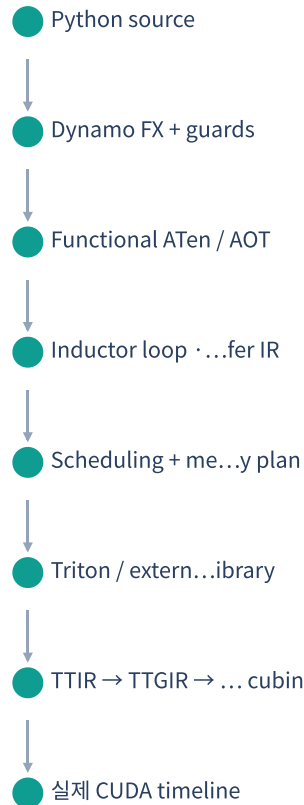
```

```
'Triton / external library', 'TTIR → TTGIR → LLVM IR → PTX / cubin', '실제 CUDA timeline'] , '같은 계산에 정보를 더하는 compiler pipeline').show()
```

그림 1

같은 계산에 정보를 더하는 compiler pipeline

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



확인해 보기 — `torch.compile(model)`을 부르는 순간 kernel 생성이 끝났을까요?

▼ 해설 펼치기

아닙니다. 보통 실제 입력으로 optimized callable을 처음 호출할 때 capture와 compilation이 일어납니다. 같은 가정을 만족하는 다음 호출은 캐시된 실행 경로를 사용할 수 있습니다.

01. Tensor와 그래프를 읽는 법

원문: §2, §3.5 · pp. 8–14 · 목차로

1.1 숫자 저장 공간과 그 공간을 읽는 규칙

Tensor는 `**storage(숫자가 들어 있는 메모리)**`와 `**metadata(shape, stride, dtype, device, storage offset)**`를 함께 가집니다.

- Shape는 축의 길이입니다. [B, T, C]는 batch, token 위치, channel입니다.
- Stride는 해당 축의 index가 1 증가할 때 storage에서 이동하는 원소 칸 수입니다.
- Storage offset은 view의 첫 원소가 storage 안에서 시작하는 위치입니다.

$$\text{offset}(i_0, \dots, i_n) = \text{storage_offset} + \sum_k i_k \text{stride}_k$$

transpose는 보통 읽는 규칙을 바꿉니다. contiguous()는 현재 배치가 연속적이지 않을 때 새 storage에 값을 옮깁니다. C 래 그림은 첫 batch의 값과 원본 storage offset을 함께 보여줍니다.

```
x = torch.arange(24, device=DEVICE).reshape(2, 3, 4)
y = x.transpose(1, 2)
z = y.contiguous()
show_summary({name: {'shape': tuple(t.shape), 'stride': t.stride(),
                    'offset': t.storage_offset(), 'contiguous': t.is_contiguous()}}
             for name, t in [('x', x), ('transpose y', y), ('contiguous z', z)])
show_summary({'x와 y의 storage 공유': x.untyped_storage().data_ptr() == y.untyped_storage().data_ptr(),
             'y와 z의 storage 공유': y.untyped_storage().data_ptr() == z.untyped_storage().data_ptr()})
fig = make_subplots(rows=1, cols=3, subplot_titles=['원본 x[0]', 'view y[0]', 'copy z[0]'])
for col, t in enumerate([x, y, z], 1):
    values = t[0].detach().cpu().numpy()
    row, column = np.indices(values.shape)
    offsets = t.storage_offset() + row * t.stride(1) + column * t.stride(2)
    fig.add_trace(go.Heatmap(z=values.tolist(), customdata=offsets.tolist(), text=values.tolist(),
                            texttemplate='%{text}', colorscale='Viridis', showscale=False,
                            hovertemplate='value=%{z}<br>storage offset=%{customdata}<extra></extra>', row=1,
                            col=col))
fig.update_yaxes(autorange='reversed')
viz.style(fig, '값은 같아도 읽는 규칙과 storage는 다르다', '현재 CUDA tensor · offset은 metadata로 계산',
420).show()
```

확인한 결과

x	{'shape': (2, 3, 4), 'stride': (12, 4, 1), 'offset': 0, 'contiguous': True}
transpose y	{'shape': (2, 4, 3), 'stride': (12, 1, 4), 'offset': 0, 'contiguous': False}
contiguous z	{'shape': (2, 4, 3), 'stride': (12, 3, 1), 'offset': 0, 'contiguous': True}

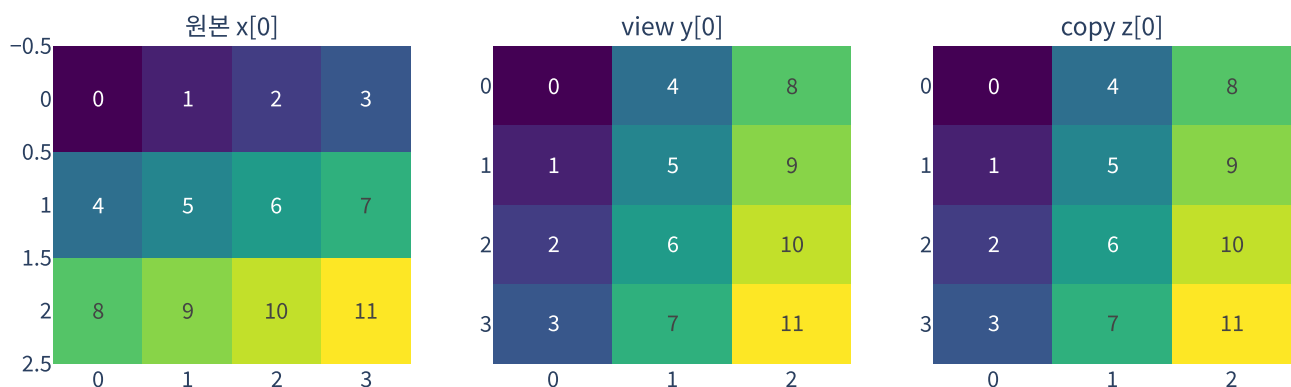
확인한 결과

x와 y의 storage 공유	True
y와 z의 storage 공유	False

그림 2

값은 같아도 읽는 규칙과 storage는 다르다

현재 CUDA tensor · offset은 metadata로 계산



확인해 보기 — y와 z의 값이 같으면 메모리 비용도 같을까요?

▼ 해설 펼치기

값만으로 알 수 없습니다. 여기서 y는 x와 storage를 공유하고, z는 새로운 연속 storage를 가집니다. Logical tensor의 numel을 모두 더하면 공유 메모리를 중복해서 셀 수 있습니다.

1.2 Graph, IR, pass

Graph의 node는 입력·연산·출력을, edge는 “이 값이 저 연산의 입력으로 쓰인다”는 의존성을 나타냅니다. 한 값은 여러 곳에서 쓰일 수 있습니다. 그림의 위아래 위치나 edge 길이는 실행 시간을 뜻하지 않습니다.

IR은 compiler가 사용하는 중간 표현이고, **pass**는 그 표현을 분석하거나 바꾸는 단계입니다. FX는 graph를 담는 형식입니다. Dynamo 직후 FX와 AOT 이후 FX는 같은 형식을 사용해도 담긴 operator와 의미가 다릅니다.

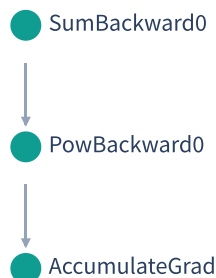
아래는 x^2 를 더한 scalar의 **실제 eager autograd 연결**입니다. 이번 그림의 edge는 backward에서 gradient를 전달하는 방향입니다. Forward의 값 흐름과 구분해 읽으세요.

```
leaf = torch.tensor([1., 2., 3.], device=DEVICE, requires_grad=True)
loss = leaf.square().sum()
loss.backward()
viz.dag(viz.autograd_graph(loss), '실제 eager autograd: scalar loss에서 leaf로').show()
fig = go.Figure()
fig.add_scatter(x=leaf.detach().tolist(), y=leaf.grad.tolist(), mode='lines+markers', name='실제 gradient')
fig.update_xaxes(title='입력 x'); fig.update_yaxes(title='dL/dx')
viz.style(fig, 'L =  $\sum x^2 \rightarrow dL/dx = 2x$ ', '현재 CUDA autograd 결과', 340).show()
```

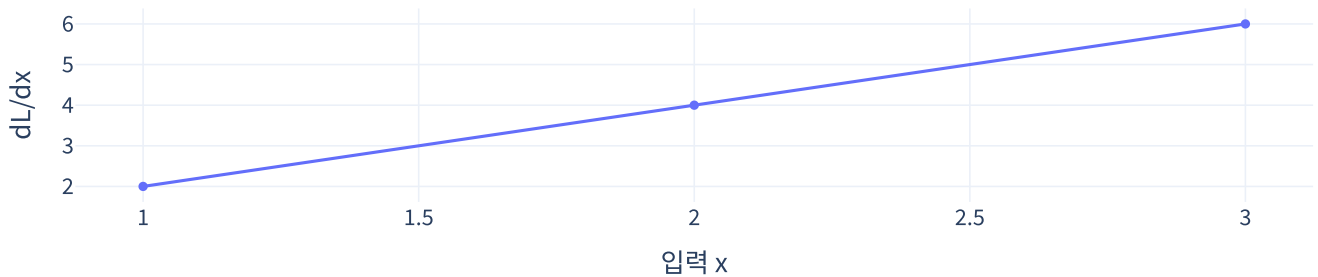
그림 3

실제 eager autograd: scalar loss에서 leaf로

현재 capture · 화살표=값의 의존성, 시간축 아님



$L = \sum x^2 \rightarrow dL/dx = 2x$
현재 CUDA autograd 결과



1.3 Parameter, buffer, 실행 모드

Parameter는 모델에 등록된 학습 대상 tensor이고, buffer는 모델 상태이지만 일반적인 parameter 최적화 대상은 아닙니다. Causal mask가 buffer의 예입니다.

`train()/eval()`은 dropout 같은 module의 동작을 바꿉니다. **Gradient 기록의 on/off는 별도 설정입니다.** targets 유무도 또 다른 문제입니다. 이 nanoGPT는 targets를 전달하면 모든 위치의 logits와 loss를 계산합니다.

```
module = nn.Sequential(nn.Linear(4, 4), nn.Dropout(.5)).to(DEVICE).eval()
input_value = torch.randn(2, 4, device=DEVICE)
normal_output = module(input_value)
with torch.no_grad():
    no_grad_output = module(input_value)
small_block = Block(GPTConfig()).to(DEVICE)
show_summary({'eval 상태': not module.training,
              'eval만 적용한 결과.requires_grad': normal_output.requires_grad,
              'eval + no_grad 결과.requires_grad': no_grad_output.requires_grad,
              'block의 buffer': [(name, list(value.shape)) for name, value in small_block.named_buffers()],
              'block의 parameter tensor 수': len(list(small_block.parameters()))})
```

확인한 결과

eval 상태	True
eval만 적용한 결과.requires_grad	True
eval + no_grad 결과.requires_grad	False
block의 buffer	[('attn.bias', [1, 1, 8, 8])]
block의 parameter tensor 수	12

확인해 보기 — FX node 하나가 GPU kernel 하나를 뜻하나요?

▼ 해설 펼치기

그렇지 않습니다. View는 kernel이 필요 없을 수 있고, 여러 pointwise node가 한 kernel로 합쳐질 수 있습니다. 복합 연산 하나가 여러 kernel을 호출할 수도 있습니다. 실제 경계는 generated wrapper와 CUDA profiler로 확인합니다.

02. GPU 실행을 이해하는 최소한의 배경

원문: §4 · pp. 15–18 · 목차로

2.1 Host, device, 그리고 비동기 실행

Host의 Python/PyTorch는 GPU stream에 작업을 넣습니다. API가 반환됐다고 GPU 계산이 끝난 것은 아닙니다. 정확한 시간 측정은 warm-up 뒤 CUDA event 또는 동기화를 사용합니다.

아래는 설명을 위해 실행 시 만든 개념도입니다. 폭은 측정값이 아닙니다. 실제 timeline은 12절에서 직접 수집합니다.

```
fig = go.Figure()
for lane, starts, durations, labels, color in [
    ('CPU host', [0, 1, 2], [.5, .5, 3.5], ['enqueue A', 'enqueue B', 'synchronize: 완료 대기'], viz.BLUE),
    ('GPU stream', [.5, 2.5], [2, 2.5], ['kernel A', 'kernel B'], viz.TEAL)]:
    fig.add_bar(y=[lane]*len(starts), x=durations, base=starts, orientation='h', name=lane,
                text=labels, marker_color=color, customdata=labels,
                hovertemplate='%{customdata}<extra></extra>')
fig.update_layout(barmode='overlay'); fig.update_xaxes(title='설명용 순서 (측정 시간 아님)',
showticklabels=False)
viz.style(fig, 'API 반환과 GPU 계산 완료는 다른 시점', '개념도 · 수치는 배치용이며 실측 아님', 320).show()
```

그림 5

API 반환과 GPU 계산 완료는 다른 시점

개념도 · 수치는 배치용이며 실측 아님



2.2 병렬 실행의 단위와 memory 계층

Kernel launch는 grid를 만듭니다. Grid에는 thread block이 있고, NVIDIA에서는 일반적으로 32개 thread가 warp를 이룹니다. SM은 block/warp를 실행하는 GPU의 처리 단위입니다.

저장 공간	주로 공유하는 범위	학습에서 볼 질문
Register	Thread	한꺼번에 살아 있는 값이 너무 많지 않은가?
Shared memory	Thread block	Tile과 reduction 결과를 같은 block 안에서 공유할 수 있는가?
L2 cache	GPU	반복 접근이 cache에 맞는가?
Global device memory	GPU	Kernel 사이에서 큰 tensor를 쓰고 다시 읽는가?

원문에서 말하는 HBM traffic은 여기서 **global device-memory traffic**으로 읽습니다. RTX 2080의 VRAM은 GDDR6입니다. Memory 계층의 학습 원리는 같고 실제 hardware 수치는 실행 환경을 따릅니다.

2.3 Pointwise, reduction, GEMM

- **Pointwise:** 각 위치의 출력이 독립적입니다. GELU, residual add가 예입니다.
- **Reduction:** 여러 값을 합쳐 통계를 만듭니다. Mean, variance, softmax의 합이 예입니다.
- **GEMM:** 행렬×행렬입니다. Linear는 $[BT, C] @ [C, 4C]$ 로 볼 수 있습니다. Row가 하나면 GEMV에 가까운 매우 얇은 경우가 됩니다.

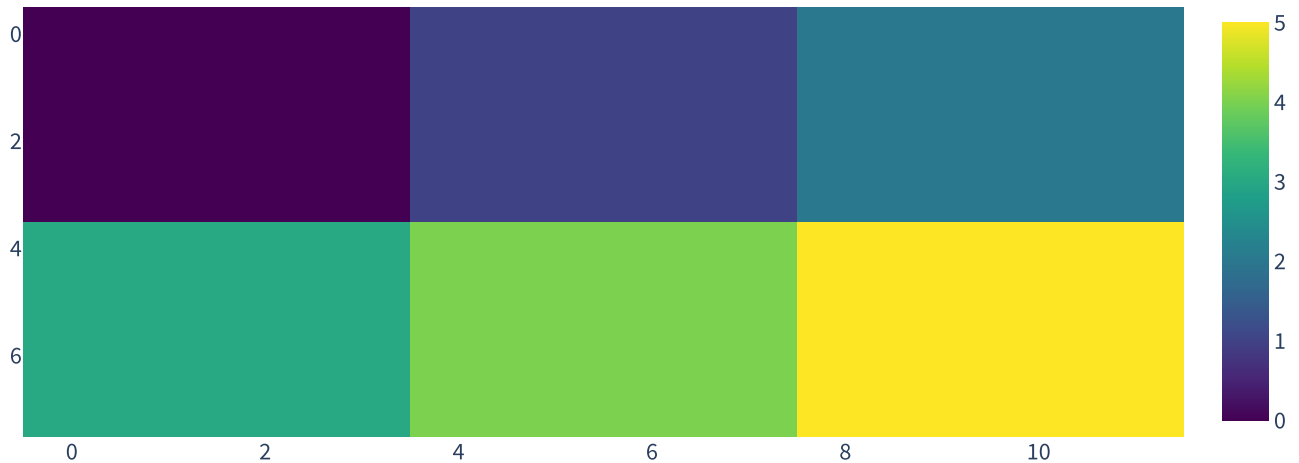
GEMM은 tile 단위로 input을 재사용합니다. Tensor Core는 지원하는 dtype과 tile 방식의 matrix 연산을 가속합니다. **FP32 기본 실험에서 Tensor Core 사용을 가정하지 않습니다.** 단순 pointwise는 데이터 이동, 큰 GEMM은 계산량이 병목이 되기 쉽지만 실제 병목은 측정해야 합니다.

```
M, K, N, TILE = 8, 8, 12, 4
row, col = np.indices((M, N))
tile_id = (row // TILE) * math.ceil(N / TILE) + col // TILE
viz.heatmap(tile_id, 'Output matrix를 tile로 나누기: 각 색은 하나의 tile',
             source='개념도 · index와 tile 크기로 실행 시 계산').show()
show_summary({'prefill 형태': '[B·T, C] × [C, 4C]', 'decode의 얇은 형태': '[1, C] × [C, 4C]',
             '주의': '같은 Linear도 shape, dtype, backend에 따라 다른 kernel을 선택할 수 있음'})
```

그림 6

Output matrix를 tile로 나누기: 각 색은 하나의 tile

개념도 · index와 tile 크기로 실행 시 계산



확인한 결과

prefill 형태	$[B \cdot T, C] \times [C, 4C]$
decode의 얇은 형태	$[1, C] \times [C, 4C]$
주의	같은 Linear도 shape, dtype, backend에 따라 다른 kernel을 선택할 수 있음

확인해 보기 — 연산을 더 많이 합치면 언제나 빨라질까요?

▼ 해설 펼치기

Launch와 global-memory 왕복은 줄어들 수 있지만 register/shared-memory 사용이 증가할 수 있습니다. Register spill이나 occupancy 감소가 이득을 상쇄할 수 있으므로 실제 latency와 자원 사용을 함께 봅니다.

03. 작은 GPT block을 값과 shape로 이해하기

원문: §5 · pp. 19–25 · 목차로

3.1 기호와 모델 입력

기호	뜻	기본 실습
B	Batch 크기	1
T	Token 위치 수	8
C	Hidden width	32
H	Attention heads	4

기호

뜻

기본 실습

D Head dimension = C/H

8

V Vocabulary 차원

128

Text→ID 전처리는 compiled tensor graph 밖입니다. 아래 단어 ID 매핑은 경계를 보여주는 작은 시연용 매핑이며 GPT-2 tokenizer가 아닙니다. 학습 targets는 input보다 한 token 뒤로 이동합니다.

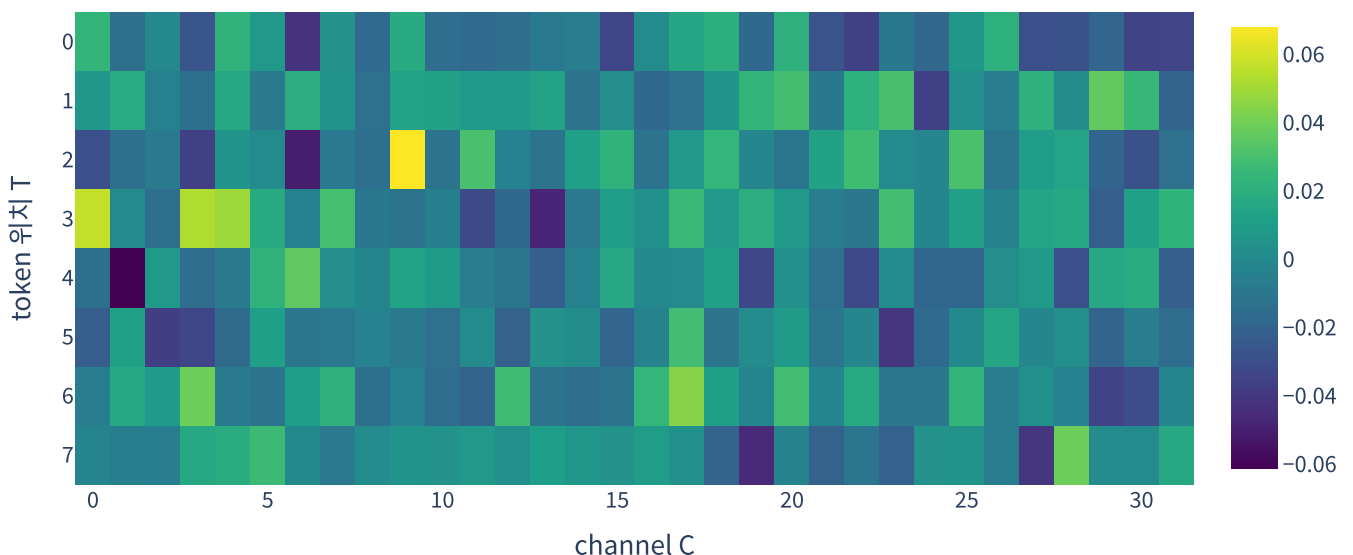
Embedding은 $[V, C]$ weight에서 ID에 해당하는 row를 고릅니다. Position embedding $[T, C]$ 는 batch 축으로 broadcast되어 $[B, T, C]$ 에 더해집니다.

```
words = 'compiler turns tensor operations into fast gpu kernels today'.split()
toy_vocabulary = {word: i for i, word in enumerate(dict.fromkeys(words))}
ids = [toy_vocabulary[word] for word in words]
tokens = torch.tensor([ids[:-1]], device=DEVICE)
targets = torch.tensor([ids[1:]], device=DEVICE)
config = GPTConfig(block_size=8, vocab_size=128, n_embd=32, n_head=4)
model = GPT(config).to(device=DEVICE, dtype=DTYPE).eval()
with torch.no_grad():
    token_embedding = model.transformer.wte(tokens)
    position_embedding = model.transformer.wpe(torch.arange(tokens.shape[1], device=DEVICE))
    hidden = token_embedding + position_embedding
viz.hmap_selector([token_embedding[0], position_embedding, hidden[0]],
                  ['token lookup', 'position lookup', 'sum / block input'],
                  'Embedding을 더해 block 입력 만들기', xaxis='channel C', yaxis='token 위치 T').show()
show_summary({'input IDs': tokens.tolist(), 'next-token targets': targets.tolist(),
             'token/position/sum shape': [list(t.shape) for t in [token_embedding, position_embedding,
             hidden]]})
```

그림 7 · token lookup

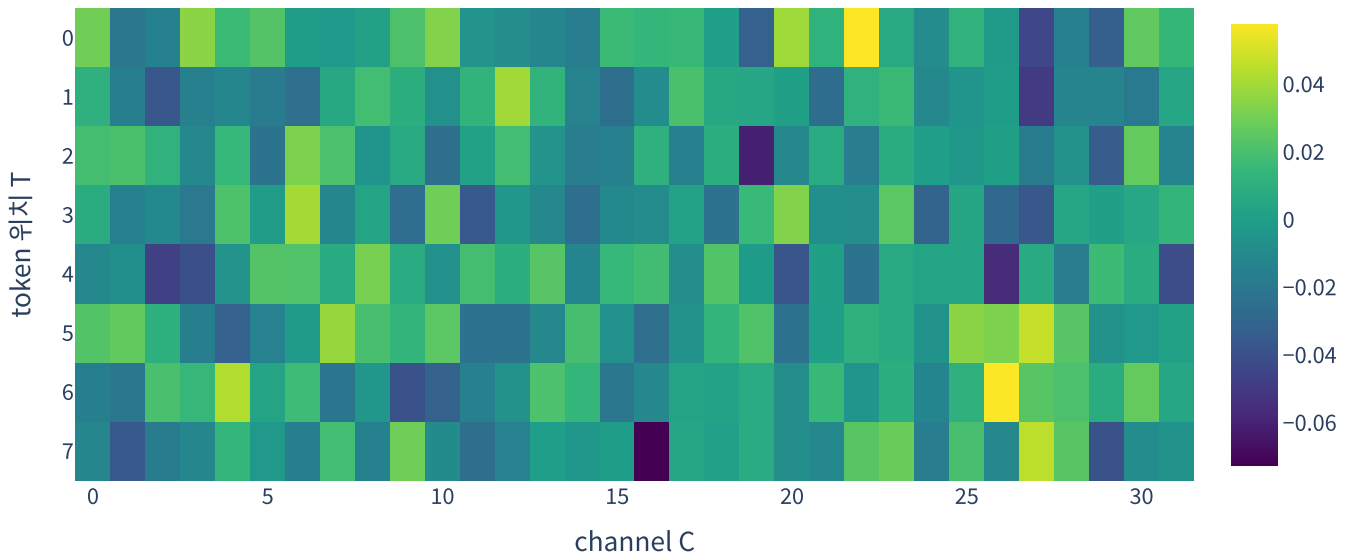
Embedding을 더해 block 입력 만들기

현재 실행에서 생성한 데이터



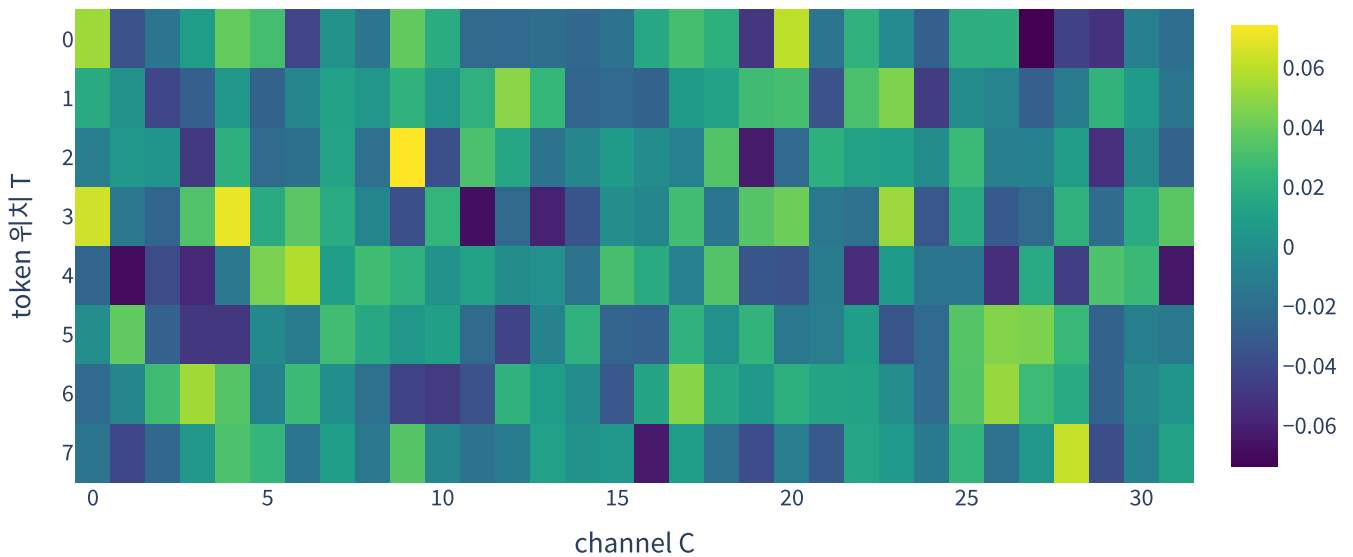
Embedding을 더해 block 입력 만들기

현재 실행에서 생성한 데이터



Embedding을 더해 block 입력 만들기

현재 실행에서 생성한 데이터



확인한 결과

input IDs	[[0, 1, 2, 3, 4, 5, 6, 7]]
next-token targets	[[1, 2, 3, 4, 5, 6, 7, 8]]
token/position/sum shape	[[1, 8, 32], [8, 32], [1, 8, 32]]

3.2 LayerNorm: 줄마다 C축을 모으고 다시 펼친다

각 token의 channel 방향으로 통계를 구합니다. Batch 전체를 평균 내지 않습니다.

$$\mu = \text{mean}_C(x), \quad \sigma^2 = \text{mean}_C((x - \mu)^2), \quad y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \gamma + \beta$$

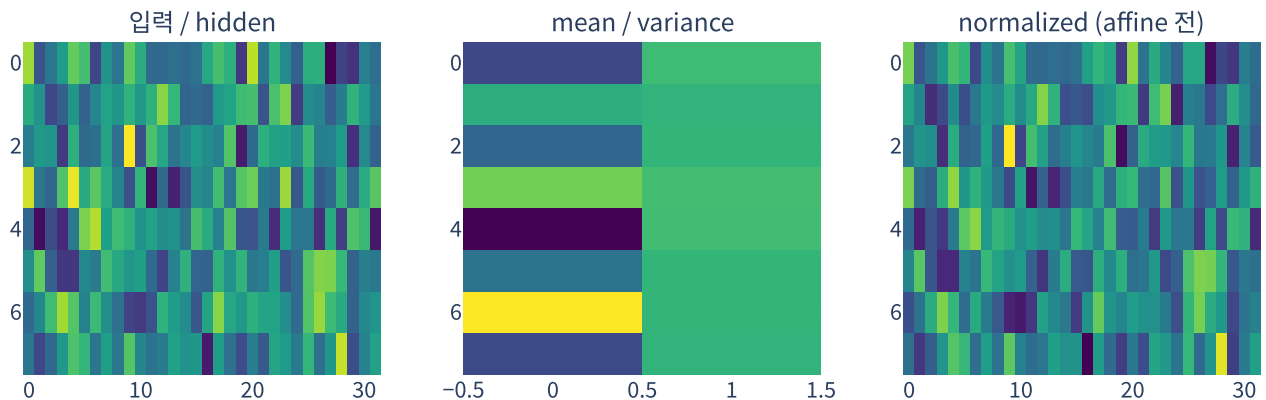
Mean/variance는 [B, T, 1], output은 [B, T, C]입니다. 분산은 C로 나누는 정의를 사용합니다. torch.var를 직접 쓴다면 correction/unbiased 설정을 일치시켜야 합니다. 아래 normalized 그림은 affine 이전 값입니다.

```
with torch.no_grad():
    mean = hidden.mean(-1, keepdim=True)
    centered = hidden - mean
    variance = centered.square().mean(-1, keepdim=True)
    normalized = centered * torch.rsqrt(variance + 1e-5)
    ln = model.transformer.h[0].ln_1
    ln_output = normalized * ln.weight + ln.bias
    torch.testing.assert_close(ln_output, ln(hidden))
fig = make_subplots(rows=1, cols=3, subplot_titles=['입력 / hidden', 'mean / variance', 'normalized (affine 전)'])
for col, matrix in [(1, hidden[0]), (2, torch.cat([mean[0], variance[0]], dim=-1)), (3, normalized[0])]:
    fig.add_trace(go.Heatmap(z=matrix.tolist(), colorscale='Viridis', showscale=False), row=1, col=col)
fig.update_yaxes(autorange='reversed')
viz.style(fig, 'Reduction 결과를 각 channel에 broadcast', '현재 CUDA 값 · manual/native LayerNorm assert_close 통과', 430).show()
```

그림 8

Reduction 결과를 각 channel에 broadcast

현재 CUDA 값 · manual/native LayerNorm assert_close 통과



3.3 Q, K, V와 causal attention

하나의 Linear가 [B, T, 3C]를 만들고 Q/K/V로 나눕니다. 각 결과를 [B, T, H, D]로 본 뒤 [B, H, T, D]로 transpose합니다.

$$S = QK^T/\sqrt{D}, \quad P = \text{softmax}(S + \text{causal mask}), \quad Y = PV$$

Softmax는 key 위치인 마지막 축을 따라 수행합니다. 미래 위치의 score는 -inf가 되어 확률이 0이 됩니다. 확률을 구한 뒤에 mask하는 순서와 구별하세요. 아래 선택 메뉴로 head와 계산 단계를 바꿔 보세요.

```
with torch.no_grad():
    attention = model.transformer.h[0].attn
    qkv = attention.c_attn(ln_output)
    q, k, value = [t.view(1, 8, 4, 8).transpose(1, 2) for t in qkv.split(32, dim=-1)]
```

```

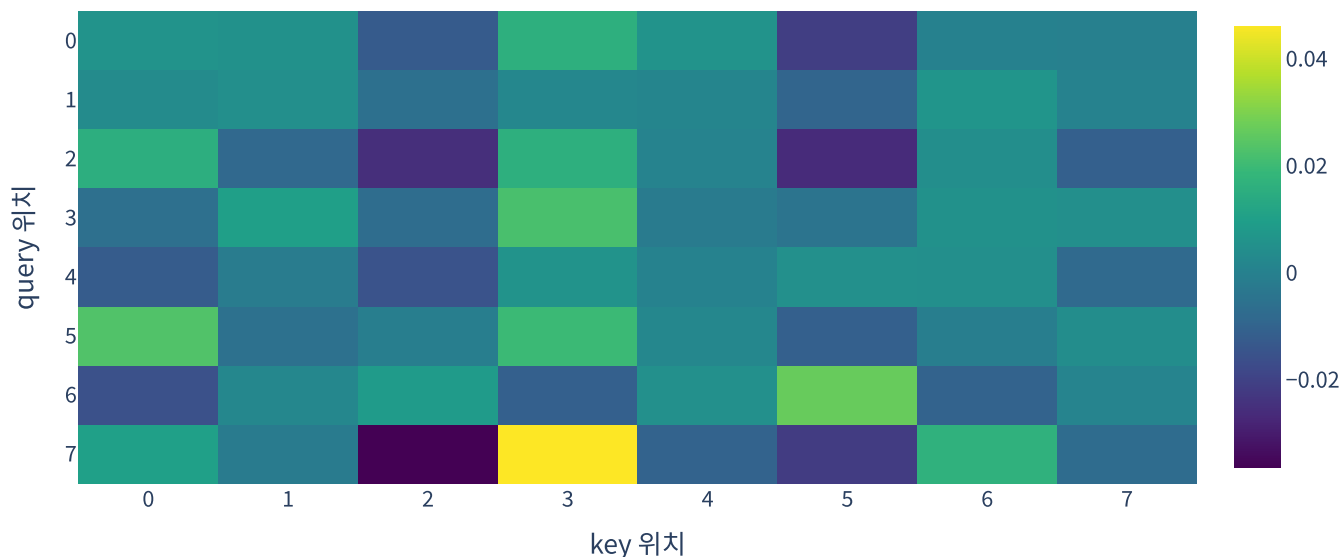
scores = q @ k.transpose(-2, -1) / math.sqrt(q.shape[-1])
causal = torch.tril(torch.ones(8, 8, device=DEVICE, dtype=torch.bool))
masked_scores = scores.masked_fill(~causal, float('-inf'))
probabilities = masked_scores.softmax(-1)
attended = probabilities @ value
sdpa = F.scaled_dot_product_attention(q, k, value, is_causal=True, dropout_p=0.0)
torch.testing.assert_close(attended, sdpa, rtol=1e-4, atol=1e-5)
torch.testing.assert_close(probabilities.sum(-1), torch.ones_like(probabilities.sum(-1)))
assert probabilities.masked_select(~causal).count_nonzero().item() == 0
matrices, labels = [], []
for head in range(4):
    for label, matrix in [('QK / sqrt(D)', scores), ('causal mask 적용', masked_scores), ('softmax
probability', probabilities)]:
        matrices.append(matrix[0, head]); labels.append(f'head {head} / {label}')
viz.heatmap_selector(matrices, labels, '같은 attention의 score → mask → probability').show()
show_summary({'Q/K/V': list(q.shape), 'score/P': list(scores.shape), 'P @ V': list(attended.shape),
            '결과': '미래 확률 0, row sum 1, manual/SDPA 결과 일치'})

```

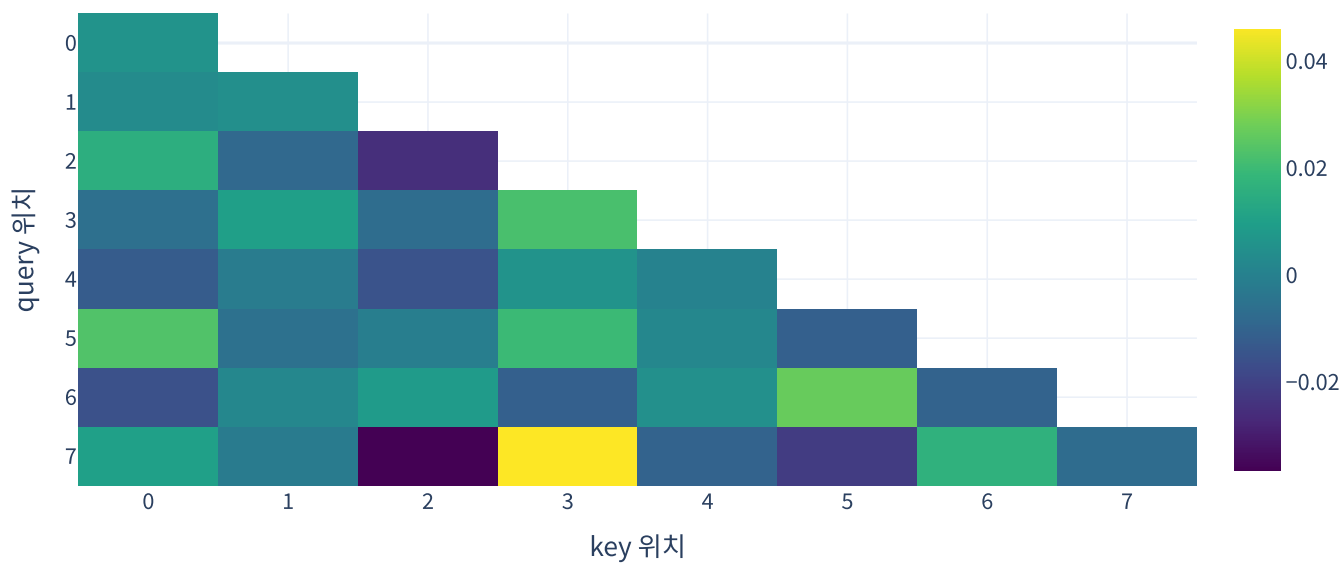
그림 9 · head 0 / QK / sqrt(D)

같은 attention의 score → mask → probability

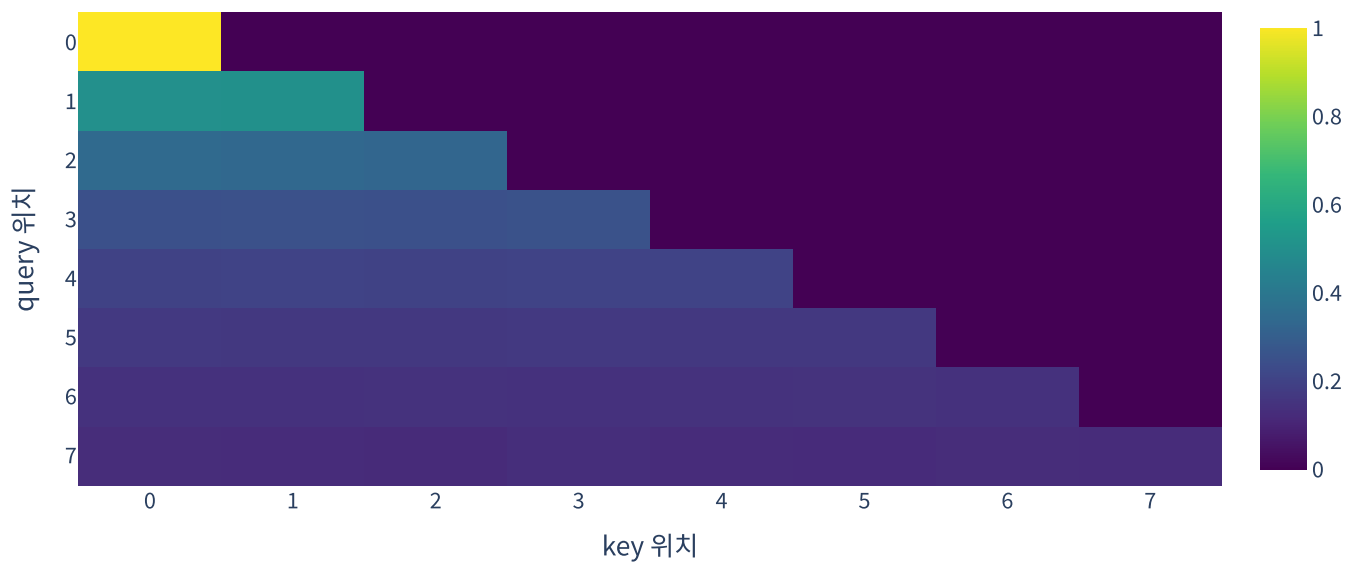
현재 실행에서 생성한 데이터



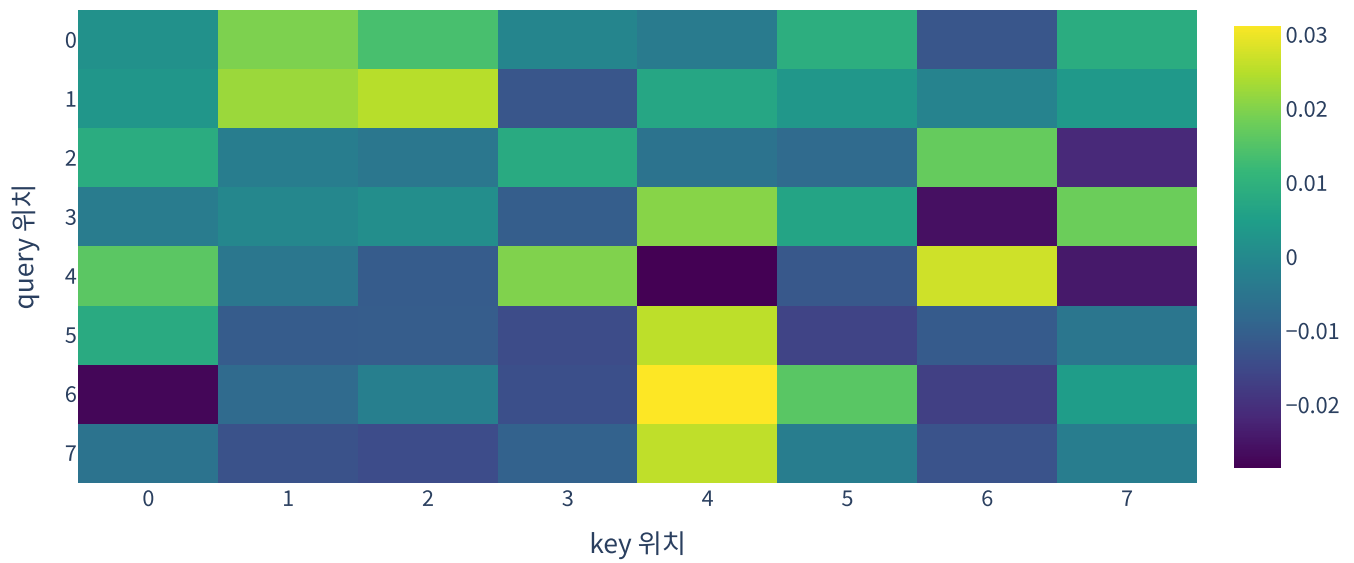
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
 현재 실행에서 생성한 데이터



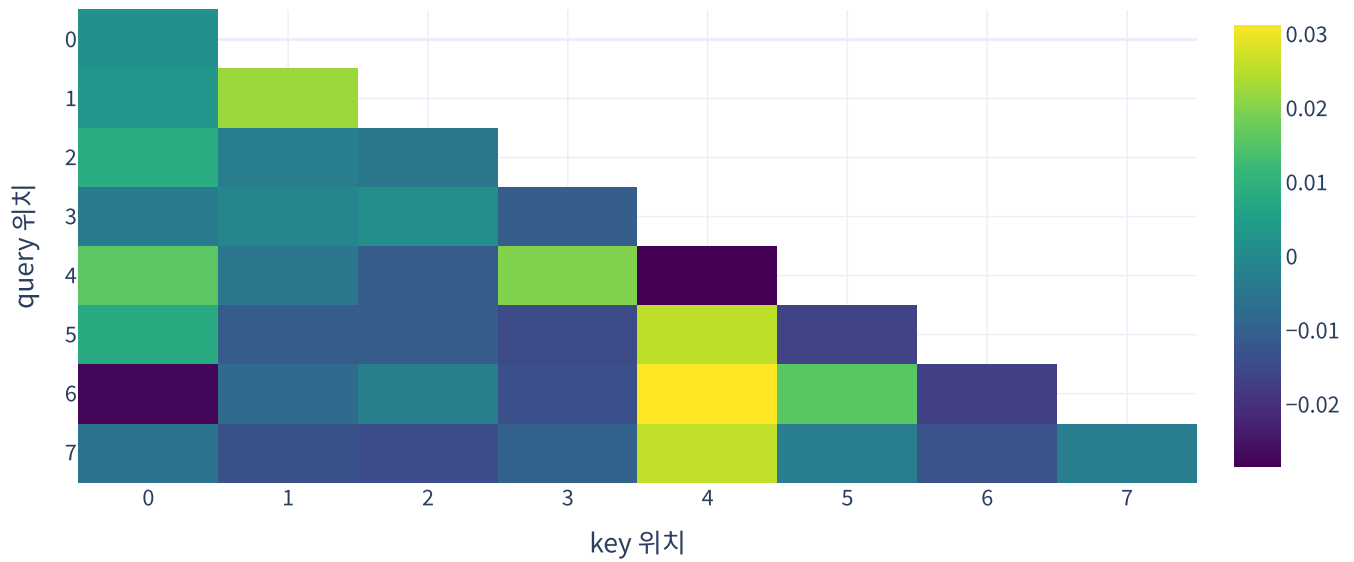
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
 현재 실행에서 생성한 데이터



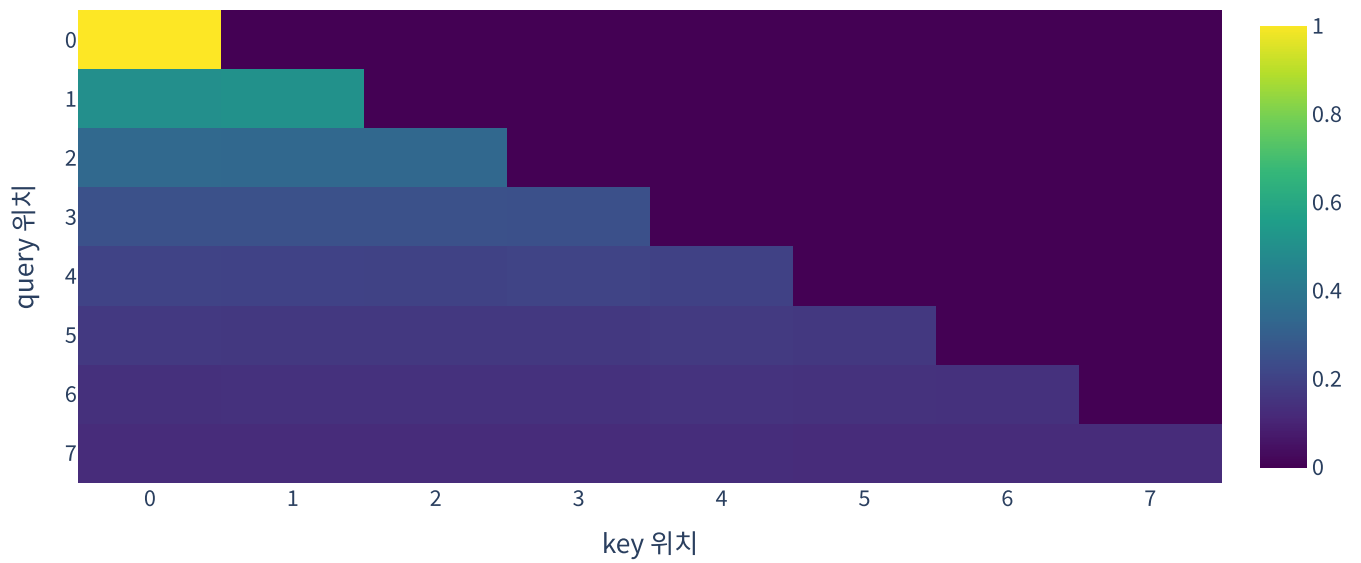
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
현재 실행에서 생성한 데이터



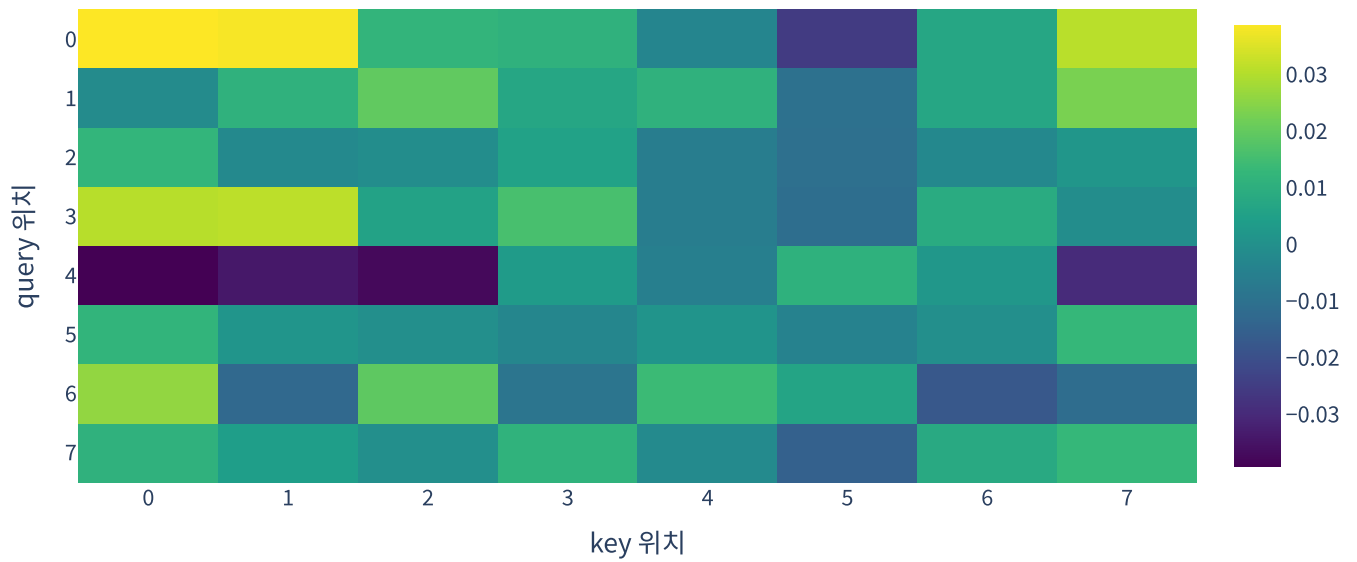
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
현재 실행에서 생성한 데이터



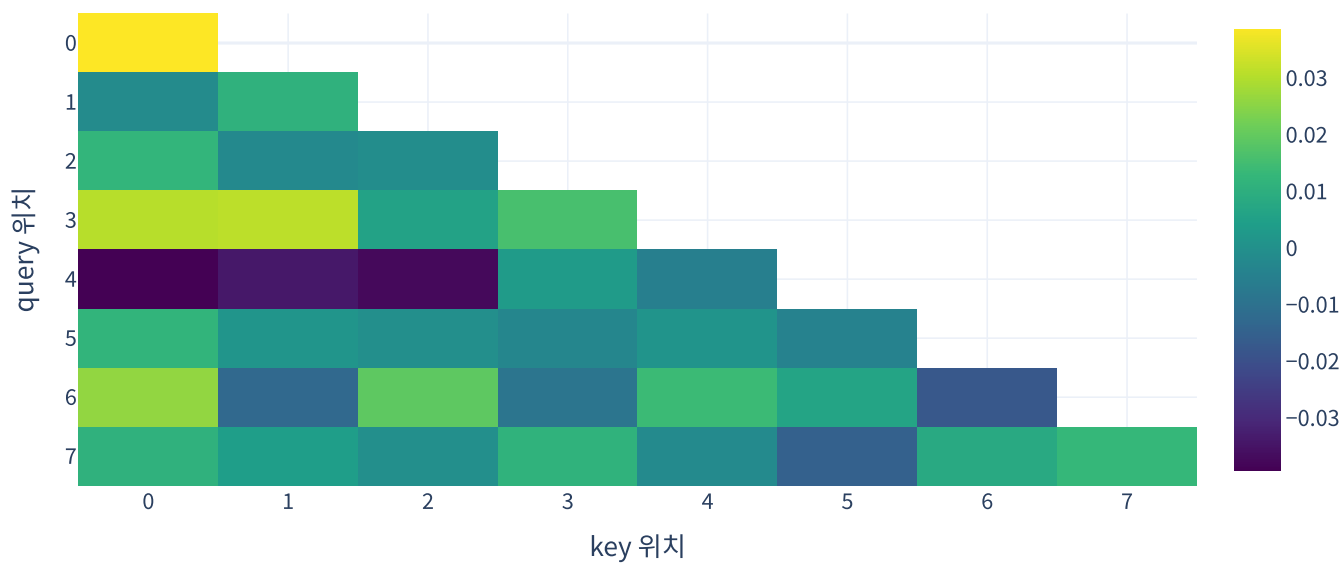
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
현재 실행에서 생성한 데이터



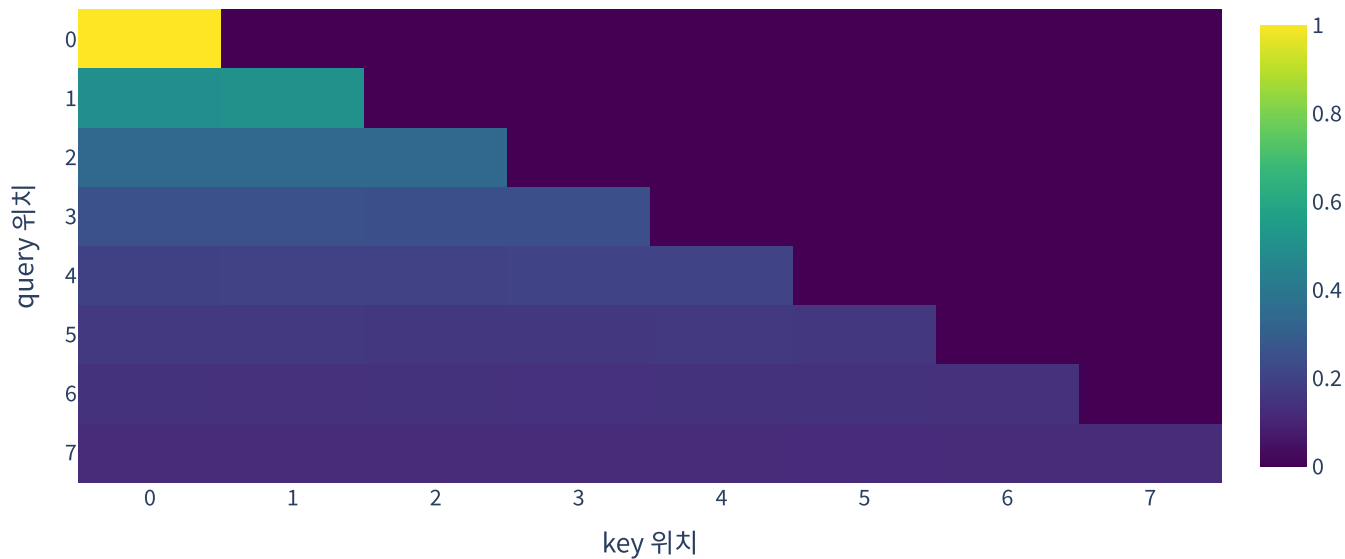
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
현재 실행에서 생성한 데이터



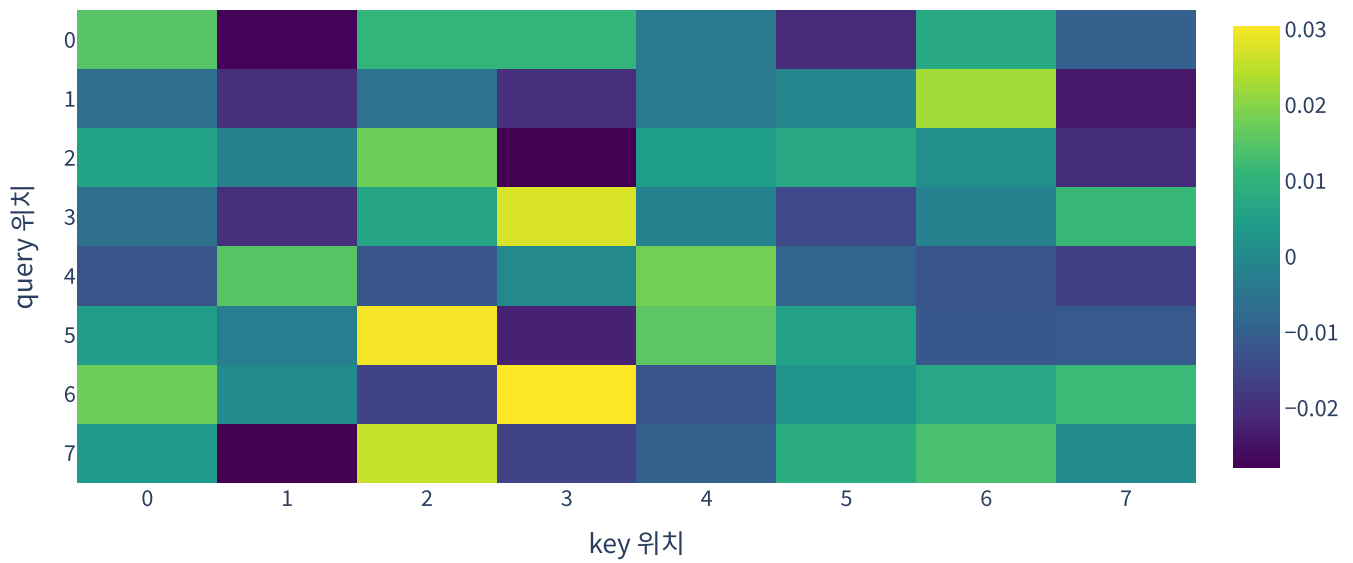
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
현재 실행에서 생성한 데이터



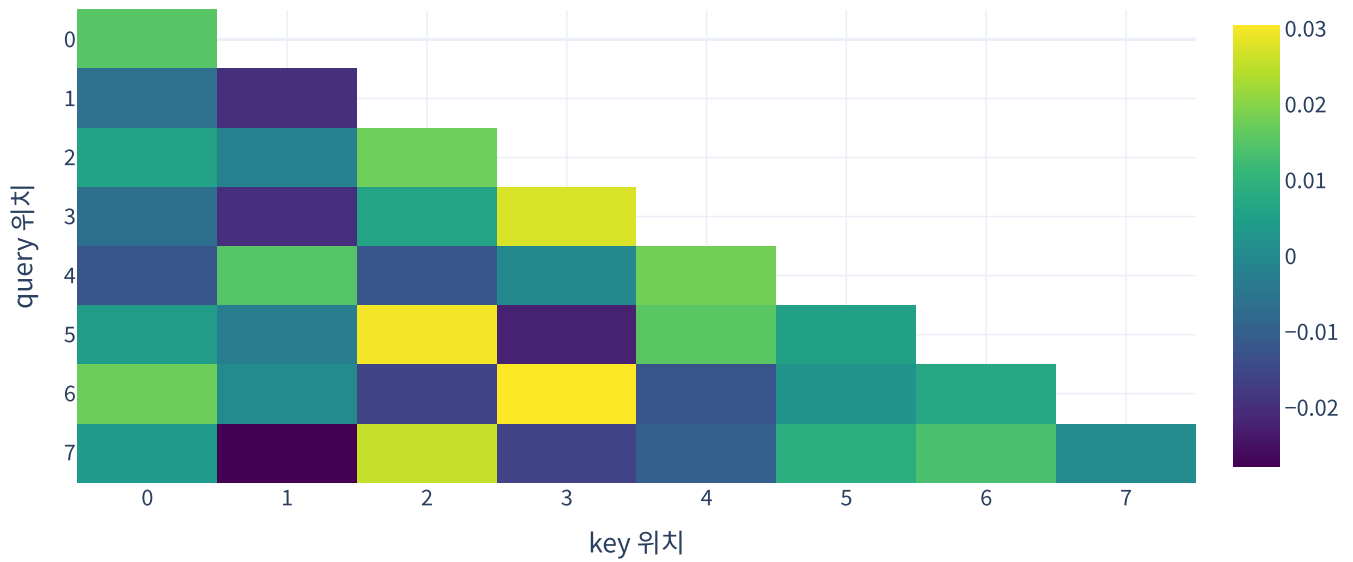
같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
현재 실행에서 생성한 데이터



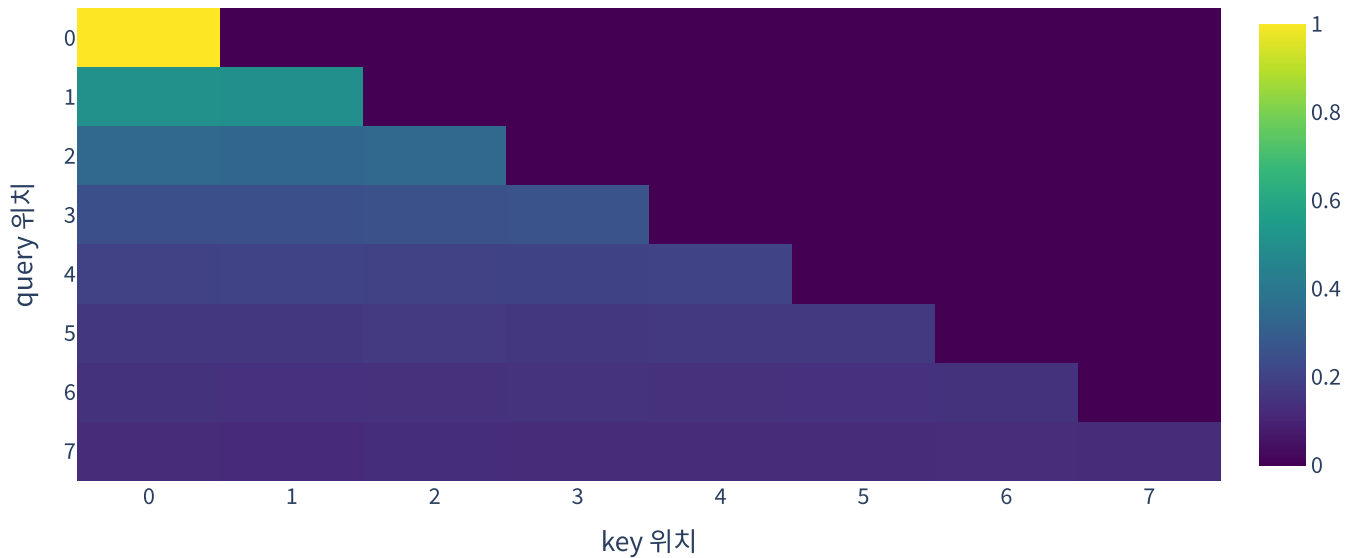
같은 attention의 score → mask → probability
현재 실행에서 생성한 데이터



같은 attention의 score → mask → probability
현재 실행에서 생성한 데이터



같은 attention의 score $\rightarrow$ mask $\rightarrow$ probability
현재 실행에서 생성한 데이터



확인한 결과

Q/K/V	[1, 4, 8, 8]
score/P	[1, 4, 8, 8]
P @ V	[1, 4, 8, 8]
결과	미래 확률 0, row sum 1, manual/SDPA 결과 일치

3.4 Residual, MLP, LM head

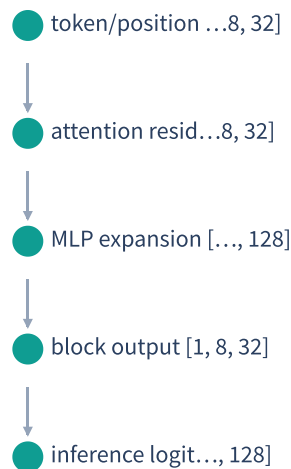
Block은 $x + \text{attention}(\text{LayerNorm}(x))$, 그다음 $x + \text{MLP}(\text{LayerNorm}(x))$ 입니다. Residual 때문에 원래 x 는 나중의 add까지 살아 있어야 합니다.

MLP는 $C \rightarrow 4C \rightarrow C$ 입니다. 이 모델의 `nn.GELU()`는 exact GELU입니다. 이후 compiler 추적용으로 따로 쓰는 `tanh` 근사 GELU와 구별합니다. LM head는 $C \rightarrow V$ 이며 embedding weight와 같은 Parameter를 공유합니다.

```
with torch.no_grad():
    after_attention = hidden + model.transformer.h[0].attn(model.transformer.h[0].ln_1(hidden))
    expanded = model.transformer.h[0].mlp_c_fc(model.transformer.h[0].ln_2(after_attention))
    block_output = model.transformer.h[0](hidden)
    inference_logits, _ = model(tokens)
    training_logits, training_loss = model(tokens, targets)
    shape_labels = [f'token/position sum {list(hidden.shape)}',
                    f'attention residual {list(after_attention.shape)}',
                    f'MLP expansion {list(expanded.shape)}',
                    f'block output {list(block_output.shape)}',
                    f'inference logits {list(inference_logits.shape)}']
    viz.concept(shape_labels, '모델의 큰 흐름 + 이번 CUDA 실행의 shape').show()
    show_summary({'embedding/LM-head weight 공유': model.transformer.wte.weight is model.lm_head.weight,
                 'inference logits': list(inference_logits.shape), 'training logits':
                 list(training_logits.shape),
                 'cross entropy': training_loss.item()})
    show_source(Block.forward)
```

모델의 큰 흐름 + 이번 CUDA 실행의 shape

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



확인한 결과

embedding/LM-head weight 공유	True
inference logits	[1, 1, 128]
training logits	[1, 8, 128]
cross entropy	4.849575996398926

▼ 실제 소스: /home/riverfog7/Workspace/mlsys_study/study02/model.py · forward

```
def forward(self, x):
    x = x + self.attn(self.ln_1(x))
    return x + self.mlp(self.ln_2(x))
```

확인해 보기 — eval() 상태에서 targets를 넘기면 마지막 token의 logits만 나오나요?

▼ 해설 펼치기

이 모델의 logits 분기는 targets 유무에 의해 결정됩니다. targets가 있으면 [B,T,V] 전체와 loss를 계산합니다. eval은 dropout 등의 동작, no_grad는 gradient 기록 여부를 결정합니다.

04. Eager 실행과 공통 GELU 예제

원문: §6 · pp. 26–29 · 목차로

4.1 한 API 호출이 실행되는 경로

Eager에서는 Python이 연산을 만날 때마다 PyTorch dispatcher로 보냅니다. ATen은 tensor 연산의 의미와 이름을 제공하는 계층입니다. Dispatcher는 device·dtype·layout·dispatch key를 보고 실제 구현을 선택합니다. Gradient 추적 중에는 Autograd 계층, autocast 안에서는 dtype 정책도 관여할 수 있습니다.

Python 호출, ATen operator, kernel은 서로 다른 단위입니다. `view`는 metadata 처리로 끝날 수 있고, 여러 연산은 compiler에 의뢰한 kernel로 합쳐질 수 있습니다.

```
viz.concept(['Python: torch.tanh(x)', 'ATen operator / schema',
            'Dispatcher: device · dtype · dispatch keys',
            'Autograd / autocast 등 활성 계층', 'CUDA backend implementation', 'CUDA kernel launch'],
            '익숙한 API 아래의 실행 계층').show()
```

그림 11

익숙한 API 아래의 실행 계층

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



4.2 끝까지 추적할 하나의 수식

$$\text{GELU}_{\tanh}(x) = \frac{x}{2} \left[1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right]$$

Pointwise이므로 각 원소를 독립적으로 계산합니다. Eager에서는 `pow·mul·add·tanh` 사이에 여러 dispatch와 중간 tensor가 생길 수 있습니다. 하나의 kernel 안에서 계산하면 입력을 읽고 최종 결과를 저장하는 사이에 중간 scalar 계산을 이어갈 수 있습니다.

곡선이 비슷하다는 이유로 `tanh approximation`과 `exact GELU`를 동일한 식으로 취급하지 않습니다. 아래에서는 값과 gradient의 차이를 직접 계산합니다.

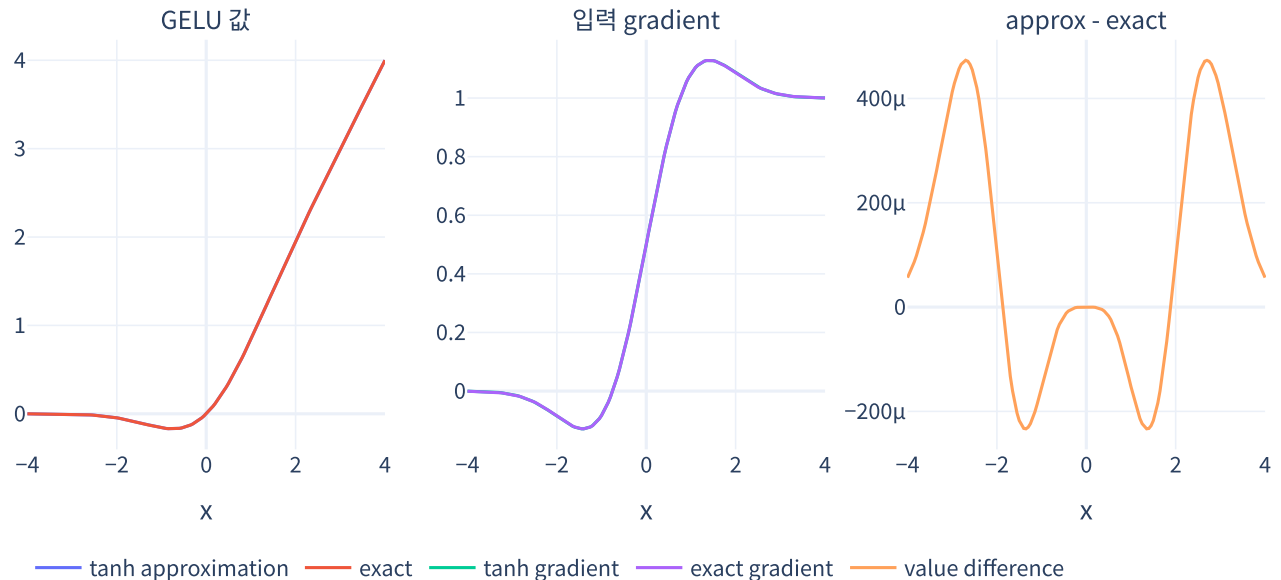
```
curve_x = torch.linspace(-4, 4, 161, device=DEVICE, requires_grad=True)
approx = ManualGELU()(curve_x)
exact = F.gelu(curve_x, approximate='none')
grad_approx = torch.autograd.grad(approx.sum(), curve_x, retain_graph=True)[0]
grad_exact = torch.autograd.grad(exact.sum(), curve_x)[0]
fig = make_subplots(rows=1, cols=3, subplot_titles=['GELU 값', '입력 gradient', 'approx - exact'])
for label, values, col in [('tanh approximation', approx, 1), ('exact', exact, 1),
                           ('tanh gradient', grad_approx, 2), ('exact gradient', grad_exact, 2),
                           ('value difference', approx-exact, 3)]:
    fig.add_scatter(x=curve_x.detach().tolist(), y=values.detach().tolist(), name=label, row=1, col=col)
```

```
fig.update_xaxes(title='x')
viz.style(fig, '같은 이름의 GELU도 수식과 decomposition은 다를 수 있다', '현재 CUDA 계산과 autograd', 480).show()
show_source(ManualGELU.forward)
```

그림 12

같은 이름의 GELU도 수식과 decomposition은 다를 수 있다

현재 CUDA 계산과 autograd



▼ 실제 소스: /home/riverfog7/Workspace/mlsys_study/study02/examples.py · forward

```
def forward(self, x):
    return 0.5 * x * (1 + torch.tanh(math.sqrt(2 / math.pi) * (x + 0.044715 * x.pow(3))))
```

확인해 보기 — 여러 pointwise 연산을 한 번에 계산하면 무엇을 줄일 수 있나요?

▼ 해설 펼치기

중간 tensor의 global-memory write/read와 allocation, 여러 kernel launch를 줄일 수 있습니다. 실제로 어디서 합쳐졌는지는 FX, pre/post IR, generated code를 함께 확인해야 합니다.

05. Python bytecode에서 symbolic 실행으로

원문: §7.1–7.8 · pp. 30–33 · 목차로

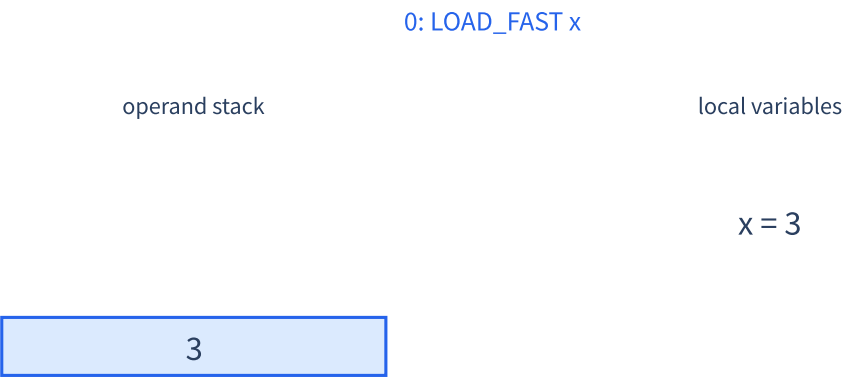
5.1 Source, code object, frame

CPython은 Python source를 bytecode로 바꾸고 interpreter가 그 명령을 실행합니다. 함수의 `__code__`에는 명령, 상수, local 이름 등이 있습니다. **Frame**은 그 함수를 실행 중인 상태로, code object·locals·globals·현재 명령 위치·operand stack 등을 연결합니다.

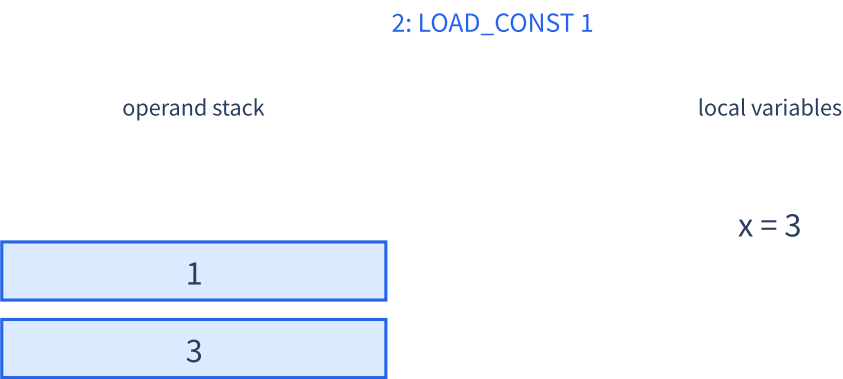
`dis`는 bytecode를 사람이 읽는 opcode 이름으로 보여줍니다. 아래 실험 함수는 실제 `.py` 파일에 정의되어 있습니다. Slider는 2 함수의 실제 `dis` 명령을 작은 교육용 stack 모델로 해석합니다. CPython 내부 stack을 직접 촬영한 결과는 아닙니다.

```
python_run = experiment('python_runtime')
viz.bytecode_steps(python_run).show()
show_summary({'실제 함수 반환값': python_run['returned'], 'tiny의 local 이름': python_run['code_object']
['varnames'],
'관찰한 frame': python_run['frame']['function'], 'frame 파일': python_run['frame']
['filename']})
show_artifact(python_run, 'raw/tiny_bytecode.txt')
```

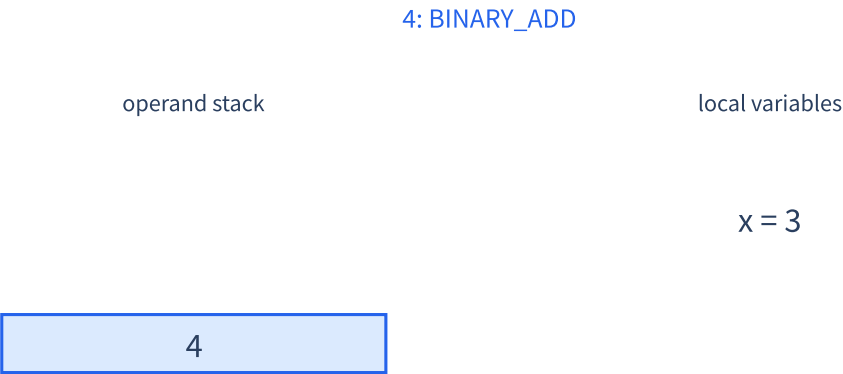
실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython value stack 실측 아님



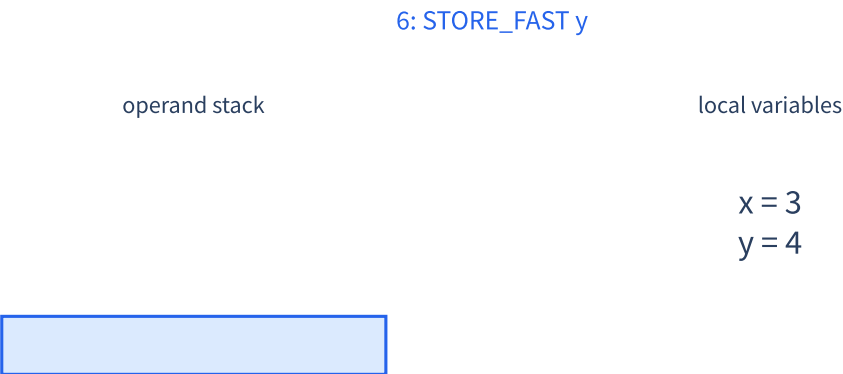
실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython value stack 실측 아님



실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython value stack 실측 아님

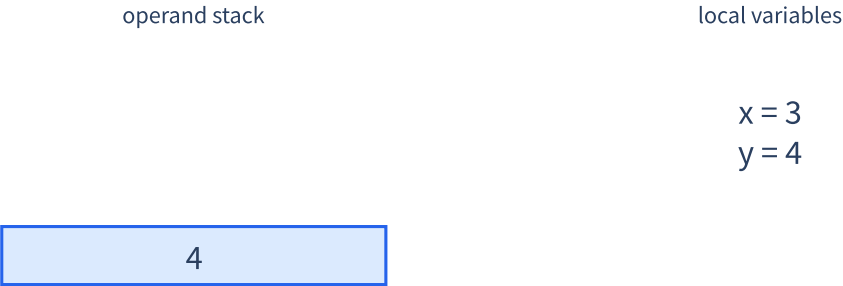


실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython value stack 실측 아님



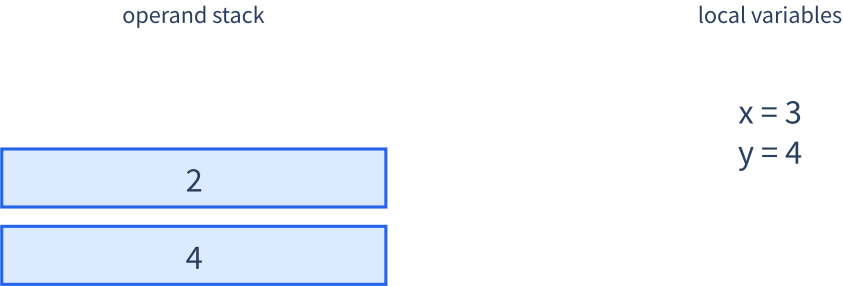
실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython value stack 실측 아님

8: `LOAD_FAST y`



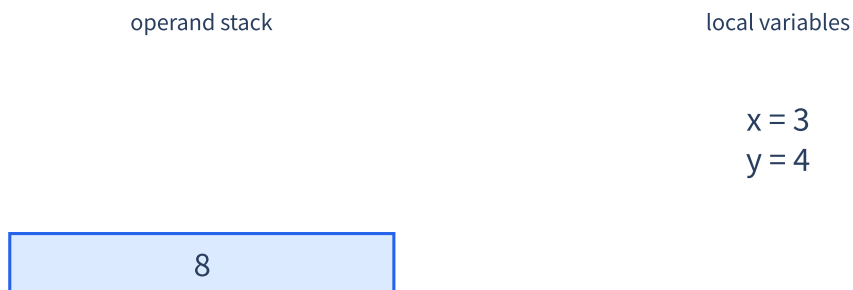
실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython value stack 실측 아님

10: `LOAD_CONST 2`



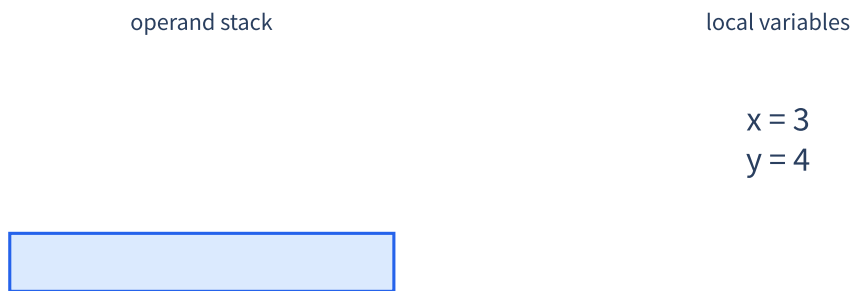
실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython `value stack` 실측 아님

12: BINARY_MULTIPLY



실제 `dis` 명령을 따라가는 작은 `stack` 모델
실제 `dis` 명령을 해석한 교육용 모델; CPython `value stack` 실측 아님

14: RETURN_VALUE



확인한 결과

실제 함수 반환값	8
tiny의 local 이름	['x', 'weight', 'hidden']
관찰한 frame	run
frame 파일	/home/riverfog7/Workspace/mlsys_study/study02/experiments/python_runtime.py

▼ 이번 실행의 원본: raw/tiny_bytecode.txt

```

1  12      0 LOAD_FAST           0 (x)
2          2 LOAD_FAST           1 (weight)
3          4 BINARY_MATRIX_MULTIPLY
4          6 STORE_FAST          2 (hidden)
5
6  13      8 LOAD_GLOBAL          0 (torch)
7          10 LOAD_METHOD          1 (relu)
8          12 LOAD_FAST           2 (hidden)
9          14 CALL_METHOD         1
10         16 RETURN_VALUE

```

5.2 Python 연산자와 GPU 연산은 다른 계층

표현	이 계층에서 하는 일
BINARY_MATRIX_MULTIPLY 또는 BINARY_OP @	CPython의 연산자 명령. 이름은 Python 버전에 따라 다름
Tensor.__matmul__	Python data model의 @ 의미를 tensor 연산에 연결
ATen matmul/mm	Tensor 연산의 schema와 수학적 의미
Dispatcher + backend	Device와 실행 상태에 맞는 구현 선택
CUDA GEMM kernel	GPU가 실제로 실행하는 계산

len, range는 built-in callable이고 __add__, __iter__는 data model method입니다. CPython intrinsic은 interpreter가 특별히 처리하는 동작, GPU intrinsic은 GPU backend가 특별히 처리하는 동작입니다. 이름이 비슷해도 가트는 계층이 아닙니다.

5.3 Dynamo는 Python 실행의 어디에 끼어드는가?

PEP 523의 frame evaluation hook은 frame 실행에 개입할 수 있게 합니다. Dynamo는 지원하는 Python bytecode를 **symbolic**하게 해석합니다. Tensor 대신 계산의 결과를 나타내는 추적 객체를 stack에 두고, 연산을 만날 때 graph node를 추가합니다.

객체	대신 표현하는 것
VariableTracker	Python 값의 종류와 symbolic 실행 상태. TensorVariable·ConstantVariable 등이 있음
FX Proxy	이미 만든 graph node의 결과
FakeTensor	큰 실제 storage 계산 없이 tensor의 shape·stride·dtype·device 동작을 추적
SymInt	하나의 상수로 고정하지 않은 정수 dimension

상수 $\sqrt{2/\pi}$ 는 Python에서 계산해 graph의 상수가 될 수 있습니다. Tensor의 `x.pow(3)`은 node가 됩니다. Metadata만으로 `[B,T,C] @ [C,4C]`의 결과 shape가 `[B,T,4C]`임을 알 수 있지만 실제 출력 값은 알 수 없습니다.

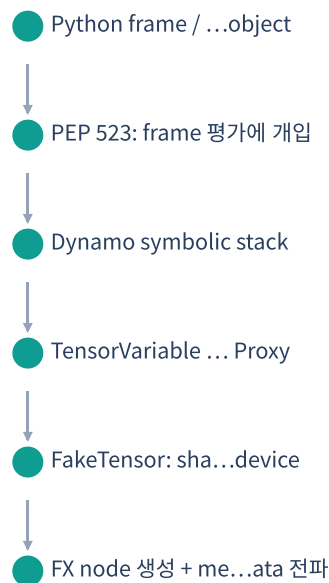
```

viz.concept(['Python frame / code object', 'PEP 523: frame 평가에 개입',
            'Dynamo symbolic stack', 'TensorVariable / FX Proxy',
            'FakeTensor: shape · stride · dtype · device', 'FX node 생성 + metadata 전파'],
            '실제 값을 계산하는 대신 계산의 의미를 기록하기').show()

```

실제 값을 계산하는 대신 계산의 의미를 기록하기

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



확인해 보기 — FakeTensor의 device가 cuda로 보이면 실제 GEMM을 수행했다는 뜻인가요?

▼ 해설 펼치기

아닙니다. CUDA tensor가 가질 metadata를 추적할 수 있다는 뜻입니다. 실제 데이터 계산을 수행했다는 증거는 실행 결과와 profiler입니다. 다음 절에서 실제 node metadata의 Python type도 확인합니다.

06. Dynamo: graph, guards, specialization

원문: §3.3, §7.9–7.16 · pp. 33–39 · 목차로

6.1 Custom backend가 받는 실제 FX graph

Backend callable은 Dynamo가 포착한 GraphModule과 example input을 받습니다. 이번 custom backend는 graph를 저장하고 gm.forward를 반환합니다. 따라서 **Dynamo capture**를 분리해서 관찰합니다. 최적화된 Triton kernel을 만든 단계는 아직 아닙니다.

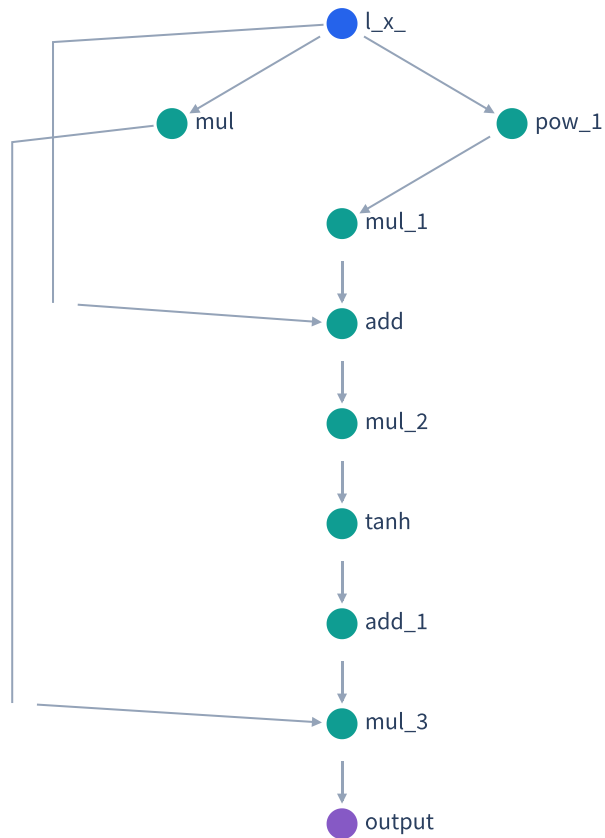
아래 node를 hover하면 operator target, dtype·shape·stride·device를 볼 수 있습니다. Placeholder는 입력 자리, output은 반환값입니다. 동일한 입력에서 갈라지는 여러 edge를 찾아보세요.

```

dynamo_run = experiment('dynamo')
viz.dag(dynamo_run['gelu_graph'], 'GELU의 실제 Dynamo FX').show()
show_summary({'입력': dynamo_run['input'], '출력': dynamo_run['output'],
              'tensor metadata 예': next(n['metadata'] for n in dynamo_run['gelu_graph']['nodes'] if
n['metadata'])})
show_artifact(dynamo_run, 'raw/dynamo_0.py')
  
```

GELU의 실제 Dynamo FX

현재 capture · 화살표=값의 의존성, 시간축 아님



● placeholder ● call_function ● call_method ● output

확인한 결과

입력 [-2.0, -1.0, 0.0, 1.0, 2.0]

출력 [-0.04540228843688965, -0.15880799293518066, 0.0, 0.8411920070648193, 1.9545977115631104]

tensor metadata 예 {'python_type': 'FakeTensor', 'shape': [5], 'stride': [1], 'dtype': 'torch.float32', 'device': 'cuda:0', 'requires_grad': False}

▼ 이번 실행의 원본: raw/dynamo_0.py

```

1
2
3
4 def forward(self, L_x_ : torch.Tensor):
5     l_x_ = L_x_
6     mul = 0.5 * l_x_
7     pow_1 = l_x_.pow(3)

```

```

8     mul_1 = 0.044715 * pow_1; pow_1 = None
9     add = l_x_ + mul_1; l_x_ = mul_1 = None
10    mul_2 = 0.7978845608028654 * add; add = None
11    tanh = torch.tanh(mul_2); mul_2 = None
12    add_1 = 1 + tanh; tanh = None
13    mul_3 = mul * add_1; mul = add_1 = None
14    return (mul_3,)
15

```

6.2 Compile 옵션과 guard

- **backend**: capture한 graph를 어디에 넘길지 정합니다. Custom backend로 graph만 관찰한 결과와 Inductor codegen을 구별합니다.
- **fullgraph=True**: graph break가 필요하면 오류로 드러냅니다. “GPU kernel 하나”를 뜻하지 않습니다.
- **dynamic=False**: 첫 실행에서는 input shape를 고정합니다. **dynamic=True**는 symbolic dimension 사용을 시도하지만 guard가 없어지는 것은 아닙니다.

Guard는 shape뿐만 아니라 dtype·device·stride·module/global 상태 등의 가정을 확인합니다. `code_list`에 모든 검사가 문자열로 나타나는 것은 아닙니다.

같은 함수에 batch 크기 2 → 2 → 5 → 7을 줍니다. Y축은 누적 **backend capture** 횟수입니다. 선이 올라갔으면 새 graph가 만들어졌습니다. 정확한 횟수는 이번 실행에서 측정합니다.

```

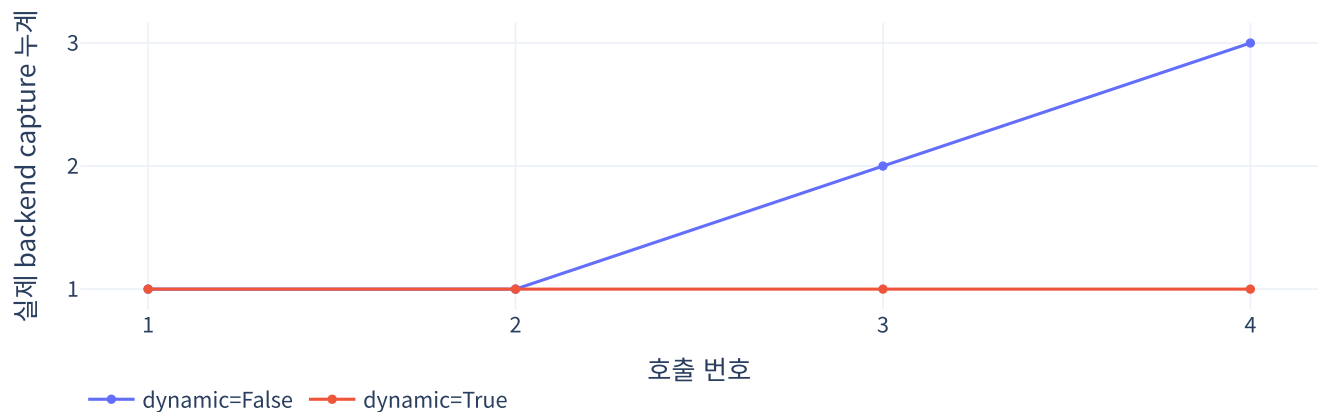
fig = go.Figure()
for dynamic in [False, True]:
    points = [p for p in dynamo_run['shape_runs'] if p['dynamic'] == dynamic]
    fig.add_scatter(x=[p['call'] for p in points], y=[p['captures_so_far'] for p in points],
                   customdata=[p['batch'] for p in points], mode='lines+markers', name=f'dynamic={
{dynamic}}',
                   hovertemplate='%{x}<br>batch=%{customdata}<br>누적 capture=%{y}<extra></extra>')
fig.update_xaxes(title='호출 번호', dtick=1); fig.update_yaxes(title='실제 backend capture 누계', dtick=1)
viz.style(fig, 'Guard와 shape: 언제 새 graph가 필요한가?', '현재 CUDA 입력의 크기를 바꾼 실측', 400).show()
guard_types = {}
for guard in dynamo_run['guards']:
    for kind in guard['types'] or ['내부 조건 / 별도 문자열 없음']:
        guard_types[kind] = guard_types.get(kind, 0) + 1
fig = go.Figure(go.Bar(x=list(guard_types), y=list(guard_types.values()), marker_color=viz.BLUE))
fig.update_yaxes(title='대표 explain 결과의 guard 수')
viz.style(fig, 'Guard가 검사하는 가정의 종류', '현재 torch._dynamo.explain', 350).show()
details('Guard 원문 구조 보기', dynamo_run['guards'])

```

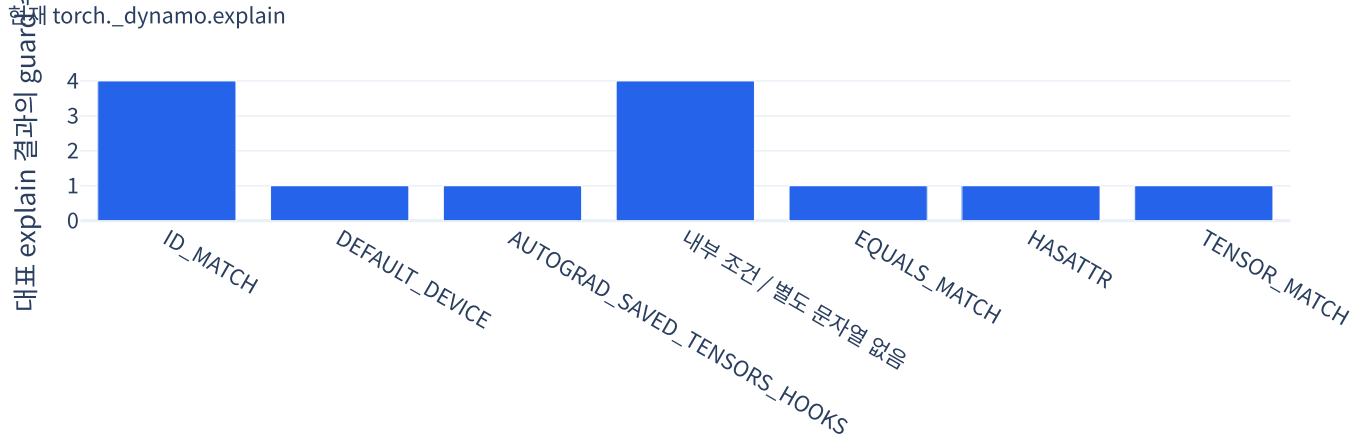
그림 16

Guard와 shape: 언제 새 graph가 필요한가?

현재 CUDA 입력의 크기를 바꾼 실측



Guard가 검사하는 가정의 종류



▼ Guard 원문 구조 보기

```
[{'name': "G['math']", 'source': 'GuardSource.GLOBAL', 'types': ['ID_MATCH'], 'conditions':
["__check_obj_id(G['math'], 125814651196736)"]}, {'name': '', 'source': 'GuardSource.GLOBAL', 'types':
['DEFAULT_DEVICE'], 'conditions': ['utils_device.CURRENT_DEVICE == None']}, {'name': '', 'source':
'GuardSource.GLOBAL', 'types': ['AUTOGRAD_SAVED_TENSORS_HOOKS'], 'conditions':
['torch._functorch.aot_autograd.utils.top_saved_tensors_hooks ids == None']}, {'name': '', 'source':
'GuardSource.GLOBAL', 'types': None, 'conditions': None}, {'name': "G['math'].pi", 'source':
'GuardSource.GLOBAL', 'types': ['EQUALS_MATCH'], 'conditions': ["G['math'].pi == 3.141592653589793"]},
{'name': "G['math'].sqrt", 'source': 'GuardSource.GLOBAL', 'types': ['ID_MATCH'], 'conditions':
["__check_obj_id(G['math'].sqrt, 125814651208624)"]}, {'name': '', 'source': 'GuardSource.GLOBAL', 'types':
None, 'conditions': None}, {'name': "L['x'].pow", 'source': 'GuardSource.LOCAL', 'types': ['HASATTR'],
'conditions': ["hasattr(L['x'], 'pow')"]}, {'name': "L['x']", 'source': 'GuardSource.LOCAL', 'types':
['TENSOR_MATCH'], 'conditions': ["hasattr(L['x'], '_dynamo_dynamic_indices') == False"]}, {'name':
"G['torch'].tanh", 'source': 'GuardSource.GLOBAL', 'types': ['ID_MATCH'], 'conditions':
["__check_obj_id(G['torch'].tanh, 125814530082992)"]}, {'name': "G['torch']", 'source':
'GuardSource.GLOBAL', 'types': ['ID_MATCH'], 'conditions': ["__check_obj_id(G['torch'], 125814649317472)"]},
{'name': '', 'source': 'GuardSource.GLOBAL', 'types': None, 'conditions': None}, {'name': '', 'source':
'GuardSource.SHAPE_ENV', 'types': None, 'conditions': None}]
```

6.3 Graph break와 guard 실패는 서로 다른 일

Guard 실패는 기존 compiled 경로의 가정이 새 호출에 맞지 않는 경우입니다. **Graph break**는 capture 도중 안전하게 이어갈 수 없어 graph를 나누는 경계입니다. Python 실행을 거친 뒤 다른 compiled region으로 재개할 수 있습니다.

명시적 `torch._dynamo.graph_break()`를 넣은 작은 함수가 실제로 나뉜 두 graph를 아래에서 봅니다. Data-dependent `i x.sum().item() > 0` 사례도 관찰하되 그 정확한 진단은 버전과 설정을 따릅니다.

```
for i, graph in enumerate(dynamo_run['break_graphs']):
    viz.dag(graph, f'Graph break로 나뉜 region {i}').show()
show_summary({'실제 graph 수': dynamo_run['graph_count'], 'break 수': dynamo_run['break_count'],
             'data-dependent 사례 graph 수': dynamo_run['data_dependent']['graphs']})
details('예상한 fullgraph 오류', dynamo_run['fullgraph_expected_error'])
details('Data-dependent branch의 실제 진단', dynamo_run['data_dependent']['reasons'])
```

그림 18

Graph break로 나뉜 region 0
현재 capture · 화살표=값의 의존성, 시간축 아님

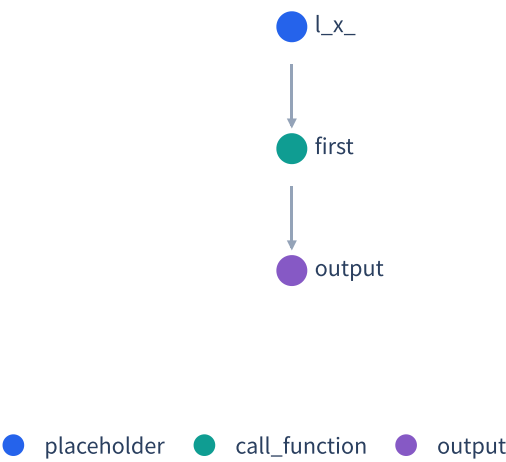
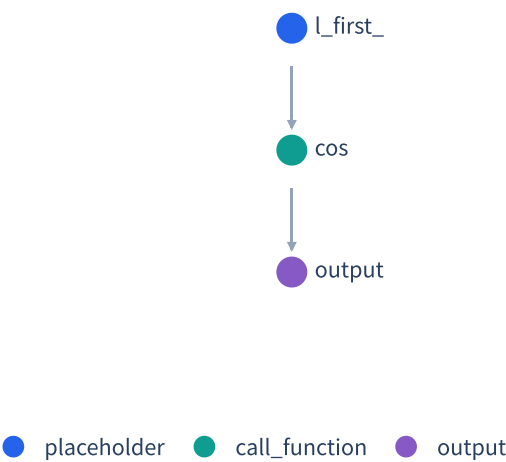


그림 19

Graph break로 나뉜 region 1
현재 capture · 화살표=값의 의존성, 시간축 아님



확인한 결과

실제 graph 수	2
break 수	1
data-dependent 사례 graph 수	2

▼ 예상한 fullgraph 오류

```
Call to `torch._dynamo.graph_break()`  
Explanation: User-inserted graph break. Message: None  
Hint: Remove the `torch._dynamo.graph_break()` call.  
  
Developer debug context: Called `torch._dynamo.graph_break()` with args `[]`, kwargs `{}`
```

from user code:

```
File "/home/riverfog7/Workspace/mlsys_study/study02/examples.py", line 23, in two_regions
    torch._dynamo.graph_break()
```

Set TORCHDYNAMO_VERBOSE=1 for the internal stack trace (please do this especially if you're reporting a bug to PyTorch). For even more developer context, set TORCH_LOGS="+dynamo"

▼ Data-dependent branch의 실제 진단

```
['Unsupported Tensor.item() call with capture_scalar_outputs=False\n Explanation: Dynamo does not support
tracing `Tensor.item()` with config.capture_scalar_outputs=False.\n Hint: Set
`torch._dynamo.config.capture_scalar_outputs = True` or `export TORCHDYNAMO_CAPTURE_SCALAR_OUTPUTS=1` to
include these operations in the captured graph.\n\n Developer debug context: call_method TensorVariable()
item () {}\n']
```

6.4 Python 조건과 loop는 어디로 갔을까?

targets is None처럼 현재 호출에서 알 수 있는 Python 조건은 한쪽 경로로 specialize할 수 있습니다. 고정 길이 ModuleList의 loop는 연산이 펼쳐진 graph가 될 수 있습니다. 아래 메뉴/그림은 실제 두 branch와 작은 module sequence입니다.

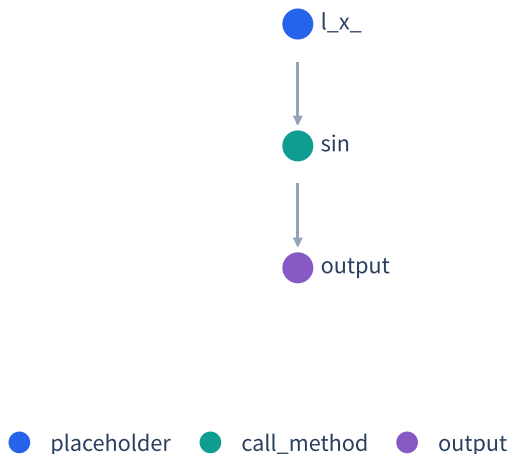
Dynamo capture는 Python 실행에서 FX를 만드는 frontend입니다. **CUDA Graph capture**는 **GPU launch sequence**를 기록해 재생하는 **runtime** 기능입니다. 둘을 같은 의미로 사용하지 않습니다.

```
for i, graph in enumerate(dynamo_run['branch_graphs']):
    viz.dag(graph, f'Python extra=None / extra=1.0의 specialization {i}').show()
viz.dag(dynamo_run['loop_graph'], '작은 module sequence가 펼쳐진 FX').show()
```

그림 20

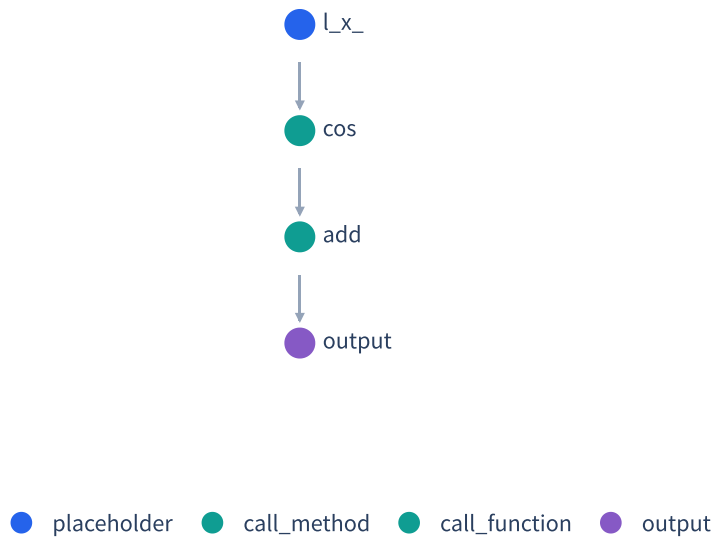
Python extra=None / extra=1.0의 specialization 0

현재 capture · 화살표=값의 의존성, 시간축 아님



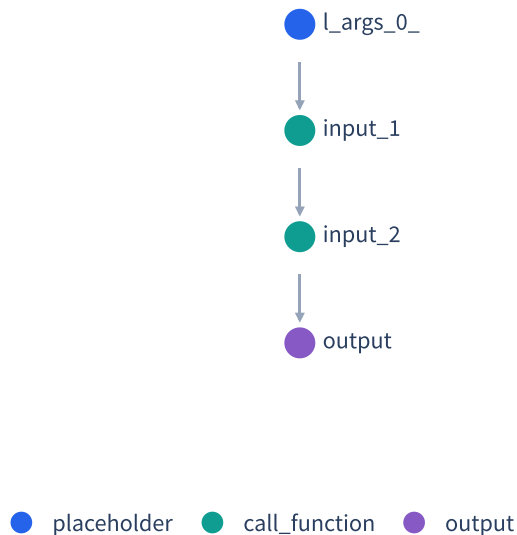
Python extra=None / extra=1.0의 specialization 1

현재 capture · 화살표=값의 의존성, 시간축 아님



작은 module sequence가 펼쳐진 FX

현재 capture · 화살표=값의 의존성, 시간축 아님



확인해 보기 — 같은 shape의 새 tensor 값이 들어올 때마다 compile해야 하나요?

▼ 해설 펼치기

일반적인 tensor 연산 graph는 입력의 실제 숫자가 달라도 같은 guard를 만족하면 재사용할 수 있습니다. 다만 숫자를 Python scalar로 꺼내 제어 흐름에 사용하면 별도의 capture/specialization 문제가 생길 수 있습니다.

07. AOTAutograd: mutation과 미분을 명시적인 graph로

원문: §8 · pp. 40–46 · 목차로

7.1 Functionalization은 무엇을 보존하는가?

`y = x.view(-1); y.add_(1)`에서는 `y`와 `x`가 가트은 storage를 공유합니다. Compiler가 연산을 바꾸려면 이 alias/mutation 의미를 알아야 합니다.

Functionalization은 내부 계산을 functional update로 표현하고 필요한 mutation을 호출 경계에 반영합니다. 호출자가 보던 input 변경을 임의로 없애는 변환이 아닙니다. 아래 실험은 원본과 functionalized 함수의 output뿐 아니라 변경된 input도 비교합니다.

```
aot_run = experiment('aot_autograd')
mutation = aot_run['mutation']
fig = make_subplots(rows=1, cols=3, subplot_titles=['실행 전', 'eager 호출 후 input', 'functionalized 호출 후 input'])
for col, key in enumerate(['before', 'eager_input_after', 'functional_input_after'], 1):
    values = mutation[key]
    fig.add_trace(go.Heatmap(z=values, text=values, texttemplate='%{text}', zmin=0, zmax=1,
                           colorscale='Blues', showscale=False), row=1, col=col)
viz.style(fig, '내부 표현을 바꿔도 호출자가 관찰하는 mutation은 보존', '현재 CUDA 실행 · input/output 동등성 검증',
360).show()
viz.dag(aot_run['functionalized_graph'], 'Functionalization 뒤의 실제 ATen graph').show()
show_source(mutate_view)
```

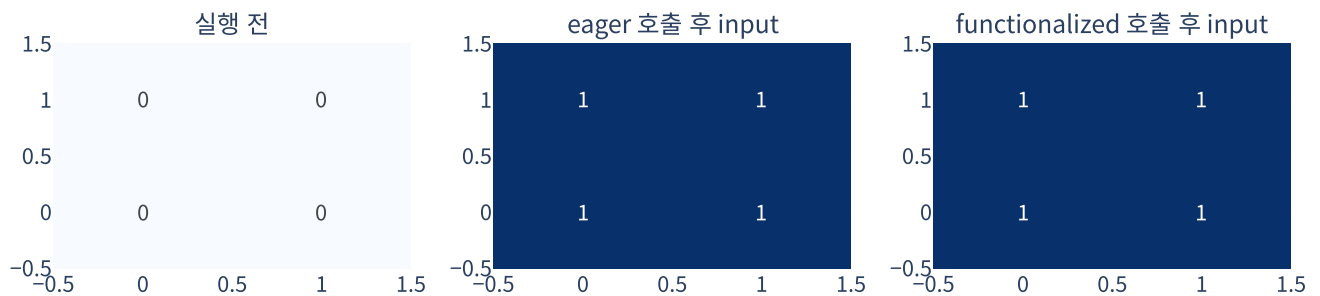
CUDA 실험 실행: `aot_autograd · aot_autograd-9e6fbfec602d`

완료 · NVIDIA GeForce RTX 2080 · 원본: `study02/artifacts/aot_autograd-9e6fbfec602d`

그림 23

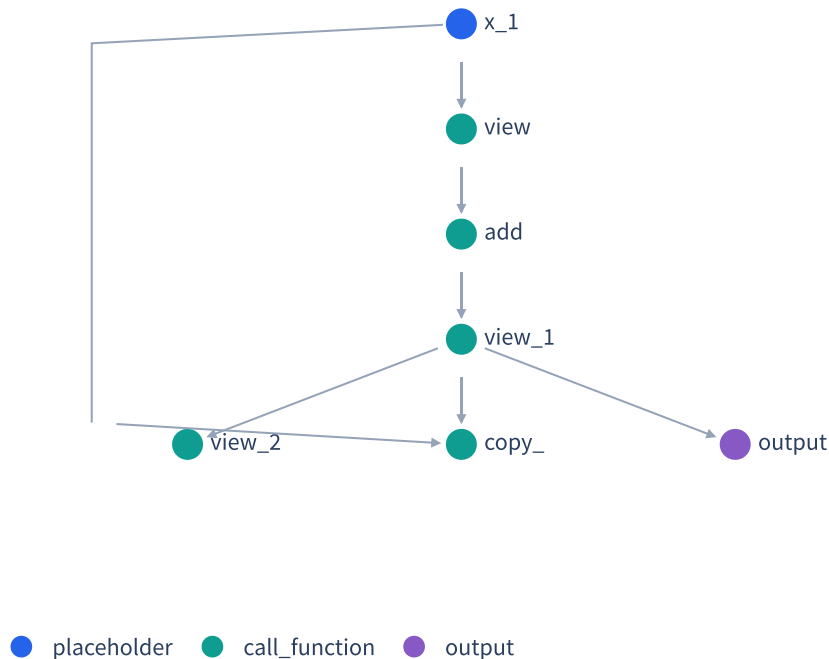
내부 표현을 바꿔도 호출자가 관찰하는 mutation은 보존

현재 CUDA 실행 · input/output 동등성 검증



Functionalization 뒤의 실제 ATen graph

현재 capture · 화살표=값의 의존성, 시간축 아님



▼ 실제 소스: /home/riverfog7/Workspace/mlsys_study/study02/examples.py · mutate_view

```
def mutate_view(x):
    view = x.view(-1)
    view.add_(1)
    return x
```

7.2 Decomposition, lowering, fusion의 차이

이름	바꾸는 것	예
Functionalization	Mutation/alias를 다루는 표현	in-place update를 functional 계산과 경계 반영으로 표현
Decomposition	큰 연산을 작은 연산으로 표현	GELU를 mul·erf/tanh·add로 펼치기
Lowering	실행에 더 구체적인 표현으로 이동	ATen 연산을 loop/index/buffer로 바꾸기
Fusion	여러 계산의 실행 경계를 결합	pointwise chain을 같은 generated kernel에 담기

모든 연산을 가장 작은 primitive로 분해하는 것이 목표는 아닙니다. GEMM이나 attention은 전문 library/template 구현을 선택할 수 있습니다.

7.3 Primals, tangents, joint graph

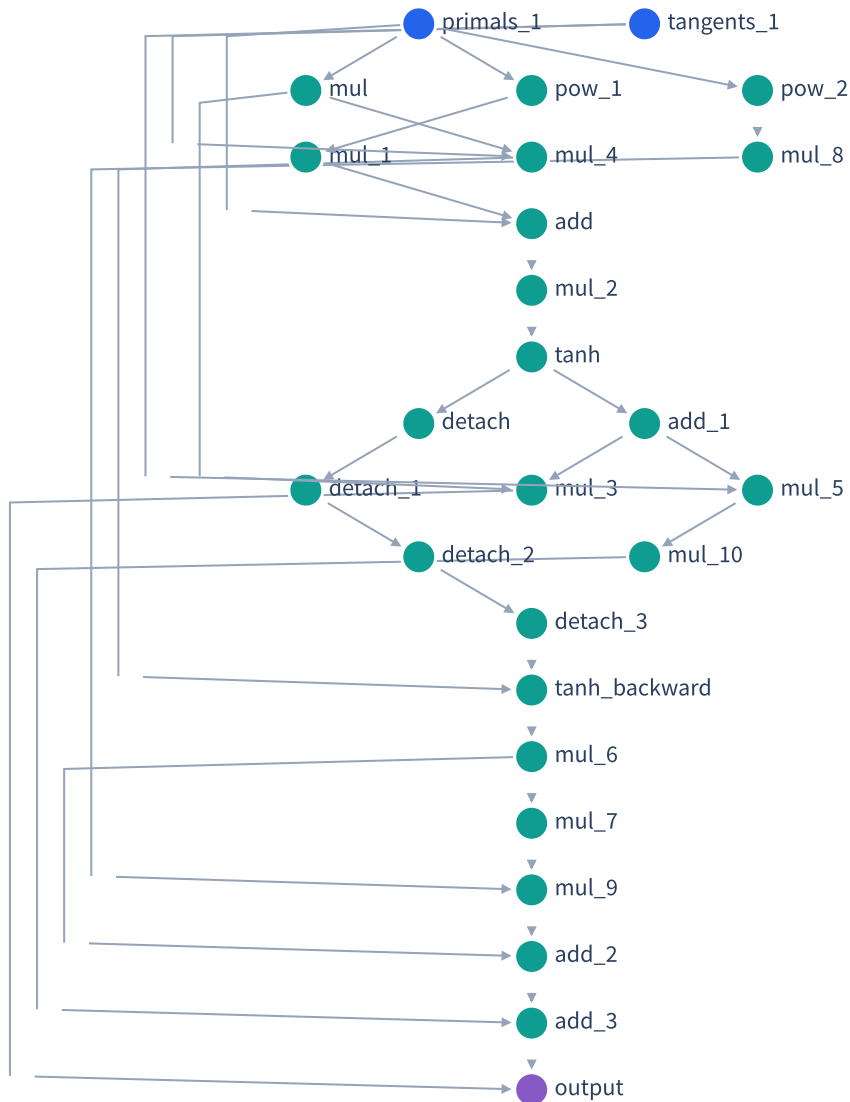
Primal은 forward의 입력·값입니다. 이 AOT backward graph에서 tangent는 output 쪽에서 들어오는 gradient입니다. `output.sum().backward()`라면 각 output 원소에 대한 gradient가 1입니다. 그 값을 입력 방향으로 전파합니다.

Joint graph는 forward와 backward를 함께 나타냅니다. 아래는 원본 파일을 읽은 예시가 아니라 이번 CUDA 실행에서 **partitioner가 받은 실제 graph**입니다. 큰 graph는 오른쪽 위 메뉴로 구간을 바꿀 수 있습니다. 회색 node는 다른 구간에서 들어온 입력입니다.

```
viz.dag(aot_run['graphs']['joint'], '실제 AOT joint graph').show()
```

실제 AOT joint graph

현재 capture · 화살표=값의 의존성, 시간축 아님



● placeholder ● call_function ● output

7.4 Forward는 왜 사용자 output 이외의 값을 반환할까?

Backward가 다시 사용할 값을 forward와 backward 사이에 전달하기 때문입니다. 사용자에게 보이는 GELU 결과와 saved values는 역할이 다릅니다. 아래에서 **보라색으로 강조한 node**가 이번 partition에서 saved value로 반환된 값입니다.

저장하면 재계산을 줄이고, 다시 계산하면 저장 공간을 줄일 수 있습니다. 실제 partition 정책과 저장되는 tensor 목록은 버전·설정·graph에 따라 달라집니다.

```

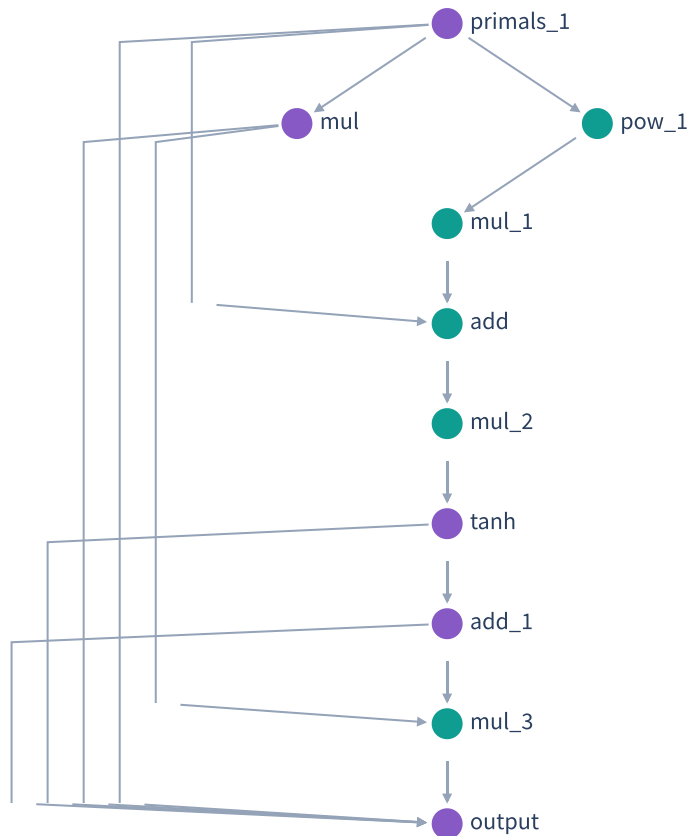
interface = aot_run['interface']
viz.dag(aot_run['graphs']['forward'], '실제 partitioned forward',
highlights=interface['saved_values']).show()
viz.dag(aot_run['graphs']['backward'], '실제 partitioned backward',
highlights=interface['saved_values']).show()
show_summary({'사용자 output': interface['user_output'], 'saved values': interface['saved_values'],
'backward input 이름': interface['backward_inputs'], '실제 input gradient':
aot_run['gradient']})
fw_names = aot_run['graphs']['forward']['returns']
fw_values = aot_run['runtime_values']['forward']['outputs']
fig = go.Figure()
for name, value in zip(fw_names, fw_values):
    fig.add_scatter(x=aot_run['input'], y=value['values'], mode='lines+markers', name=name)
fig.update_xaxes(title='원래 입력 x'); fig.update_yaxes(title='실제 반환 tensor 값')
viz.style(fig, 'Forward 경계를 넘어 전달한 실제 값', '현재 CUDA AOT runtime · 첫 항목은 사용자 output, 나머지는 saved
values', 430).show()

```

그림 26

실제 partitioned forward

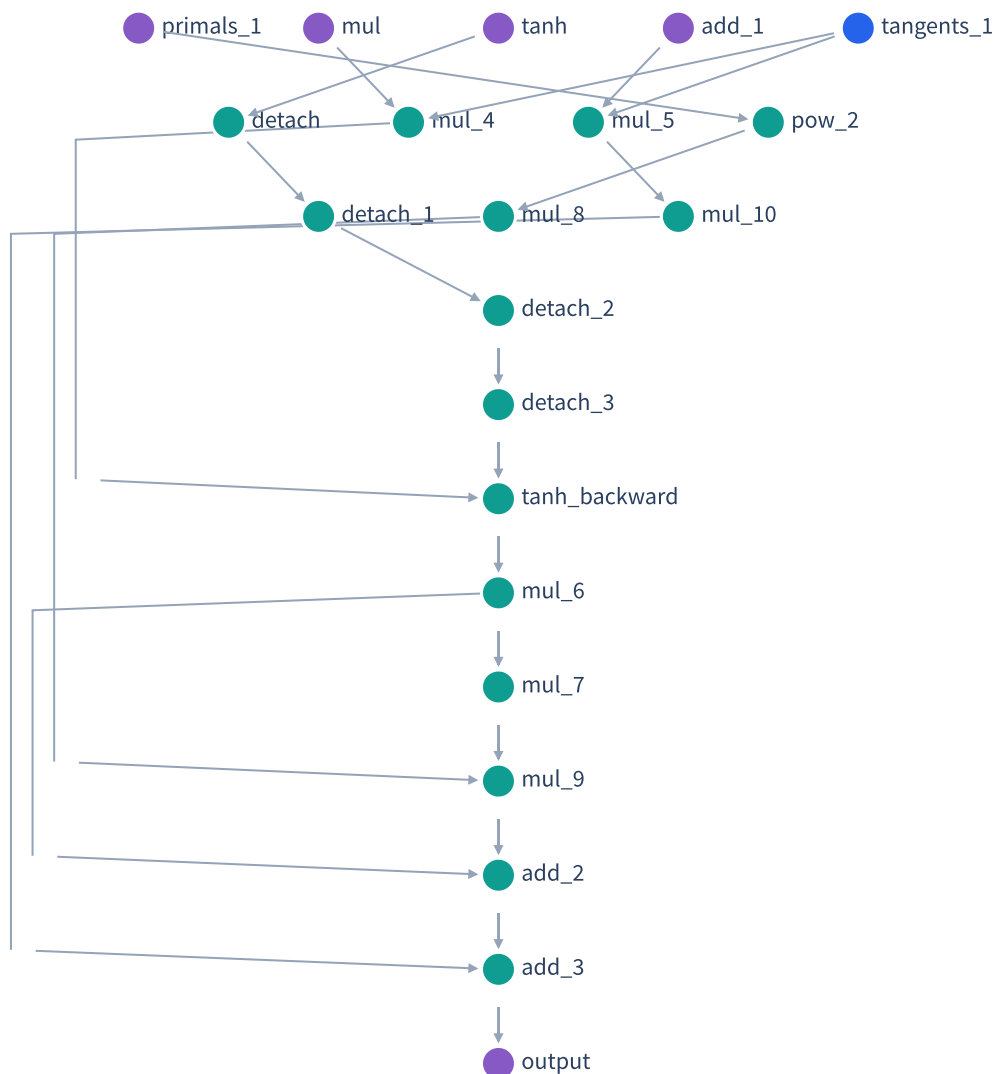
현재 capture · 화살표=값의 의존성, 시간축 아님



● placeholder ● call_function ● output

실제 partitioned backward

현재 capture · 화살표=값의 의존성, 시간축 아님



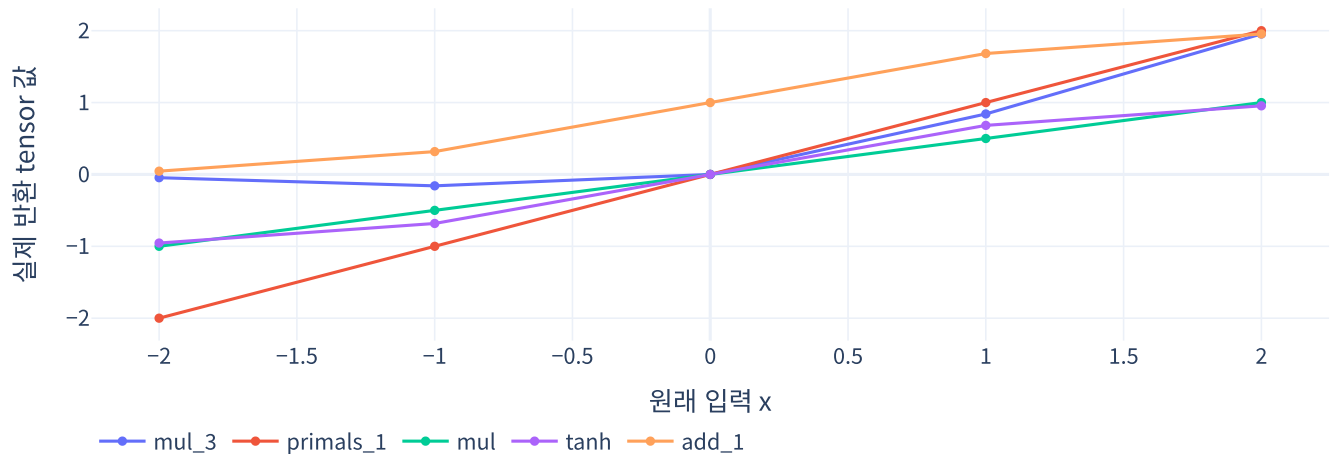
● placeholder ● call_function ● output

확인한 결과

사용자 output	mul_3
saved values	['primals_1', 'mul', 'tanh', 'add_1']
backward input 이름	['primals_1', 'mul', 'tanh', 'add_1', 'tangents_1']
실제 input gradient	[-0.08609922230243683, -0.08296409249305725, 0.5, 1.0829640626907349, 1.0860992670059204]

Forward 경계를 넘어 전달한 실제 값

현재 CUDA AOT runtime · 첫 항목은 사용자 output, 나머지는 saved values



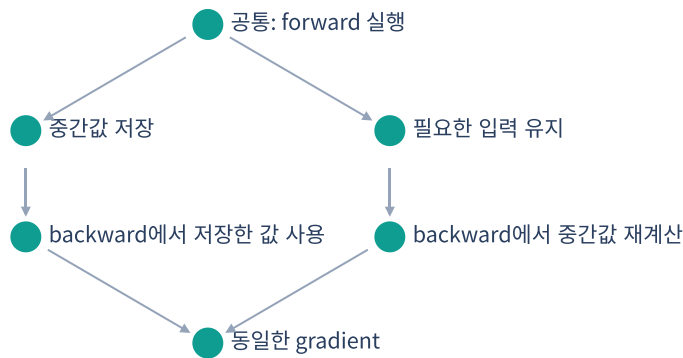
저장과 재계산의 선택을 그림으로 읽기

두 경로는 같은 gradient를 얻으려는 전략입니다. 아래 첫 그림은 선택의 개념도이고, 막대는 이번 AOT가 실제 반환한 saved value의 논리적 byte 크기입니다. Input alias와 allocator를 고려한 추가 allocation 양과는 다릅니다. 실제 partitioner가 선택하지 않은 경로의 성능은 측정가스로 표시하지 않습니다.

```
viz.concept(['공통: forward 실행', '중간값 저장', 'backward에서 저장한 값 사용',
            '필요한 입력 유지', 'backward에서 중간값 재계산', '동일한 gradient'],
            'Saved tensor와 recomputation의 교환관계',
            edges=[(0,1),(1,2),(2,5),(0,3),(3,4),(4,5)]).show()
saved_records = list(zip-fw_names[1:], fw_values[1:])
bytes_per_element = torch.empty((), device=DEVICE, dtype=DTTYPE).element_size()
fig = go.Figure(go.Bar(x=[name for name, value in saved_records],
                        y=[math.prod(value['shape'])*bytes_per_element for name, value in saved_records],
                        marker_color=viz.PURPLE))
fig.update_yaxes(title='전달 tensor의 logical bytes')
viz.style(fig, '이번 forward가 backward로 넘긴 값의 크기',
          '현재 AOT 반환 shape로 계산 · 실제 추가 allocation/peak memory 아님', 360).show()
```

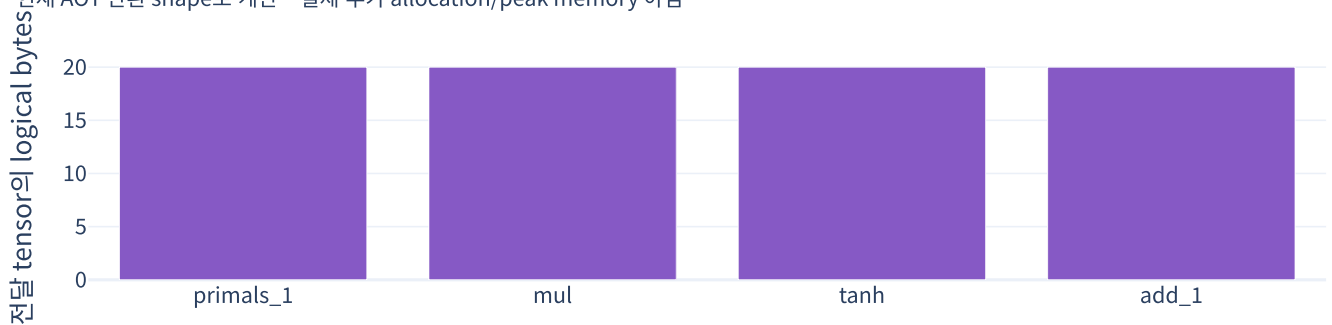
Saved tensor와 recomputation의 교환관계

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



이번 forward가 backward로 넘긴 값의 크기

현재 AOT 반환 shape로 계산 · 실제 추가 allocation/peak memory 아님



확인해 보기 — Saved tensor 4개가 모든 AOT 실행에 필요한 고정 규칙인가요?

▼ 해설 펼치기

아닙니다. 이 실험의 실제 forward signature를 읽어야 합니다. 저장과 재계산의 선택은 partitioner와 graph에 따라 달라집니다. 특히 inference, training, 모델 크기와 compiler 버전을 바꾸면 결과가 달라질 수 있습니다.

08. Inductor: tensor 연산을 loop와 buffer로

원문: §9 · pp. 47–52 · 목차로

8.1 이제 진짜 Inductor와 CUDA codegen을 실행한다

앞의 custom backend는 capture와 AOT 관찰에 집중했습니다. 여기서는 backend="inductor"로 compile하고 실제 GPU output을 eager와 비교합니다. 새 프로세스의 cache와 debug 경로도 새로 만들므로 이번 실행의 FX·IR·Triton 파일을 연습니다.

Inductor는 functional ATen FX를 받고 graph-level transformation을 수행한 뒤 lowering합니다. Input FX와 transformed FX가 가트을 수도 있습니다. **파일이 같다고 뒤의 lowering/codegen까지 아무 일도 안 했다는 뜻은 아닙니다.**

```
gelu_ir = experiment('inductor', workload='gelu', channels=16)
viz.dag(gelu_ir['graphs']['input'][0], 'Inductor가 받은 실제 functional ATen FX').show()
viz.dag(gelu_ir['graphs']['transformed'][0], 'Graph pass 뒤의 실제 FX').show()
show_summary(gelu_ir['correctness'], 'CUDA eager / compiled 비교')
```

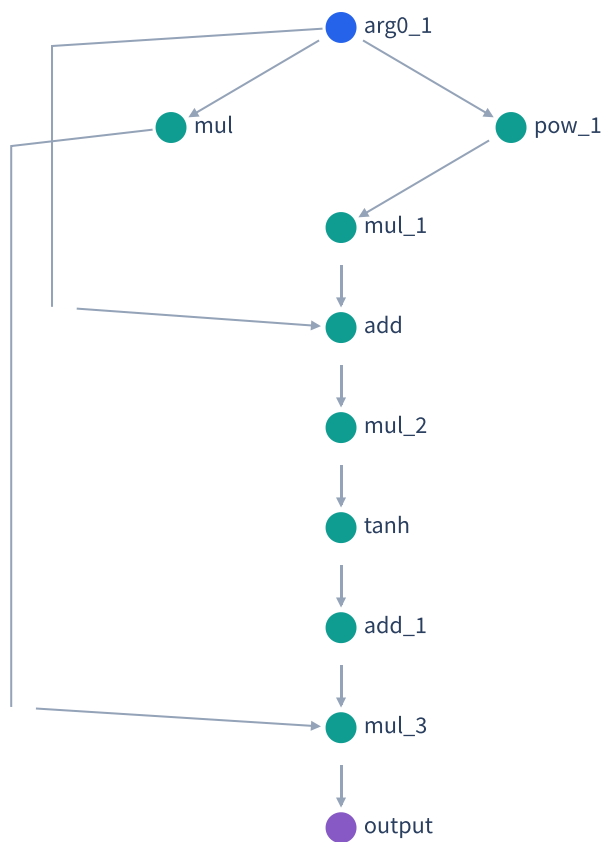
CUDA 실험 실행: inductor · inductor-41b12ff94e55

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/inductor-41b12ff94e55

그림 31

Inductor가 받은 실제 functional ATen FX

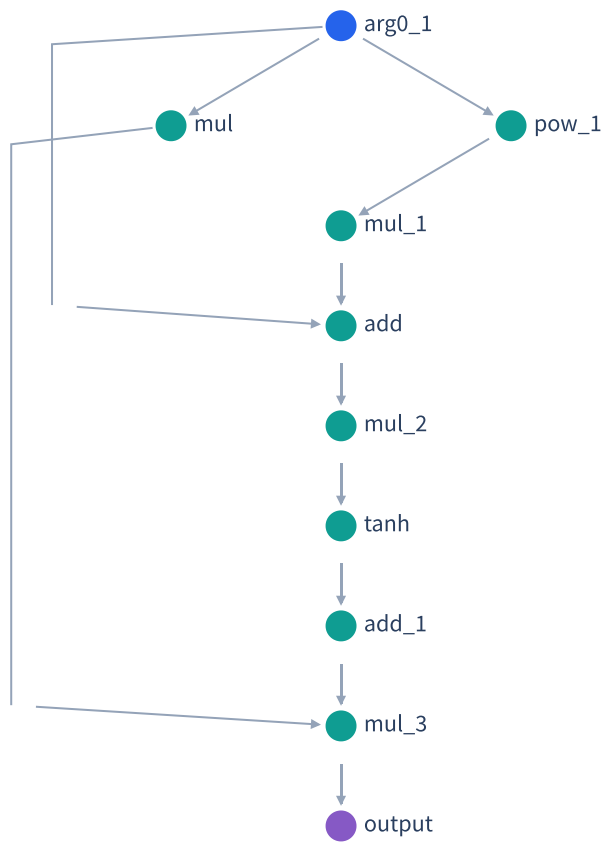
현재 capture · 화살표=값의 의존성, 시간축 아님



● placeholder ● call_function ● output

Graph pass 뒤의 실제 FX

현재 capture · 화살표=값의 의존성, 시간축 아님



● placeholder ● call_function ● output

CUDA eager / compiled 비교

passed	True
rtol	0.0001
atol	1e-05
gradients_checked	0
output_shape	[1, 8, 16]
max_abs_error	4.470348358154297e-08

8.2 “무엇을 계산?”에서 “어떤 index를 순회?”로

Pointwise add는 $z = x + y$ 에서 각 원소의 load/add/store로 내려갑니다. LayerNorm의 mean은 outer 축 [B, T]마다 C축을 줄이는 reduction입니다.

Pointwise	Output index 하나의 값을 어떻게 계산하는가?
Reduction	어떤 축의 여러 값을 합치는가?
ComputedBuffer	이 계산 결과를 어떤 논리 buffer로 나타내는가?
View/Layout	Logical index가 storage offset으로 어떻게 대응하는가?
ExternKernel/template	Body를 직접 생성할지, 전문 구현을 호출할지?

GraphLowering은 FX node의 ATen target을 보고 lowering registry에서 handler를 찾습니다. 내부 파일 위치는 설치된 PyTorch에서 직접 확인합니다. 아래 원본 소스는 지금 환경의 구현이며, 모든 helper 이름을 외울 필요는 없습니다.

```
import inspect
import torch._inductor.graph as inductor_graph
import torch._inductor.lowering as inductor_lowering
show_summary({'설치된 GraphLowering 소스': inspect.getsourcefile(inductor_graph.GraphLowering),
              'lowering registry 소스': inspect.getsourcefile(inductor_lowering),
              '등록된 target 수': len(inductor_lowering.lowerings)})
details('GraphLowering.call_function: 현재 구현 앞부분',
        '\n'.join(inspect.getsource(inductor_graph.GraphLowering.call_function).splitlines()[55]))
handler = inductor_lowering.lowerings.get(torch.ops.aten.add.Tensor)
if handler is not None:
    details('aten.add.Tensor의 실제 registry handler', inspect.getsource(handler)[3500])
outer, channel = np.indices((8, 16))
offsets = outer * 16 + channel
viz.heatmap(offsets, 'outer 8개 x reduction C=16: logical index → offset',
             source='개념도 · 실제 실험 shape [1,8,16]의 contiguous layout에서 계산').show()
```

확인한 결과

설치된 GraphLowering 소스	/home/riverfog7/Workspace/mlsys_study/.venv/lib/python3.9/site-packages/torch/_inductor/graph.py
lowering registry 소스	/home/riverfog7/Workspace/mlsys_study/.venv/lib/python3.9/site-packages/torch/_inductor/lowering.py
등록된 target 수	1735

▼ GraphLowering.call_function: 현재 구현 앞부분

```
def call_function(self, target: Callable, args: Any, kwargs: dict[str, Any]) -> Any: # type: ignore[type-arg, override]
    if target is operator.getitem and isinstance(args[0], (list, tuple, dict)):
        return super().call_function(target, args, kwargs)

    # hasattr on OpOverloadPacket is slow, check isinstance first
    if not isinstance(target, torch._ops.OpOverloadPacket) and hasattr(
        target, "_inductor_lowering_function"
    ):
        # passthrough lowerings from .pattern_matcher
        return target(*args, **kwargs)

    if target not in lowerings:
        assert isinstance(target, torch._ops.OpOverload), (
            f"{target} is not an OpOverload"
        )
        base_name = target.name().split(".")[0]
        if base_name in FALLBACK_ALLOW_LIST:
            make_fallback(target, warn=False, override_decomp=True)
        elif config.implicit_fallbacks:
            error = (
                MissingOperatorWithDecomp
                if get_decompositions([target])
                else MissingOperatorWithoutDecomp
            )
```

```

log.info(
    "Creating implicit fallback for:\n%s",
    error.operator_str(target, args, kwargs),
)

tag = get_layout_constraint_tag(target, with_default=False)
if (
    tag is None
    and torch._library.utils.is_builtin(target)
    and self.is_backward
):
    # for implicit fallback ATen ops during backward, if there
    # is no layout constraint tag, we conservatively require contiguous
    # input since some eager kernels do not
    # support non-contiguous inputs. Otherwise they may silently cause
    # accuracy problems. Check https://github.com/pytorch/pytorch/issues/140452
    # We only do this For ATen ops and for backward.
    #
    # TODO: should really switch to "needs_fixed_stride" constraint on these
    # and identify them one by one.
    decided_constraint = require_contiguous # type: ignore[assignment]
else:
    tag = get_layout_constraint_tag(target, with_default=True)
    decided_constraint = tag_to_layout_constraint(tag)

make_fallback(target, layout_constraint=decided_constraint)

elif get_decompositions([target]):
    # There isn't a good way to dynamically patch this in
    # since AOT Autograd already ran. The error message tells
    # the user how to fix it.

```

▼ aten.add.Tensor의 실제 registry handler

```

def inner(*inputs: TensorBox, alpha=None):
    if triton_fallback is not None and any(
        isinstance(inp, IRNode) and is_triton(inp) for inp in inputs
    ):
        assert not allow_alpha # not implemented
        return triton_fallback(*inputs)

    inputs = promote_constants(inputs, override_return_dtype)
    if allow_alpha:
        if alpha is not None and alpha != 1:
            inputs = list(inputs)
            inputs[-1] = mul(inputs[-1], alpha)
    else:
        assert alpha is None
    loaders = [x.make_loader() for x in inputs]
    ranges = inputs[0].get_size()
    dtype = override_return_dtype or inputs[0].get_dtype()

    for other in inputs[1:]:
        assert isinstance(other, ir.BaseConstant) or len(ranges) == len(
            other.get_size()
        ), f"ndim mismatch {fn} {ranges} {other.get_size()}"

    # in tracing, we will annotate pointwise nodes that correspond to the output of
    # a pointwise node that would have been run in eager. intermediary pointwise nodes
    # during decompositions are not annotated.
    low_pr_fp = (torch.bfloat16, torch.float16)
    emulate_precision_casts = (
        V.graph is not None
        and getattr(V.graph, "current_node", None) is not None

```

```

        and V.graph.current_node.meta is not None
        and V.graph.current_node.meta.get("low_precision_pointwise_barrier", False)
        and dtype in low_pr_fp
    )

def inner_fn(index):
    assert len(index) == len(ranges), f"wrong ndim {index} {ranges}"
    if dtype == torch.bool and override_fn_when_input_bool is not None:
        return override_fn_when_input_bool(*[load(index) for load in loaders])
    else:
        inputs_loaded = []
        for inp_index, load in enumerate(loaders):
            out = load(index)
            inp_dtype = inputs[inp_index].get_dtype()
            if emulate_precision_casts and inp_dtype in low_pr_fp:
                downcast = ops.to_dtype(out, inp_dtype, use_compute_types=False)
                out = ops.to_dtype(downcast, inp_dtype)
            inputs_loaded.append(out)

        out = fn(*inputs_loaded)
        if emulate_precision_casts:
            # fp16/bf16 kernels are computed in fp32. Casting down to fp16/bf16 here,
            # then upcasting again, to emulate casts that eager would do.
            downcast = ops.to_dtype(out, dtype, use_compute_types=False)
            return ops.to_dtype(downcast, dtype)
        return out

if not override_device:
    device = None
    for i in inputs:
        if is_gpu(i.get_device().type):
            device = i.get_device()
            break
    if not device:
        device = inputs[0].get_device()

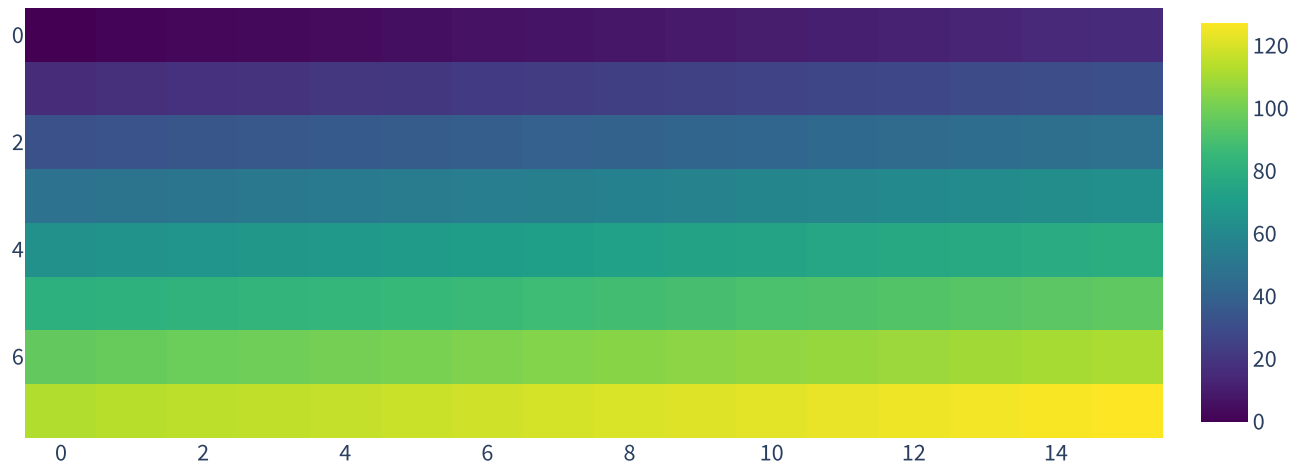
device = override_device or device

return Pointwise.create(
    device=device, # type: ignore[arg-type]
    dtype=dtype,
    inner_fn=inner_fn,
    ranges=ranges,
)

```

outer 8개 × reduction C=16: logical index → offset

개념도 · 실제 실험 shape [1,8,16]의 contiguous layout에서 계산



8.3 같은 GELU가 하나의 composed loop가 되는 경우

아래 IR graph는 이번 dump에서 추출한 scheduler node입니다. Hover의 domain과 원본 loop body를 함께 읽습니다. 원문의 GPU 없는 환경과 같은 수를 가정하지 않습니다.

주목할 지점은 `ops.load`, scalar 식, `ops.store`입니다. FX에서 여러 node였던 계산이 pre-fusion IR에서 이미 같은 loop body에 담길 수 있습니다. 이 경우 pre/post 파일만 비교해서 “fusion이 없었다”고 결론 내리면 안 됩니다.

```

gelu_capture = gelu_ir['captures'][0]
viz.dag(gelu_capture['pre'], '현재 GELU의 loop / buffer IR').show()
viz.fusion_groups(gelu_capture).show()
show_artifact(gelu_ir, gelu_capture['files']['ir_pre_fusion.txt'], max_lines=95)
show_summary({'FX operation node': sum(n['op'].startswith('call_') for n in gelu_ir['graphs']['input'][0]
['nodes']),
              'pre scheduler nodes': len(gelu_capture['pre']['nodes']),
              'post scheduler groups': len(gelu_capture['post']['nodes']),
              'wrapper 호출': [c['name'] for c in gelu_capture['wrapper']['calls']]})

```

현재 GELU의 loop / buffer IR

현재 capture · 화살표=값의 의존성, 시간축 아님

● op0

Scheduler 전후: 실제 node 이름으로 확인한 묶음

현재 IR · 선의 폭=포함된 pre node 수 · GPU 시간/메모리 양 아님



▼ 이번 실행의 원본: raw/debug/torch_compile_debug/run_2026_09_26_11_31_39_904631-pid_3408696/torchinductor/model__0_inference_0.0/ir_pre_fusion.txt

```

1  op0: SchedulerNode(ComputedBuffer)
2  op0.writes = [MemoryDep('buf0', c0, {c0: 128})]
3  op0.unmet_dependencies = []
4  op0.met_dependencies = [MemoryDep('arg0_1', c0, {c0: 128})]
5  op0.outputs = [
6      buf0: ComputedBuffer
7      buf0.layout = FixedLayout('cuda:0', torch.float32, size=[1, 8, 16], stride=[128, 16, 1])
8      buf0.users = [NodeUser(node=OUTPUT, can_inplace=False, is_weak=False)]
9  ]

```

```

10 op0.group.device = cuda:0
11 op0.group.iteration = (128, 1)
12 op0.sizes = ([128], [])
13 arg0_1_layout = FixedLayout('cuda:0', torch.float32, size=[1, 8, 16], stride=[128, 16, 1])
14 buf0_layout = FixedLayout('cuda:0', torch.float32, size=[1, 8, 16], stride=[128, 16, 1])
15 class op0_loop_body:
16     var_ranges = {p0: 128}
17     index0 = p0
18     def body(self, ops):
19         get_index = self.get_index('index0')
20         load = ops.load('arg0_1', get_index)
21         constant = ops.constant(0.5, torch.float32)
22         mul = ops.mul(load, constant)
23         get_index_1 = self.get_index('index0')
24         load_1 = ops.load('arg0_1', get_index_1)
25         get_index_2 = self.get_index('index0')
26         load_2 = ops.load('arg0_1', get_index_2)
27         mul_1 = ops.mul(load_2, load_2)
28         mul_2 = ops.mul(mul_1, load_2)
29         constant_1 = ops.constant(0.044715, torch.float32)
30         mul_3 = ops.mul(mul_2, constant_1)
31         add = ops.add(load_1, mul_3)
32         constant_2 = ops.constant(0.7978845608028654, torch.float32)
33         mul_4 = ops.mul(add, constant_2)
34         tanh = ops.tanh(mul_4)
35         constant_3 = ops.constant(1.0, torch.float32)
36         add_1 = ops.add(tanh, constant_3)
37         mul_5 = ops.mul(mul, add_1)
38         get_index_3 = self.get_index('index0')
39         store = ops.store('buf0', get_index_3, mul_5, None)
40         return store
41
42

```

확인한 결과

FX operation node	8
pre scheduler nodes	1
post scheduler groups	1
wrapper 호출	['triton_poi_fused_add_mul_pow_tanh_0.run']

확인해 보기 — IR의 ComputedBuffer마다 CUDA allocation이 생기나요?

▼ 해설 펼치기

항상 그렇지 않습니다. 같은 kernel 안에서 소비되는 값은 scalar/register 계산으로만 남을 수 있습니다. 실제 allocation과 kernel 경계는 generated wrapper와 profiler에서 확인합니다.

09. Scheduling과 memory planning

원문: §10–11 · pp. 53–60 · 목차로

9.1 어떤 순서로, 어떤 묶음으로 실행할까?

Producer가 쓴 값을 consumer가 읽으면 순서를 바꿀 수 없습니다. Scheduler는 read/write dependency를 보존하는 순서를 만들고 합칠 수 있는 계산을 묶습니다.

- **Vertical grouping:** producer의 output을 consumer가 이어 계산합니다.
- **Horizontal grouping:** 같은 input/domain을 쓰는 독립 계산을 함께 배치합니다.
- Reduction, 다른 layout, 외부 library, mutation 의존성, 자원 비용은 경계를 만들 수 있습니다.

이 scheduling은 compiler가 kernel을 생성하기 전에 하는 일입니다. GPU가 실행 중 warp를 배치하는 hardware scheduling과는 계층이 다릅니다.

```
viz.concept(['x', 'x*x', 'x*x+1'], 'Vertical: 값의 생산과 소비를 연결').show()
viz.concept(['x', 'sin(x)', 'cos(x)', '두 결과 반환'], 'Horizontal: 같은 input의 두 분기',
            edges=[(0,1),(0,2),(1,3),(2,3)]).show()
layernorm_ir = experiment('inductor', workload='layernorm', channels=16)
residual_ir = experiment('inductor', workload='residual_layernorm', channels=16)
residual_capture = residual_ir['captures'][0]
viz.dag(residual_capture['pre'], 'Residual + LayerNorm: scheduling 이전').show()
viz.dag(residual_capture['post'], 'Residual + LayerNorm: scheduling 이후').show()
viz.fusion_groups(residual_capture).show()
show_summary({'pre nodes': len(residual_capture['pre']['nodes']),
              'post groups': len(residual_capture['post']['nodes']),
              '실제 wrapper 호출': [c['name'] for c in residual_capture['wrapper']['calls']]})
```

그림 36

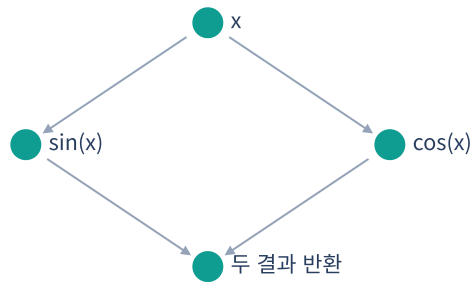
Vertical: 값의 생산과 소비를 연결

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



Horizontal: 같은 input의 두 분기

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



CUDA 실험 실행: `inductor · inductor-8c13408a2fbb`

완료 · NVIDIA GeForce RTX 2080 · 원본: `study02/artifacts/inductor-8c13408a2fbb`

CUDA 실험 실행: `inductor · inductor-8dcfade14a8d`

완료 · NVIDIA GeForce RTX 2080 · 원본: `study02/artifacts/inductor-8dcfade14a8d`

그림 38

Residual + LayerNorm: scheduling 이전

현재 capture · 화살표=값의 의존성, 시간축 아님

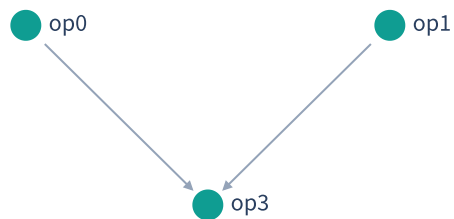


그림 39

Residual + LayerNorm: scheduling 이후

현재 capture · 화살표=값의 의존성, 시간축 아님

● op0_op1_op3

그림 40

Scheduler 전후: 실제 node 이름으로 확인한 묶음

현재 IR · 선의 폭=포함된 pre node 수 · GPU 시간/메모리 양 아님



확인한 결과

pre nodes	3
post groups	1
실제 wrapper 호출	['triton_per_fused_add_native_layer_norm_0.run']

9.2 Memory 문제는 세 계층으로 나누어 읽는다

계층	결정·관찰하는 것
AOT partitioner	Forward 값을 backward까지 저장할지 재계산할지
Inductor/wrapper	Graph 실행 안에서 temporary를 언제 만들고 재사용할지
CUDA caching allocator	Process가 확보한 device memory block을 어떻게 보관·재사용할지

allocated는 살아 있는 tensor 등에 할당된 양, reserved는 allocator가 확보한 양입니다. del로 Python 참조를 지워도 reserved가 바로 줄지 않을 수 있습니다.

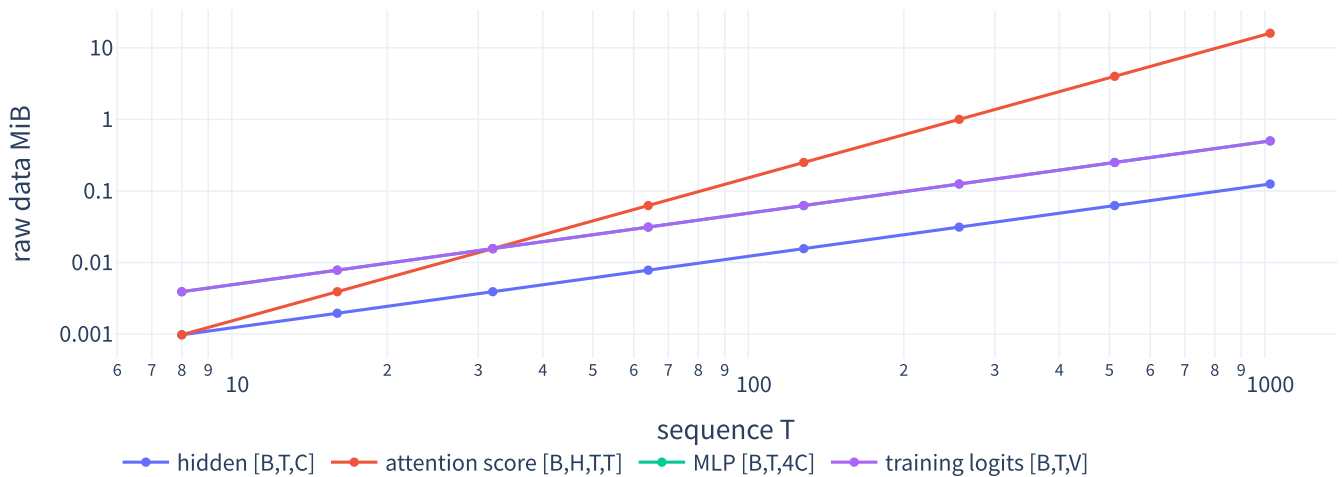
Contiguous tensor의 raw data 크기는 $\text{numel} * \text{element_size}$ 입니다. Metadata와 allocator overhead는 포함하지 않습니다. View와 weight tying은 같은 storage를 공유하므로 단순 합산에 주의합니다.

```
B, C, H, V = 1, 32, 4, 128
lengths = [8, 16, 32, 64, 128, 256, 512, 1024]
item_bytes = torch.empty(), device=DEVICE, dtype=DTYPE).element_size()
fig = go.Figure()
for label, elements in [
    ('hidden [B,T,C]', [B*t*C for t in lengths]),
    ('attention score [B,H,T,T]', [B*H*t*t for t in lengths]),
    ('MLP [B,T,4C]', [B*t*4*C for t in lengths]),
    ('training logits [B,T,V]', [B*t*V for t in lengths]):
    fig.add_scatter(x=lengths, y=[n*item_bytes/2**20 for n in elements], mode='lines+markers', name=label)
fig.update_xaxes(title='sequence T', type='log'); fig.update_yaxes(title='raw data MiB', type='log')
viz.style(fig, 'T가 커질 때 attention score는 제곱으로 증가', '이론 계산 · 현재 dtype의 element_size 사용 · allocator/gradient 비용 제외', 440).show()
```

그림 41

T가 커질 때 attention score는 제곱으로 증가

이론 계산 · 현재 dtype의 element_size 사용 · allocator/gradient 비용 제외



9.3 같은 kernel 안에서 사라지는 intermediate

GELU와 residual+LayerNorm의 generated wrapper에서 실제 allocation을 확인합니다. 아래 막대는 wrapper 소스 줄의 정적 참조 구간입니다. GPU wall-clock timeline이나 정확한 allocator 해제 시각을 뜻하지 않습니다.

같은 storage 색을 공유하면 alias/reinterpret/reuse 관계를 읽을 수 있습니다. 더 큰 GPT block에서는 이 관계가 풍부해집니다. 크기를 확정할 수 없는 symbolic allocation은 추측하지 않습니다.

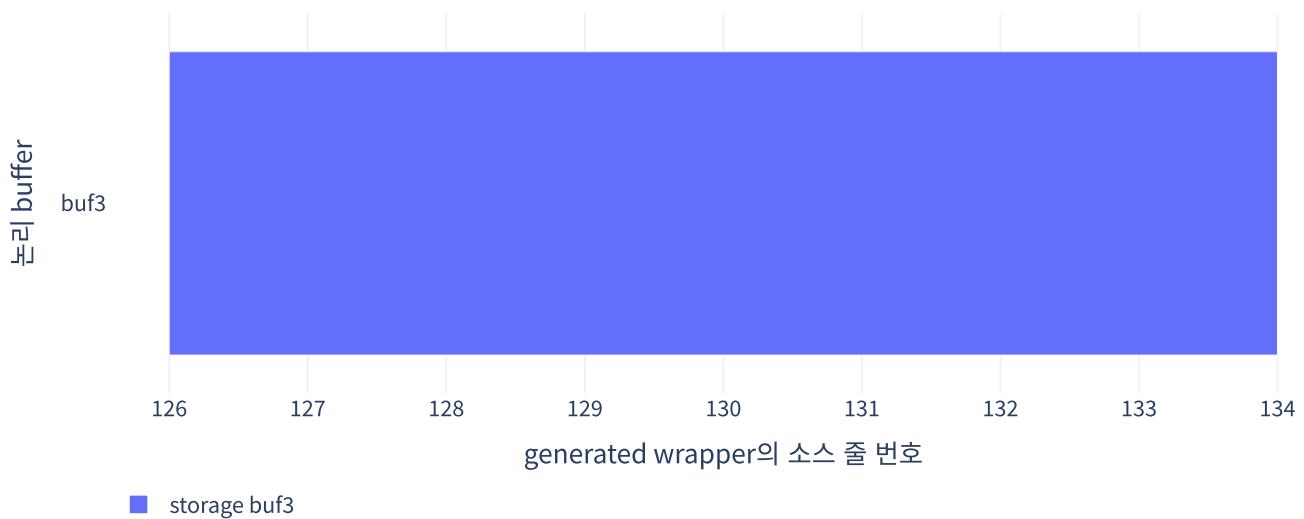
```
viz.lifetimes(residual_capture['wrapper']).show()
```

```
show_artifact(residual_ir, residual_capture['files']['output_code.py'], max_lines=155)
```

그림 42

Buffer lifetime / storage reuse

현재 wrapper · 정적 참조 구간이며 GPU wall time이 아님



▼ 이번 실행의 원본: raw/debug/torch_compile_debug/run_2026_09_26_11_31_54_416657-pid_3410020/torchinductor/model__0_inference_0.0/output_code.py

```
1 # AOT ID: ['0_inference']
2 from ctypes import c_void_p, c_long, c_int
3 import torch
4 import math
5 import random
6 import os
7 import tempfile
8 from math import inf, nan
9 from cmath import nanj
10 from torch._inductor.hooks import run_intermediate_hooks
11 from torch._inductor.utils import maybe_profile
12 from torch._inductor.codegen.memory_planning import _align as align
13 from torch import device, empty_strided
14 from torch._inductor.async_compile import AsyncCompile
15 from torch._inductor.select_algorithm import extern_kernels
16 import triton
17 import triton.language as tl
18 from torch._inductor.runtime.triton_heuristics import start_graph, end_graph
19 from torch._C import _cuda_getCurrentRawStream as get_raw_stream
20 from torch._C import _cuda_getCurrentRawStream as get_raw_stream
21
22 aten = torch.ops.aten
23 inductor_ops = torch.ops.inductor
24 _quantized = torch.ops._quantized
25 assert_size_stride = torch._C._dynamo.guards.assert_size_stride
26 assert_alignment = torch._C._dynamo.guards.assert_alignment
27 empty_strided_cpu = torch._C._dynamo.guards._empty_strided_cpu
28 empty_strided_cuda = torch._C._dynamo.guards._empty_strided_cuda
29 empty_strided_xpu = torch._C._dynamo.guards._empty_strided_xpu
30 reinterpret_tensor = torch._C._dynamo.guards.reinterpret_tensor
31 alloc_from_pool = torch.ops.inductor._alloc_from_pool
32 async_compile = AsyncCompile()
33 empty_strided_p2p = torch._C._distributed_c10d._SymmetricMemory.empty_strided_p2p
34
```

```

35
36 # kernel path: /home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-
8dcfade14a8d/raw/inductor_cache/ja/cja7jmc7uuahaprgjcpdugn5ctpmgze4a56bw7gmmmya7a7f42.py
37 # Topologically Sorted Source Nodes: [hidden, layer_norm], Original ATen: [aten.add,
aten.native_layer_norm]
38 # Source node to ATen node mapping:
39 #   hidden => add
40 #   layer_norm => add_1, add_2, mul, mul_1, rsqrt, sub, var_mean
41 # Graph fragment:
42 #   %add : [num_users=2] = call_function[target=torch.ops.aten.add.Tensor](args = (%arg0_1, %arg1_1),
kwargs = {})
43 #   %var_mean : [num_users=2] = call_function[target=torch.ops.aten.var_mean.correction](args = (%add,
[2]), kwargs = {correction: 0, keepdim: True})
44 #   %sub : [num_users=1] = call_function[target=torch.ops.aten.sub.Tensor](args = (%add, %getitem_1),
kwargs = {})
45 #   %add_1 : [num_users=1] = call_function[target=torch.ops.aten.add.Tensor](args = (%getitem, 1e-05),
kwargs = {})
46 #   %rsqrt : [num_users=1] = call_function[target=torch.ops.aten.rsqrt.default](args = (%add_1,),
kwargs = {})
47 #   %mul : [num_users=1] = call_function[target=torch.ops.aten.mul.Tensor](args = (%sub, %rsqrt),
kwargs = {})
48 #   %mul_1 : [num_users=1] = call_function[target=torch.ops.aten.mul.Tensor](args = (%mul, %arg2_1),
kwargs = {})
49 #   %add_2 : [num_users=1] = call_function[target=torch.ops.aten.add.Tensor](args = (%mul_1, %arg3_1),
kwargs = {})
50 triton_per_fused_add_native_layer_norm_0 =
async_compile.triton('triton_per_fused_add_native_layer_norm_0', '''
51 import triton
52 import triton.language as tl
53
54 from torch._inductor.runtime import triton_helpers, triton_heuristics
55 from torch._inductor.runtime.triton_helpers import libdevice, math as tl_math
56 from torch._inductor.runtime.hints import AutotuneHint, ReductionHint, TileHint, DeviceProperties
57 triton_helpers.set_driver_to_gpu()
58
59 @triton_heuristics.persistent_reduction(
60     size_hints={'x': 8, 'r0': 16},
61     reduction_hint=ReductionHint.INNER,
62     filename=__file__,
63     triton_meta={'signature': {'in_ptr0': '*fp32', 'in_ptr1': '*fp32', 'in_ptr2': '*fp32', 'in_ptr3':
'*fp32', 'out_ptr2': '*fp32', 'xnumel': 'i32', 'r0_numel': 'i32', 'XBLOCK': 'constexpr'}, 'device':
DeviceProperties(type='cuda', index=0, multi_processor_count=46, cc=75, major=7,
regs_per_multiprocessor=65536, max_threads_per_multi_processor=1024, warp_size=32), 'constants': {},
'configs': [{(0,): [[['tt.divisibility', 16]], (1,): [[['tt.divisibility', 16]], (2,): [[['tt.divisibility',
16]], (3,): [[['tt.divisibility', 16]], (4,): [[['tt.divisibility', 16]], (6,): [[['tt.divisibility', 16]]]}],
64     inductor_meta={'grid_type': 'Grid1D', 'autotune_hints': set(), 'kernel_name':
'triton_per_fused_add_native_layer_norm_0', 'mutated_arg_names': [], 'optimize_mem': True, 'no_x_dim': False,
'num_load': 4, 'num_reduction': 4, 'backend_hash':
'B2E490B6EF1B770E6903DC43C133BFE2771D4923CBF2CCDB0EF9CD0DA6ADE45C', 'are_deterministic_algorithms_enabled':
False, 'assert_indirect_indexing': True, 'autotune_local_cache': True, 'autotune_pointwise': True,
'autotune_remote_cache': None, 'force_disable_caches': False, 'dynamic_scale_rblock': True, 'max_autotune':
False, 'max_autotune_pointwise': False, 'min_split_scan_rblock': 256, 'spill_threshold': 16, 'store_cubin':
False, 'tiling_scores': {'x': 0, 'r0': 2176}}
65 )
66 @triton.jit
67 def triton_per_fused_add_native_layer_norm_0(in_ptr0, in_ptr1, in_ptr2, in_ptr3, out_ptr2, xnumel,
r0_numel, XBLOCK : tl.constexpr):
68     xnumel = 8
69     r0_numel = 16
70     R0_BLOCK: tl.constexpr = 16
71     rnumel = r0_numel
72     RBLOCK: tl.constexpr = R0_BLOCK
73     xoffset = tl.program_id(0) * XBLOCK

```

```

74     xindex = xoffset + tl.arange(0, XBLOCK)[: , None]
75     xmask = xindex < xnumel
76     r0_index = tl.arange(0, R0_BLOCK)[None, :]
77     r0_offset = 0
78     r0_mask = tl.full([XBLOCK, R0_BLOCK], True, tl.int1)
79     roffset = r0_offset
80     rindex = r0_index
81     r0_1 = r0_index
82     x0 = xindex
83     tmp0 = tl.load(in_ptr0 + (r0_1 + 16*x0), xmask, other=0.0)
84     tmp1 = tl.load(in_ptr1 + (r0_1 + 16*x0), xmask, other=0.0)
85     tmp26 = tl.load(in_ptr2 + (r0_1), None, eviction_policy='evict_last')
86     tmp28 = tl.load(in_ptr3 + (r0_1), None, eviction_policy='evict_last')
87     tmp2 = tmp0 + tmp1
88     tmp3 = tl.broadcast_to(tmp2, [XBLOCK, R0_BLOCK])
89     tmp5 = tl.where(xmask, tmp3, 0)
90     tmp6 = tl.broadcast_to(tmp3, [XBLOCK, R0_BLOCK])
91     tmp8 = tl.where(xmask, tmp6, 0)
92     tmp9 = tl.sum(tmp8, 1)[: , None]
93     tmp10 = tl.full([XBLOCK, 1], 16, tl.int32)
94     tmp11 = tmp10.to(tl.float32)
95     tmp12 = (tmp9 / tmp11)
96     tmp13 = tmp3 - tmp12
97     tmp14 = tmp13 * tmp13
98     tmp15 = tl.broadcast_to(tmp14, [XBLOCK, R0_BLOCK])
99     tmp17 = tl.where(xmask, tmp15, 0)
100    tmp18 = tl.sum(tmp17, 1)[: , None]
101    tmp19 = tmp2 - tmp12
102    tmp20 = 16.0
103    tmp21 = (tmp18 / tmp20)
104    tmp22 = 1e-05
105    tmp23 = tmp21 + tmp22
106    tmp24 = libdevice.rsqrt(tmp23)
107    tmp25 = tmp19 * tmp24
108    tmp27 = tmp25 * tmp26
109    tmp29 = tmp27 + tmp28
110    tl.store(out_ptr2 + (r0_1 + 16*x0), tmp29, xmask)
111    ''' , device_str='cuda')
112
113
114    async_compile.wait(globals())
115    del async_compile
116
117    def call(args):
118        arg0_1, arg1_1, arg2_1, arg3_1 = args
119        args.clear()
120        assert_size_stride(arg0_1, (1, 8, 16), (128, 16, 1))
121        assert_size_stride(arg1_1, (1, 8, 16), (128, 16, 1))
122        assert_size_stride(arg2_1, (16, ), (1, ))
123        assert_size_stride(arg3_1, (16, ), (1, ))
124        with torch.cuda._DeviceGuard(0):
125            torch.cuda.set_device(0)
126            buf3 = empty_strided_cuda((1, 8, 16), (128, 16, 1), torch.float32)
127            # Topologically Sorted Source Nodes: [hidden, layer_norm], Original ATen: [aten.add,
aten.native_layer_norm]
128            stream0 = get_raw_stream(0)
129            triton_per_fused_add_native_layer_norm_0.run(arg0_1, arg1_1, arg2_1, arg3_1, buf3, 8, 16,
stream=stream0)
130            del arg0_1
131            del arg1_1
132            del arg2_1
133            del arg3_1
134            return (buf3, )

```

```

135
136
137 def benchmark_compiled_module(times=10, repeat=10):
138     from torch._dynamo.testing import rand_strided
139     from torch._inductor.utils import print_performance
140     arg0_1 = rand_strided((1, 8, 16), (128, 16, 1), device='cuda:0', dtype=torch.float32)
141     arg1_1 = rand_strided((1, 8, 16), (128, 16, 1), device='cuda:0', dtype=torch.float32)
142     arg2_1 = rand_strided((16, ), (1, ), device='cuda:0', dtype=torch.float32)
143     arg3_1 = rand_strided((16, ), (1, ), device='cuda:0', dtype=torch.float32)
144     fn = lambda: call([arg0_1, arg1_1, arg2_1, arg3_1])
145     return print_performance(fn, times=times, repeat=repeat)
146
147
148 if __name__ == "__main__":
149     from torch._inductor.wrapper_benchmark import compiled_module_main
150     compiled_module_main('None', benchmark_compiled_module)

```

확인해 보기 — Mean과 variance가 IR에 보이면 별도 global-memory tensor로 반드시 남을까요?

▼ 해설 펼치기

IR의 logical 값과 실제 allocation은 다릅니다. Group 내부에서 소비되면 별도 output allocation으로 남지 않을 수 있습니다. 이번 wrapper의 allocation과 kernel argument를 확인해야 합니다.

10. Code generation: Triton에서 PTX와 SASS까지

원문: §12 · pp. 61–66 · 목차로

10.1 Generated code와 external call

output_code.py는 graph 실행을 연결하는 wrapper입니다. Shape/stride 검사, allocation/reuse, generated Triton 정의·호출, 외부 GEMM/attention 호출이 보입니다.

Generated 경로는 Inductor가 kernel body를 만듭니다. External 경로는 library 등의 구현을 호출합니다. 외부 cuBLAS body를 Inductor가 생성한 것처럼 해당 TTIR을 찾으면 안 됩니다. 실제 선택은 이번 wrapper와 trace를 함께 읽습니다.

```

show_artifact(gelu_ir, gelu_capture['files']['output_code.py'], max_lines=150)
show_summary({stage: len(paths) for stage, paths in gelu_ir['backend_stages'].items()},
             '이번 Inductor CUDA 실행이 생성한 backend stage 파일 수')

```

▼ 이번 실행의 원본: raw/debug/torch_compile_debug/run_2026_09_26_11_31_39_904631-pid_3408696/torchinductor/model__0_inference_0.0/output_code.py

```

1 # AOT ID: ['0_inference']
2 from ctypes import c_void_p, c_long, c_int
3 import torch
4 import math
5 import random
6 import os
7 import tempfile
8 from math import inf, nan
9 from cmath import nanj
10 from torch._inductor.hooks import run_intermediate_hooks
11 from torch._inductor.utils import maybe_profile
12 from torch._inductor.codegen.memory_planning import _align as align
13 from torch import device, empty_strided
14 from torch._inductor.async_compile import AsyncCompile
15 from torch._inductor.select_algorithm import extern_kernels
16 import triton
17 import triton.language as tl
18 from torch._inductor.runtime.triton_heuristics import start_graph, end_graph

```

```

19 from torch._C import _cuda_getCurrentRawStream as get_raw_stream
20 from torch._C import _cuda_getCurrentRawStream as get_raw_stream
21
22 aten = torch.ops.aten
23 inductor_ops = torch.ops.inductor
24 _quantized = torch.ops._quantized
25 assert_size_stride = torch._C._dynamo.guards.assert_size_stride
26 assert_alignment = torch._C._dynamo.guards.assert_alignment
27 empty_strided_cpu = torch._C._dynamo.guards._empty_strided_cpu
28 empty_strided_cuda = torch._C._dynamo.guards._empty_strided_cuda
29 empty_strided_xpu = torch._C._dynamo.guards._empty_strided_xpu
30 reinterpret_tensor = torch._C._dynamo.guards._reinterpret_tensor
31 alloc_from_pool = torch.ops.inductor._alloc_from_pool
32 async_compile = AsyncCompile()
33 empty_strided_p2p = torch._C._distributed_c10d._SymmetricMemory.empty_strided_p2p
34
35
36 # kernel path: /home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-
41b12ff94e55/raw/inductor_cache/xq/cxqzkfumxgbdmxa4e4uxphbtg2ddhjbbjbffx56fu6soq6tobwgq.py
37 # Topologically Sorted Source Nodes: [mul, pow_1, mul_1, add, mul_2, tanh, add_1, mul_3], Original
ATen: [aten.mul, aten.pow, aten.add, aten.tanh]
38 # Source node to ATen node mapping:
39 #   add => add
40 #   add_1 => add_1
41 #   mul => mul
42 #   mul_1 => mul_1
43 #   mul_2 => mul_2
44 #   mul_3 => mul_3
45 #   pow_1 => pow_1
46 #   tanh => tanh
47 # Graph fragment:
48 #   %mul : [num_users=1] = call_function[target=torch.ops.aten.mul.Tensor](args = (%arg0_1, 0.5),
kwargs = {})
49 #   %pow_1 : [num_users=1] = call_function[target=torch.ops.aten.pow.Tensor_Scalar](args = (%arg0_1,
3), kwargs = {})
50 #   %mul_1 : [num_users=1] = call_function[target=torch.ops.aten.mul.Tensor](args = (%pow_1, 0.044715),
kwargs = {})
51 #   %add : [num_users=1] = call_function[target=torch.ops.aten.add.Tensor](args = (%arg0_1, %mul_1),
kwargs = {})
52 #   %mul_2 : [num_users=1] = call_function[target=torch.ops.aten.mul.Tensor](args = (%add,
0.7978845608028654), kwargs = {})
53 #   %tanh : [num_users=1] = call_function[target=torch.ops.aten.tanh.default](args = (%mul_2,), kwargs
= {})
54 #   %add_1 : [num_users=1] = call_function[target=torch.ops.aten.add.Tensor](args = (%tanh, 1), kwargs
= {})
55 #   %mul_3 : [num_users=1] = call_function[target=torch.ops.aten.mul.Tensor](args = (%mul, %add_1),
kwargs = {})
56 triton_poi_fused_add_mul_pow_tanh_0 = async_compile.triton('triton_poi_fused_add_mul_pow_tanh_0', '''
57 import triton
58 import triton.language as tl
59
60 from torch._inductor.runtime import triton_helpers, triton_heuristics
61 from torch._inductor.runtime.triton_helpers import libdevice, math as tl_math
62 from torch._inductor.runtime.hints import AutotuneHint, ReductionHint, TileHint, DeviceProperties
63 triton_helpers.set_driver_to_gpu()
64
65 @triton_heuristics.pointwise(
66     size_hints={'x': 128},
67     filename=__file__,
68     triton_meta={'signature': {'in_ptr0': '*fp32', 'out_ptr0': '*fp32', 'xnumel': 'i32', 'XBLOCK':
'constexpr'}, 'device': DeviceProperties(type='cuda', index=0, multi_processor_count=46, cc=75, major=7,
regs_per_multiprocessor=65536, max_threads_per_multi_processor=1024, warp_size=32), 'constants': {}},
'configs': [(0,): [[('tt.divisibility', 16)], (1,): [[('tt.divisibility', 16)], (2,): [[('tt.divisibility',

```

```

16]]]]},
69     inductor_meta={'grid_type': 'Grid1D', 'autotune_hints': set(), 'kernel_name':
'triton_poi_fused_add_mul_pow_tanh_0', 'mutated_arg_names': [], 'optimize_mem': True, 'no_x_dim': False,
'num_load': 1, 'num_reduction': 0, 'backend_hash':
'B2E490B6EF1B770E6903DC43C133BFE2771D4923CBF2CCDB0EF9CD0DA6ADE45C', 'are_deterministic_algorithms_enabled':
False, 'assert_indirect_indexing': True, 'autotune_local_cache': True, 'autotune_pointwise': True,
'autotune_remote_cache': None, 'force_disable_caches': False, 'dynamic_scale_rblock': True, 'max_autotune':
False, 'max_autotune_pointwise': False, 'min_split_scan_rblock': 256, 'spill_threshold': 16, 'store_cubin':
False, 'tiling_scores': {'x': 1536}},
70     min_elem_per_thread=0
71 )
72 @triton.jit
73 def triton_poi_fused_add_mul_pow_tanh_0(in_ptr0, out_ptr0, xnumel, XBLOCK : tl.constexpr):
74     xnumel = 128
75     xoffset = tl.program_id(0) * XBLOCK
76     xindex = xoffset + tl.arange(0, XBLOCK)[:xnumel]
77     xmask = xindex < xnumel
78     x0 = xindex
79     tmp0 = tl.load(in_ptr0 + (x0), xmask)
80     tmp1 = 0.5
81     tmp2 = tmp0 * tmp1
82     tmp3 = tmp0 * tmp0
83     tmp4 = tmp3 * tmp0
84     tmp5 = 0.044715
85     tmp6 = tmp4 * tmp5
86     tmp7 = tmp0 + tmp6
87     tmp8 = 0.7978845608028654
88     tmp9 = tmp7 * tmp8
89     tmp10 = libdevice.tanh(tmp9)
90     tmp11 = 1.0
91     tmp12 = tmp10 + tmp11
92     tmp13 = tmp2 * tmp12
93     tl.store(out_ptr0 + (x0), tmp13, xmask)
94     '', device_str='cuda')
95
96
97 async_compile.wait(globals())
98 del async_compile
99
100 def call(args):
101     arg0_1, = args
102     args.clear()
103     assert_size_stride(arg0_1, (1, 8, 16), (128, 16, 1))
104     with torch.cuda._DeviceGuard(0):
105         torch.cuda.set_device(0)
106         buf0 = empty_strided_cuda((1, 8, 16), (128, 16, 1), torch.float32)
107         # Topologically Sorted Source Nodes: [mul, pow_1, mul_1, add, mul_2, tanh, add_1, mul_3],
Original ATen: [aten.mul, aten.pow, aten.add, aten.tanh]
108         stream0 = get_raw_stream(0)
109         triton_poi_fused_add_mul_pow_tanh_0.run(arg0_1, buf0, 128, stream=stream0)
110         del arg0_1
111         return (buf0, )
112
113
114 def benchmark_compiled_module(times=10, repeat=10):
115     from torch._dynamo.testing import rand_strided
116     from torch._inductor.utils import print_performance
117     arg0_1 = rand_strided((1, 8, 16), (128, 16, 1), device='cuda:0', dtype=torch.float32)
118     fn = lambda: call([arg0_1])
119     return print_performance(fn, times=times, repeat=repeat)
120
121
122 if __name__ == "__main__":

```

```

123     from torch._inductor.wrapper_benchmark import compiled_module_main
124     compiled_module_main('None', benchmark_compiled_module)

```

이번 Inductor CUDA 실행이 생성한 backend stage 파일 수

ttir	2
ttgir	2
llir	2
ptx	2
cubin	2

10.2 작은 Triton kernel을 직접 실행해 보기

아래 add_one_kernel은 구조를 이해하기 위한 직접 작성한 kernel입니다. 위의 Inductor-generated GELU와 구별합니다.

- `program_id(0)`는 이번 program instance의 번호입니다.
- BLOCK개 element offset을 만들고 유효 범위를 mask합니다.
- `tl.load` → 계산 → `tl.store`를 실행합니다.
- **Program 하나와 CUDA thread 하나는 다릅니다.** Compiler가 element들을 warp/thread에 배치합니다.

N=529, BLOCK=256이면 마지막 program에서 17개 element만 유효합니다. 이 계산을 그림으로 확인하고, 실제 kernel output을 PyTorch와 비교합니다.

```

triton_run = experiment('triton_codegen', elements=529, block=256, nsight=RUN_NCU)
viz.triton_mask(triton_run['elements'], triton_run['block']).show()
show_summary(triton_run['resources'], '이번 compiled kernel의 실제 resource metadata')
from study02.kernels import add_one_kernel
show_source(add_one_kernel.fn)
show_summary({name: value['status'] for name, value in triton_run['stages'].items()}, '이번 실행의 산출물')

```

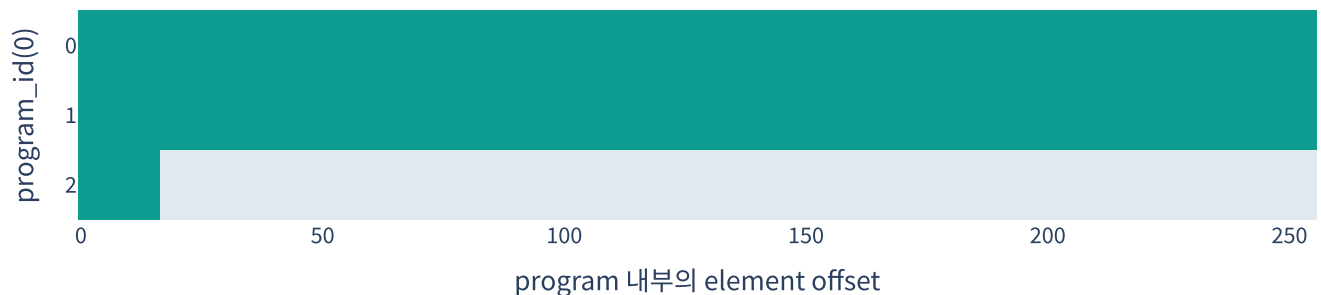
CUDA 실험 실행: triton_codegen · triton_codegen-48d95aaafde4

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/triton_codegen-48d95aaafde4

그림 43

529 elements / BLOCK=256 → 3 programs

현재 launch 인자로 계산 · program 하나는 CUDA thread 하나와 다름



이번 compiled kernel의 실제 resource metadata

registers_per_thread	12
shared_memory_bytes	0
num_warps	4

▼ 실제 소스: /home/riverfog7/Workspace/mlsys_study/study02/kernels.py · add_one_kernel

```
@triton.jit
def add_one_kernel(x_ptr, y_ptr, n_elements, BLOCK: tl.constexpr):
    offsets = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n_elements
    value = tl.load(x_ptr + offsets, mask=mask, other=0.0)
    tl.store(y_ptr + offsets, value + 1.0, mask=mask)
```

이번 실행의 산출물

ttir	generated
ttgir	generated
llir	generated
ptx	generated
cubin	generated
sass	generated

10.3 각 표현에 어떤 정보가 추가되는가?

표현	불 내용
Triton source	Program 단위의 block 계산, pointer-offset-mask
TTIR	Load/store/arithmetic 등을 표현한 높은 수준의 IR. 이름이 붙은 값이 어디서 만들어지는지 읽기
TTGIR	GPU layout과 data의 thread/warp 분배 정보
LLVM IR	Address space와 target 연산으로 더 낮아진 표현
PTX	NVIDIA virtual instruction set
Cubin / SASS	Target GPU용 binary / 그 안의 실제 instruction

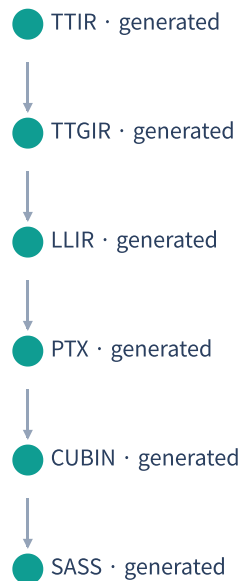
IR의 SSA라는 말은 각 값의 이름이 한 번 정의되고 사용처가 연결되는 표현을 뜻합니다. Source 한 줄이 PTX/SASS 한 명령과 1:1 대응하지는 않습니다. Tanh 같은 함수도 backend에 따라 여러 명령이나 구현으로 바뀔 수 있습니다.

아래 그림의 상태는 CompiledKernel.asm과 disassembler의 실제 결과입니다. 파일이 제공되지 않은 단계는 사유를 표시합니다.

```
stage_labels = [f"{name.upper()} · {info['status']}" for name, info in triton_run['stages'].items()]
viz.concept(stage_labels, '이번 Triton compile의 실제 생성 상태').show()
for name in ['ttir', 'ttgir', 'llir', 'ptx', 'sass']:
    info = triton_run['stages'][name]
    if info['status'] == 'generated':
        show_artifact(triton_run, info['path'], max_lines=60)
    else:
        show_summary({name: info['reason']})
```

이번 Triton compile의 실제 생성 상태

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



▼ 이번 실행의 원본: raw/add_one.ttir

```

1  #loc = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0)
2  module {
3      tt.func public @add_one_kernel(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32}
loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0), %arg1: !tt.ptr<f32> {tt.divisibility =
16 : i32} loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0), %arg2: i32
loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0)) attributes {noinline = false} {
4      %cst = arith.constant dense<1.000000e+00> : tensor<256xf32> loc(#loc1)
5      %cst_0 = arith.constant dense<0.000000e+00> : tensor<256xf32> loc(#loc1)
6      %c256_i32 = arith.constant 256 : i32 loc(#loc1)
7      %0 = tt.get_program_id x : i32 loc(#loc2)
8      %1 = arith.muli %0, %c256_i32 : i32 loc(#loc3)
9      %2 = tt.make_range {end = 256 : i32, start = 0 : i32} : tensor<256xi32> loc(#loc4)
10     %3 = tt.splat %1 : i32 -> tensor<256xi32> loc(#loc5)
11     %4 = arith.addi %3, %2 : tensor<256xi32> loc(#loc5)
12     %5 = tt.splat %arg2 : i32 -> tensor<256xi32> loc(#loc6)
13     %6 = arith.cmpi slt, %4, %5 : tensor<256xi32> loc(#loc6)
14     %7 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<256x!tt.ptr<f32>> loc(#loc7)
15     %8 = tt.addptr %7, %4 : tensor<256x!tt.ptr<f32>>, tensor<256xi32> loc(#loc7)
16     %9 = tt.load %8, %6, %cst_0 : tensor<256x!tt.ptr<f32>> loc(#loc8)
17     %10 = tt.splat %arg1 : !tt.ptr<f32> -> tensor<256x!tt.ptr<f32>> loc(#loc9)
18     %11 = tt.addptr %10, %4 : tensor<256x!tt.ptr<f32>>, tensor<256xi32> loc(#loc9)
19     %12 = arith.addf %9, %cst : tensor<256xf32> loc(#loc10)
20     tt.store %11, %12, %6 : tensor<256x!tt.ptr<f32>> loc(#loc11)
21     tt.return loc(#loc12)
22 } loc(#loc)
23 } loc(#loc)
24 #loc1 = loc(unknown)
25 #loc2 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:28)

```

```

26 #loc3 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:33)
27 #loc4 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:54)
28 #loc5 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:41)
29 #loc6 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":9:21)
30 #loc7 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":10:28)
31 #loc8 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":10:20)
32 #loc9 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:21)
33 #loc10 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:38)
34 #loc11 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:30)
35 #loc12 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:4)

```

▼ 이번 실행의 원본: raw/add_one.ttgir

```

1  #blocked = #ttg.blocked<{sizePerThread = [2], threadsPerWarp = [32], warpsPerCTA = [4], order = [0]}>
2  #loc = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0)
3  module attributes {"ttg.num-ctas" = 1 : i32, "ttg.num-warps" = 4 : i32, ttg.target = "cuda:75",
"ttg.threads-per-warp" = 32 : i32} {
4      tt.func public @add_one_kernel(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32}
loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0), %arg1: !tt.ptr<f32> {tt.divisibility =
16 : i32} loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0), %arg2: i32
loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":7:0)) attributes {noinline = false} {
5          %cst = arith.constant dense<1.000000e+00> : tensor<256xf32, #blocked> loc(#loc1)
6          %c256_i32 = arith.constant 256 : i32 loc(#loc1)
7          %cst_0 = arith.constant dense<0.000000e+00> : tensor<256xf32, #blocked> loc(#loc1)
8          %0 = tt.get_program_id x : i32 loc(#loc2)
9          %1 = arith.muli %0, %c256_i32 : i32 loc(#loc3)
10         %2 = tt.make_range {end = 256 : i32, start = 0 : i32} : tensor<256xi32, #blocked> loc(#loc4)
11         %3 = tt.splat %1 : i32 -> tensor<256xi32, #blocked> loc(#loc5)
12         %4 = arith.addi %3, %2 : tensor<256xi32, #blocked> loc(#loc5)
13         %5 = tt.splat %arg2 : i32 -> tensor<256xi32, #blocked> loc(#loc6)
14         %6 = arith.cmpi slt, %4, %5 : tensor<256xi32, #blocked> loc(#loc6)
15         %7 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<256x!tt.ptr<f32>, #blocked> loc(#loc7)
16         %8 = tt.addptr %7, %4 : tensor<256x!tt.ptr<f32>, #blocked>, tensor<256xi32, #blocked> loc(#loc7)
17         %9 = tt.load %8, %6, %cst_0 : tensor<256x!tt.ptr<f32>, #blocked> loc(#loc8)
18         %10 = tt.splat %arg1 : !tt.ptr<f32> -> tensor<256x!tt.ptr<f32>, #blocked> loc(#loc9)
19         %11 = tt.addptr %10, %4 : tensor<256x!tt.ptr<f32>, #blocked>, tensor<256xi32, #blocked> loc(#loc9)
20         %12 = arith.addf %9, %cst : tensor<256xf32, #blocked> loc(#loc10)
21         tt.store %11, %12, %6 : tensor<256x!tt.ptr<f32>, #blocked> loc(#loc11)
22         tt.return loc(#loc12)
23     } loc(#loc)
24 } loc(#loc)
25 #loc1 = loc(unknown)
26 #loc2 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:28)
27 #loc3 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:33)
28 #loc4 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:54)
29 #loc5 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":8:41)
30 #loc6 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":9:21)
31 #loc7 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":10:28)
32 #loc8 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":10:20)
33 #loc9 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:21)
34 #loc10 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:38)
35 #loc11 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:30)
36 #loc12 = loc("/home/riverfog7/Workspace/mlsys_study/study02/kernels.py":11:4)

```

▼ 이번 실행의 원본: raw/add_one.llir

```

1  ; ModuleID = 'LLVMDialectModule'
2  source_filename = "LLVMDialectModule"
3  target datalayout = "e-p3:32:32-p4:32:32-p5:32:32-p6:32:32-p7:32:32-i64:64-i128:128-v16:16-v32:32-
n16:32:64"
4
5  define ptx_kernel void @add_one_kernel(ptr addrspace(1) %0, ptr addrspace(1) %1, i32 %2, ptr
addrspace(1) readnone captures(none) %3) local_unnamed_addr #0 !dbg !5 {
6      %5 = tail call i32 @llvm.nvvm.read.ptx.sreg.ctaid.x(), !dbg !8

```

```

7    %6 = shl i32 %5, 8, !dbg !9
8    %7 = tail call i32 @llvm.nvvm.read.ptx.sreg.tid.x(), !dbg !10
9    %8 = shl nuw nsw i32 %7, 1, !dbg !10
10   %9 = and i32 %8, 254, !dbg !10
11   %10 = or disjoint i32 %9, %6, !dbg !11
12   %11 = or disjoint i32 %10, 1, !dbg !11
13   %12 = icmp slt i32 %10, %2, !dbg !12
14   %13 = icmp slt i32 %11, %2, !dbg !12
15   %14 = sext i32 %10 to i64, !dbg !13
16   %15 = getelementptr float, ptr @addrspace(1) %0, i64 %14, !dbg !13
17   %16 = sext i32 %11 to i64, !dbg !13
18   %17 = getelementptr float, ptr @addrspace(1) %0, i64 %16, !dbg !13
19   %18 = tail call i32 @asm.sideeffect "mov.u32 $0, $1;\0A\09@$3 ld.global.b32 { $0 }, [ $2 + 0 ];",
"=r,r,l,b"(i32 0, ptr @addrspace(1) %15, i1 %12) #2, !dbg !14
20   %19 = bitcast i32 %18 to float, !dbg !14
21   %20 = tail call i32 @asm.sideeffect "mov.u32 $0, $1;\0A\09@$3 ld.global.b32 { $0 }, [ $2 + 0 ];",
"=r,r,l,b"(i32 0, ptr @addrspace(1) %17, i1 %13) #2, !dbg !14
22   %21 = bitcast i32 %20 to float, !dbg !14
23   %22 = getelementptr float, ptr @addrspace(1) %1, i64 %14, !dbg !15
24   %23 = getelementptr float, ptr @addrspace(1) %1, i64 %16, !dbg !15
25   %24 = fadd float %19, 1.000000e+00, !dbg !16
26   %25 = fadd float %21, 1.000000e+00, !dbg !16
27   %26 = bitcast float %24 to i32, !dbg !17
28   tail call void @asm.sideeffect "@$2 st.global.b32 [ $1 + 0 ], { $0 };", "r,l,b"(i32 %26, ptr
@addrspace(1) %22, i1 %12) #2, !dbg !17
29   %27 = bitcast float %25 to i32, !dbg !17
30   tail call void @asm.sideeffect "@$2 st.global.b32 [ $1 + 0 ], { $0 };", "r,l,b"(i32 %27, ptr
@addrspace(1) %23, i1 %13) #2, !dbg !17
31   ret void, !dbg !18
32 }
33
34 ; Function Attrs: mustprogress nocallback nofree nosync nounwind speculatable willreturn memory(none)
35 declare noundef range(i32 0, 2147483647) i32 @llvm.nvvm.read.ptx.sreg.ctaid.x() #1
36
37 ; Function Attrs: mustprogress nocallback nofree nosync nounwind speculatable willreturn memory(none)
38 declare noundef range(i32 0, 1024) i32 @llvm.nvvm.read.ptx.sreg.tid.x() #1
39
40 attributes #0 = { "nvvm.reqntid"="128" }
41 attributes #1 = { mustprogress nocallback nofree nosync nounwind speculatable willreturn memory(none) }
42 attributes #2 = { nounwind }
43
44 !llvm.dbg.cu = !{!0}
45 !llvm.module.flags = !{!2, !3}
46 !llvm.ident = !{!4}
47
48 !0 = distinct !DICompileUnit(language: DW_LANG_C, file: !1, producer: "triton", isOptimized: true,
runtimeVersion: 0, emissionKind: LineTablesOnly)
49 !1 = !DIFile(filename: "kernels.py", directory: "/home/riverfog7/Workspace/mlsys_study/study02")
50 !2 = !{i32 2, !"Debug Info Version", i32 3}
51 !3 = !{i32 4, !"nvvm-reflect-ftz", i32 1}
52 !4 = !{"clang version 3.8.0 (tags/RELEASE_380/final)"}
53 !5 = distinct !DISubprogram(name: "add_one_kernel", linkageName: "add_one_kernel", scope: !1, file: !1,
line: 7, type: !6, scopeLine: 7, spFlags: DISPFlagDefinition | DISPFlagOptimized, unit: !0)
54 !6 = !DISubroutineType(cc: DW_CC_normal, types: !7)
55 !7 = !{}
56 !8 = !DILocation(line: 8, column: 28, scope: !5)
57 !9 = !DILocation(line: 8, column: 33, scope: !5)
58 !10 = !DILocation(line: 8, column: 54, scope: !5)
59 !11 = !DILocation(line: 8, column: 41, scope: !5)
60 !12 = !DILocation(line: 9, column: 21, scope: !5)
... 전체 66줄: /home/riverfog7/Workspace/mlsys_study/study02/artifacts/triton_codegen-
48d95aaaafde4/raw/add_one.llir

```

▼ 이번 실행의 원본: raw/add_one.ptx

```
1 //
2 // Generated by LLVM NVPTX Back-End
3 //
4
5 .version 8.7
6 .target sm_75
7 .address_size 64
8
9 // .globl      add_one_kernel      // -- Begin function add_one_kernel
10                                     // @add_one_kernel
11 .visible .entry add_one_kernel(
12     .param .u64 .ptr .global .align 1 add_one_kernel_param_0,
13     .param .u64 .ptr .global .align 1 add_one_kernel_param_1,
14     .param .u32 add_one_kernel_param_2,
15     .param .u64 .ptr .global .align 1 add_one_kernel_param_3
16 )
17 .reqntid 128
18 {
19     .reg .pred      %p<5>;
20     .reg .b32       %r<15>;
21     .reg .b64       %rd<8>;
22     .loc    1 7 0                                     // kernels.py:7:0
23 $L__func_begin0:
24     .loc    1 7 0                                     // kernels.py:7:0
25
26 // %bb.0:
27     ld.param.b64    %rd5, [add_one_kernel_param_0];
28     ld.param.b64    %rd6, [add_one_kernel_param_1];
29 $L__tmp0:
30     .loc    1 8 28                                     // kernels.py:8:28
31     mov.u32        %r7, %ctaid.x;
32     .loc    1 8 33                                     // kernels.py:8:33
33     shl.b32        %r8, %r7, 8;
34     ld.param.b32    %r9, [add_one_kernel_param_2];
35     .loc    1 8 54                                     // kernels.py:8:54
36     mov.u32        %r10, %tid.x;
37     shl.b32        %r11, %r10, 1;
38     and.b32        %r12, %r11, 254;
39     .loc    1 8 41                                     // kernels.py:8:41
40     or.b32         %r13, %r12, %r8;
41     or.b32         %r14, %r13, 1;
42     .loc    1 9 21                                     // kernels.py:9:21
43     setp.lt.s32     %p1, %r13, %r9;
44     setp.lt.s32     %p2, %r14, %r9;
45     .loc    1 10 28                                    // kernels.py:10:28
46     mul.wide.s32    %rd7, %r13, 4;
47     add.s64         %rd1, %rd5, %rd7;
48     add.s64         %rd2, %rd1, 4;
49     mov.b32         %r2, 0;
50     .loc    1 10 20                                    // kernels.py:10:20
51 // begin inline asm
52     mov.u32 %r1, %r2;
53     @%p1 ld.global.b32 { %r1 }, [ %rd1 + 0 ];
54 // end inline asm
55 // begin inline asm
56     mov.u32 %r3, %r2;
57     @%p2 ld.global.b32 { %r3 }, [ %rd2 + 0 ];
58 // end inline asm
59     .loc    1 11 21                                    // kernels.py:11:21
60     add.s64        %rd3, %rd6, %rd7;
```

... 전체 174줄: /home/riverfog7/Workspace/mlsys_study/study02/artifacts/triton_codegen-48d95aaafde4/raw/add_one.ptx

▼ 이번 실행의 원본: raw/add_one.sass

```
1      .headerflags      @"EF_CUDA_TEXMODE_UNIFIED EF_CUDA_64BIT_ADDRESS EF_CUDA_SM75
EF_CUDA_VIRTUAL_SM(EF_CUDA_SM75)"
2      .elftype          @"ET_EXEC"
3
4
5      //----- .debug_frame -----
6      .section          .debug_frame,"",@progbits
7      .debug_frame:
8          /*0000*/ .byte   0xff, 0xff, 0xff, 0xff, 0x24, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff,
0xff, 0xff, 0xff
9          /*0010*/ .byte   0xff, 0xff, 0xff, 0xff, 0x03, 0x00, 0x04, 0x7c, 0xff, 0xff, 0xff, 0xff, 0x0f,
0x0c, 0x81, 0x80
10         /*0020*/ .byte   0x80, 0x28, 0x00, 0x08, 0xff, 0x81, 0x80, 0x28, 0x08, 0x81, 0x80, 0x80, 0x28,
0x00, 0x00, 0x00
11         /*0030*/ .byte   0xff, 0xff, 0xff, 0xff, 0x34, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00
12         /*0040*/ .byte   0x00, 0x00, 0x00, 0x00
13         /*0044*/ .dword   add_one_kernel
14         /*004c*/ .byte   0x80, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0x04, 0x00, 0x00, 0x00,
0x04, 0x44, 0x00
15         /*005c*/ .byte   0x00, 0x00, 0x0c, 0x81, 0x80, 0x80, 0x28, 0x00, 0x04, 0x08, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00
16         /*006c*/ .byte   0x00, 0x00, 0x00, 0x00
17
18
19     //----- .debug_line -----
20     .section          .debug_line,"",@progbits
21     .debug_line:
22         /*0000*/ .byte   0x81, 0x00, 0x00, 0x00, 0x02, 0x00, 0x51, 0x00, 0x00, 0x00, 0x01, 0x01, 0xfb,
0x0e, 0x0a, 0x00
23         /*0010*/ .byte   0x01, 0x01, 0x01, 0x01, 0x00, 0x00, 0x00, 0x01, 0x2f, 0x68, 0x6f, 0x6d, 0x65,
0x2f, 0x72, 0x69
24         /*0020*/ .byte   0x76, 0x65, 0x72, 0x66, 0x6f, 0x67, 0x37, 0x2f, 0x57, 0x6f, 0x72, 0x6b, 0x73,
0x70, 0x61, 0x63
25         /*0030*/ .byte   0x65, 0x2f, 0x6d, 0x6c, 0x73, 0x79, 0x73, 0x5f, 0x73, 0x74, 0x75, 0x64, 0x79,
0x2f, 0x73, 0x74
26         /*0040*/ .byte   0x75, 0x64, 0x79, 0x30, 0x32, 0x00, 0x00, 0x6b, 0x65, 0x72, 0x6e, 0x65, 0x6c,
0x73, 0x2e, 0x70
27         /*0050*/ .byte   0x79, 0x00, 0x01, 0xda, 0xba, 0xde, 0xd5, 0x06, 0x9f, 0x03, 0x00, 0x00, 0x00,
0x02
28         /*005e*/ .dword   add_one_kernel
29         /*0066*/ .byte   0x04, 0x01, 0x03, 0x06, 0x01, 0xf0, 0xf1, 0x03, 0x7e, 0x02, 0x20, 0x01, 0x03,
0x01, 0x02, 0xc0
30         /*0076*/ .byte   0x00, 0x01, 0xf0, 0xed, 0xf1, 0xee, 0xf1, 0xee, 0xf0, 0x02, 0x90, 0x01, 0x00,
0x01, 0x01
31
32
33     //----- .nv_debug_line_sass -----
34     .section          .nv_debug_line_sass,"",@progbits
35     .nv_debug_line_sass:
36         /*0000*/ .byte   0x60, 0x00, 0x00, 0x00, 0x02, 0x00, 0x10, 0x00, 0x00, 0x00, 0x01, 0x01, 0xfb,
0x0e, 0x0a, 0x00
37         /*0010*/ .byte   0x01, 0x01, 0x01, 0x01, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x09, 0x02
38         /*001d*/ .dword   add_one_kernel
39         /*0025*/ .byte   0x04, 0x00, 0x03, 0x11, 0x01, 0x03, 0x12, 0x02, 0x10, 0x01, 0x03, 0x0b, 0x02,
0x10, 0x01, 0xf5
40         /*0035*/ .byte   0x03, 0x6a, 0x02, 0x10, 0x01, 0xf5, 0xf0, 0xf1, 0xf2, 0xf3, 0x03, 0x7a, 0x02,
0x10, 0x01, 0x03
41         /*0045*/ .byte   0x0c, 0x02, 0x10, 0x01, 0x03, 0x77, 0x02, 0x10, 0x01, 0x03, 0x10, 0x02, 0x10,
```

```

0x01, 0xec, 0xf5
42      /*0055*/ .byte  0xf3, 0xf2, 0x03, 0x7a, 0x02, 0x10, 0x01, 0xf5, 0xf2, 0x02, 0xc0, 0x00, 0x00,
0x01, 0x01
43
44
45 //----- .nv_debug_ptx_txt -----
46 .section      .nv_debug_ptx_txt,"",@progbits
47 .nv_debug_ptx_txt:
48      /*0000*/ .byte  0x00, 0x00, 0x00, 0x00, 0x2e, 0x76, 0x65, 0x72, 0x73, 0x69, 0x6f, 0x6e, 0x20,
0x38, 0x2e, 0x37
49      /*0010*/ .byte  0x00, 0x2e, 0x74, 0x61, 0x72, 0x67, 0x65, 0x74, 0x20, 0x73, 0x6d, 0x5f, 0x37,
0x35, 0x00, 0x2e
50      /*0020*/ .byte  0x61, 0x64, 0x64, 0x72, 0x65, 0x73, 0x73, 0x5f, 0x73, 0x69, 0x7a, 0x65, 0x20,
0x36, 0x34, 0x00
51      /*0030*/ .byte  0x00, 0x00, 0x00, 0x2e, 0x76, 0x69, 0x73, 0x69, 0x62, 0x6c, 0x65, 0x20, 0x2e,
0x65, 0x6e, 0x74
52      /*0040*/ .byte  0x72, 0x79, 0x20, 0x61, 0x64, 0x64, 0x5f, 0x6f, 0x6e, 0x65, 0x5f, 0x6b, 0x65,
0x72, 0x6e, 0x65
53      /*0050*/ .byte  0x6c, 0x28, 0x00, 0x2e, 0x70, 0x61, 0x72, 0x61, 0x6d, 0x20, 0x2e, 0x75, 0x36,
0x34, 0x20, 0x2e
54      /*0060*/ .byte  0x70, 0x74, 0x72, 0x20, 0x2e, 0x67, 0x6c, 0x6f, 0x62, 0x61, 0x6c, 0x20, 0x2e,
0x61, 0x6c, 0x69
55      /*0070*/ .byte  0x67, 0x6e, 0x20, 0x31, 0x20, 0x61, 0x64, 0x64, 0x5f, 0x6f, 0x6e, 0x65, 0x5f,
0x6b, 0x65, 0x72
56      /*0080*/ .byte  0x6e, 0x65, 0x6c, 0x5f, 0x70, 0x61, 0x72, 0x61, 0x6d, 0x5f, 0x30, 0x2c, 0x00,
0x2e, 0x70, 0x61
57      /*0090*/ .byte  0x72, 0x61, 0x6d, 0x20, 0x2e, 0x75, 0x36, 0x34, 0x20, 0x2e, 0x70, 0x74, 0x72,
0x20, 0x2e, 0x67
58      /*00a0*/ .byte  0x6c, 0x6f, 0x62, 0x61, 0x6c, 0x20, 0x2e, 0x61, 0x6c, 0x69, 0x67, 0x6e, 0x20,
0x31, 0x20, 0x61
59      /*00b0*/ .byte  0x64, 0x64, 0x5f, 0x6f, 0x6e, 0x65, 0x5f, 0x6b, 0x65, 0x72, 0x6e, 0x65, 0x6c,
0x5f, 0x70, 0x61
60      /*00c0*/ .byte  0x72, 0x61, 0x6d, 0x5f, 0x31, 0x2c, 0x00, 0x2e, 0x70, 0x61, 0x72, 0x61, 0x6d,
0x20, 0x2e, 0x75
... 전체 371줄: /home/riverfog7/Workspace/mlsys_study/study02/artifacts/triton_codegen-
48d95aaafde4/raw/add_one.sass

```

확인해 보기 — Register 수와 shared-memory 크기를 알면 occupancy도 바로 알 수 있을까요?

▼ 해설 펼치기

충분하지 않습니다. GPU의 자원 한계, block/warp 배치와 실행 조건도 필요합니다. Resource metadata와 실제 occupancy 측정은 구별합니다. RUN_NCU=True는 별도 Nsight 실험으로 hardware counter를 요청하며 권한이 없으면 사유를 남깁니다.

11. 한 GPT block을 source부터 kernel까지 따라가기

원문: §13 · pp. 67–71 · 목차로

11.1 MLP에서 먼저 external 경계를 읽는다

MLP에는 GEMM → GELU → GEMM이 있습니다. GEMM 뒤의 bias/activation이 epilogue로 합쳐질 수도 있고, 외부 library 경계 때문에 별도 kernel로 남을 수도 있습니다. 아래 결과는 이 GPU:shape:dtype에서의 선택입니다.

```

mlp_ir = experiment('inductor', workload='mlp', **MLP_SHAPE)
mlp_capture = mlp_ir['captures'][0]
viz.dag(mlp_capture['post'], '현재 MLP의 scheduler groups / external nodes').show()
mlp_calls = mlp_capture['wrapper']['calls']
fig = go.Figure(go.Bar(x=list(range(1, len(mlp_calls)+1)), y=[1]*len(mlp_calls),
                        text=[c['kind'] for c in mlp_calls],
                        customdata=[c['name'] for c in mlp_calls],
                        marker_color=[viz.TEAL if c['kind']=='generated' else viz.ORANGE for c in

```

```
mlp_calls],
        hovertemplate='wrapper 호출 %x}<br>{%customdata}<extra></extra>'))
fig.update_xaxes(title='wrapper의 소스상 호출 순서', dtick=1); fig.update_yaxes(visible=False)
viz.style(fig, 'Generated body와 external 호출의 경계', '이번 wrapper의 정적 호출 목록 · GPU launch 수 자체와 구
별', 340).show()
```

CUDA 실험 실행: inductor · inductor-3f58222f26a

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/inductor-3f58222f26a

그림 45

현재 MLP의 scheduler groups / external nodes

현재 capture · 화살표=값의 의존성, 시간축 아님

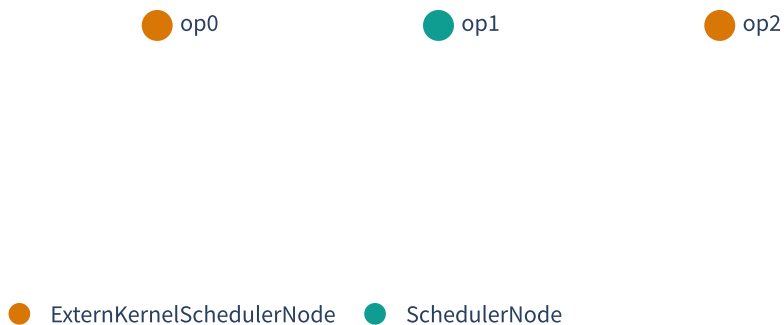
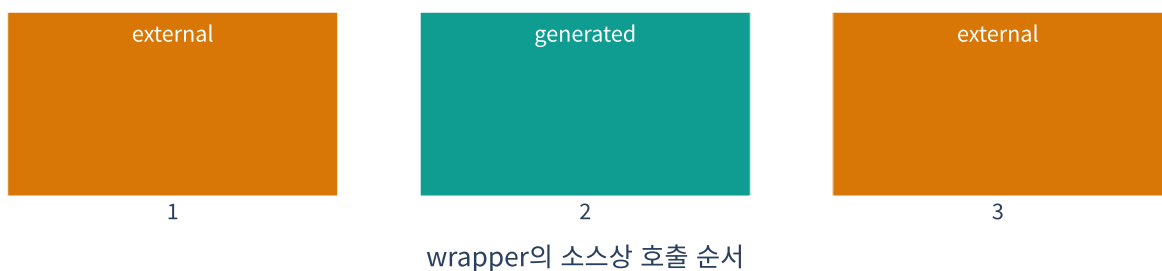


그림 46

Generated body와 external 호출의 경계

이번 wrapper의 정적 호출 목록 · GPU launch 수 자체와 구별



11.2 작은 block inference: manual attention

이제 embedding과 LM head를 포함한 한 block입니다. 이 절과 뒤의 성능 실험은 모두 $B=4, T=128, C=256, H=4, V=1024$ 를 사용해 wrapper와 실제 CUDA timeline을 같은 조건으로 읽습니다. 앞의 모델 구조 설명은 읽기 쉬운 작은 shape를 사용했습니다. Python의 ModuleList loop와 `targets is None` 분기는 현재 호출의 경로로 specialize됩니다. Manual attention은 QK matmul, softmax, AV로 펼쳐져 있습니다.

첫 graph의 입력에는 token뿐 아니라 모델 parameter도 나타날 수 있습니다. Node를 hover하고, 큰 graph의 구간 메뉴를 이동하면서 embedding → QKV → attention → MLP → 마지막 token 선택 → LM head를 찾아보세요.

```

block_ir = experiment('inductor', workload='block1', attention='manual', **BLOCK_SHAPE)
block_capture = block_ir['captures'][0]
viz.dag(block_ir['graphs']['input'][0], '현재 한 block의 functional ATen FX').show()
viz.dag(block_capture['post'], '현재 한 block의 scheduled groups').show()
viz.lifetimes(block_capture['wrapper']).show()
show_summary({'output shape': block_ir['correctness']['output_shape'],
              'wrapper 호출': len(block_capture['wrapper']['calls']),
              'generated 호출': sum(c['kind']=='generated' for c in block_capture['wrapper']['calls']),
              'external 호출': sum(c['kind']=='external' for c in block_capture['wrapper']['calls'])},
              '이번 CUDA block의 실행 계획')

```

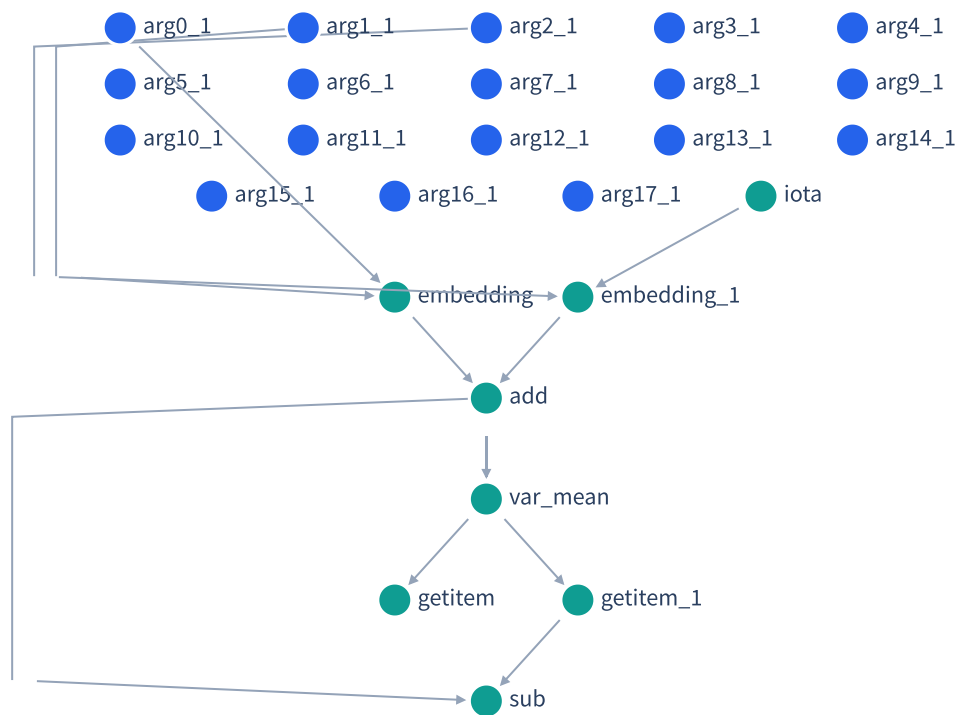
CUDA 실험 실행: inductor · inductor-63f0066fbf2d

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/inductor-63f0066fbf2d

그림 47 · 구간 1/5

현재 한 block의 functional ATen FX

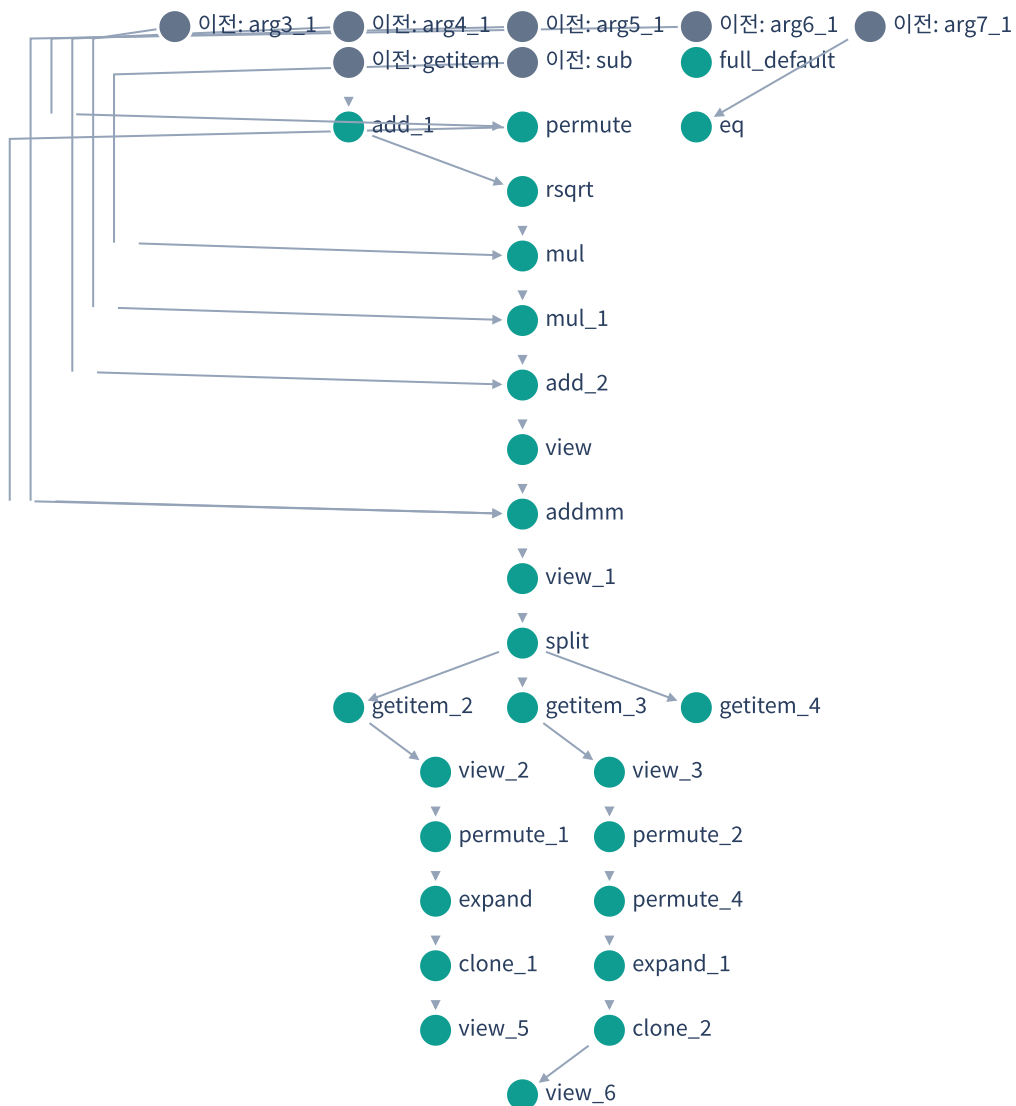
현재 capture · node 1-26/118 · 회색=이전 구간 입력



● placeholder ● call_function

현재 한 block의 functional ATen FX

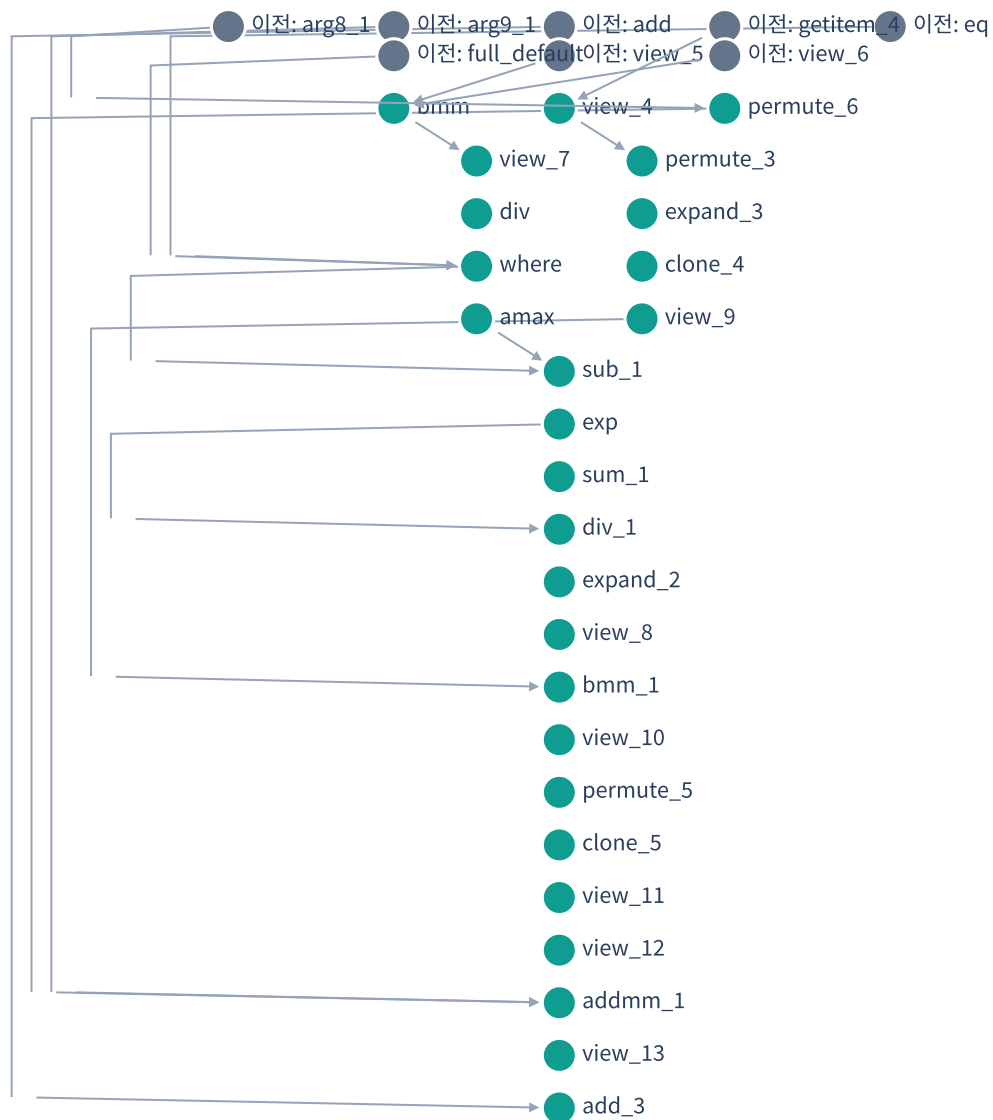
현재 capture · node 27-52/118 · 회색=이전 구간 입력



● boundary ● call_function

현재 한 block의 functional ATen FX

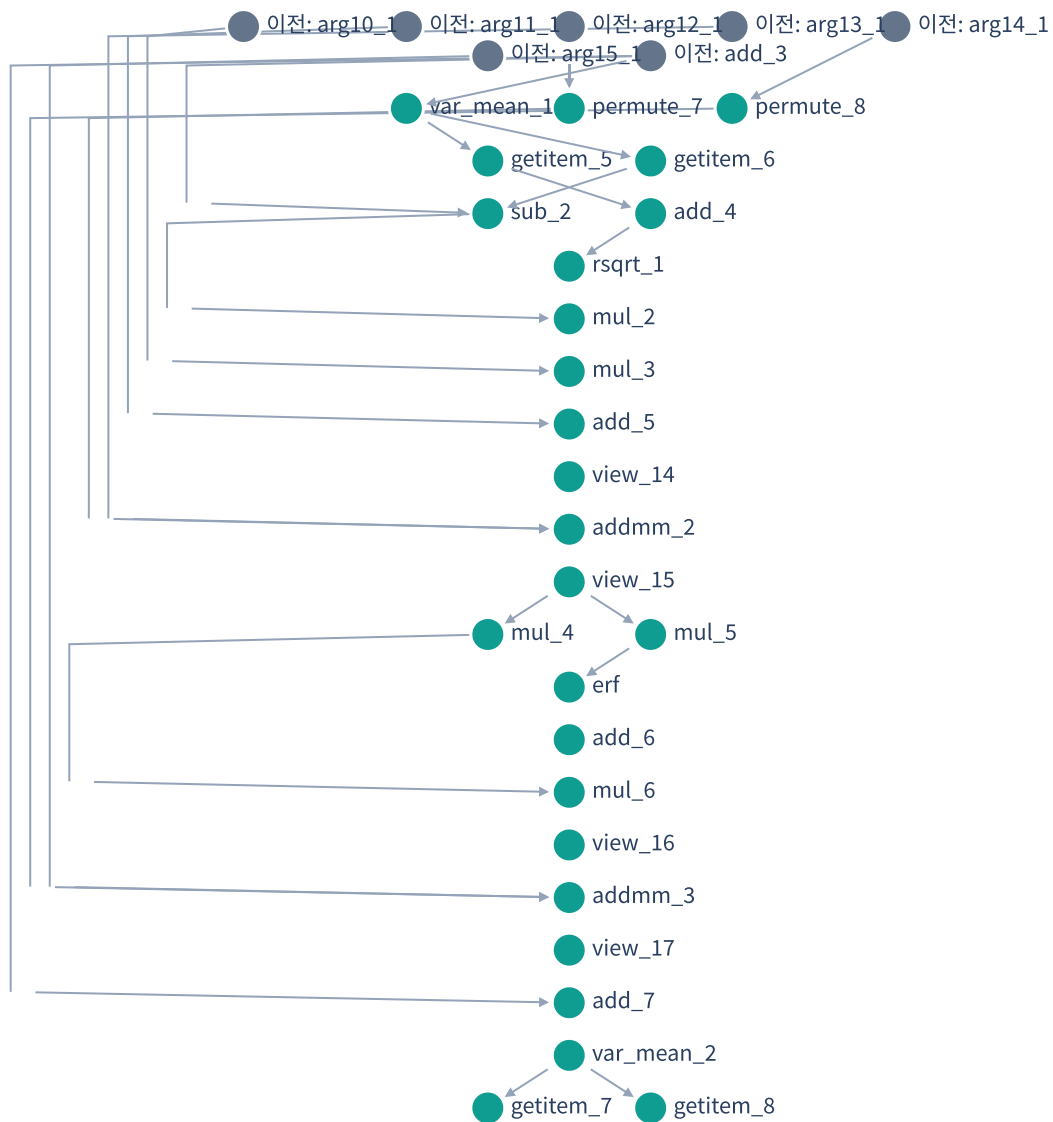
현재 capture · node 53-78/118 · 회색=이전 구간 입력



● boundary ● call_function

현재 한 block의 functional ATen FX

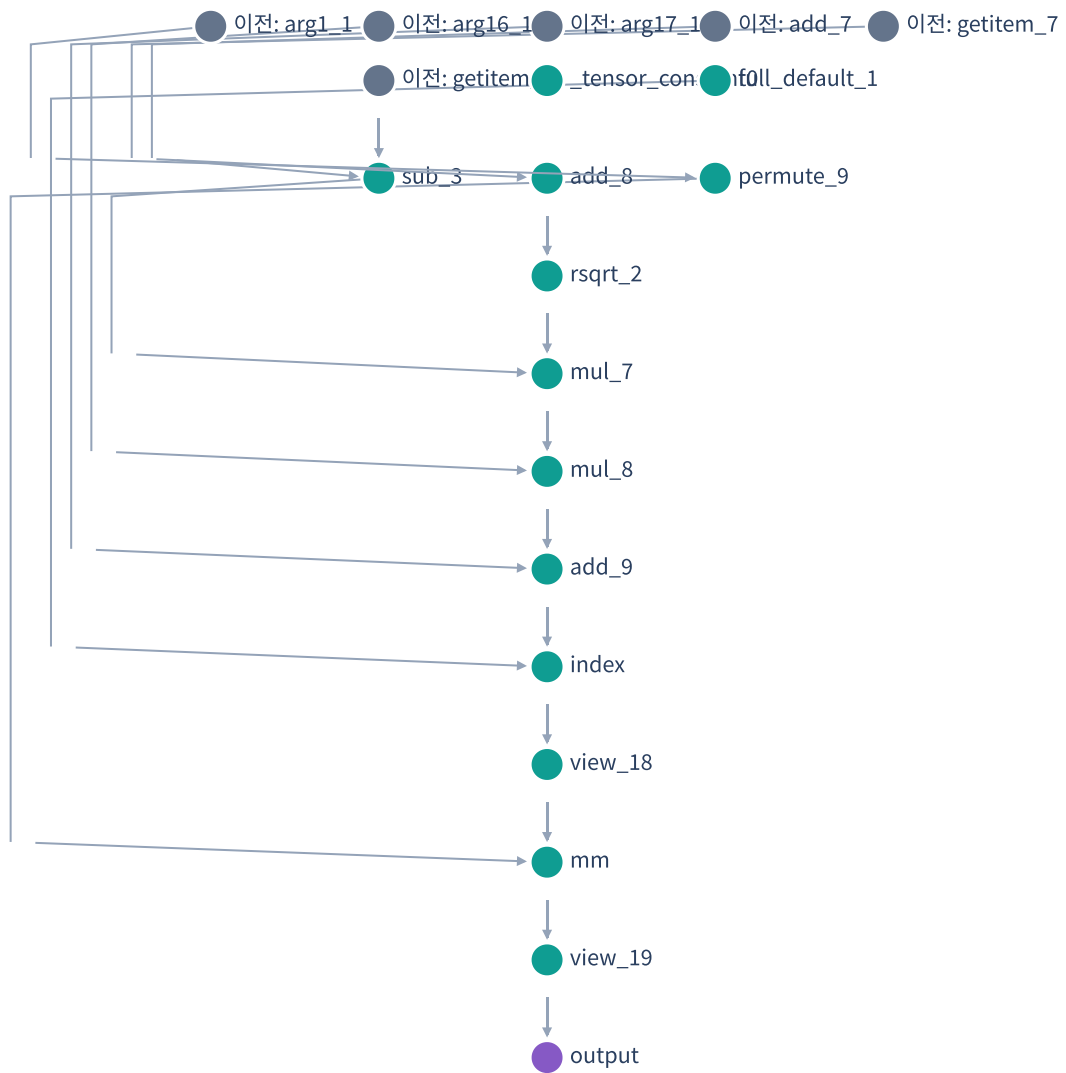
현재 capture · node 79-104/118 · 회색=이전 구간 입력



● boundary ● call_function

현재 한 block의 functional ATen FX

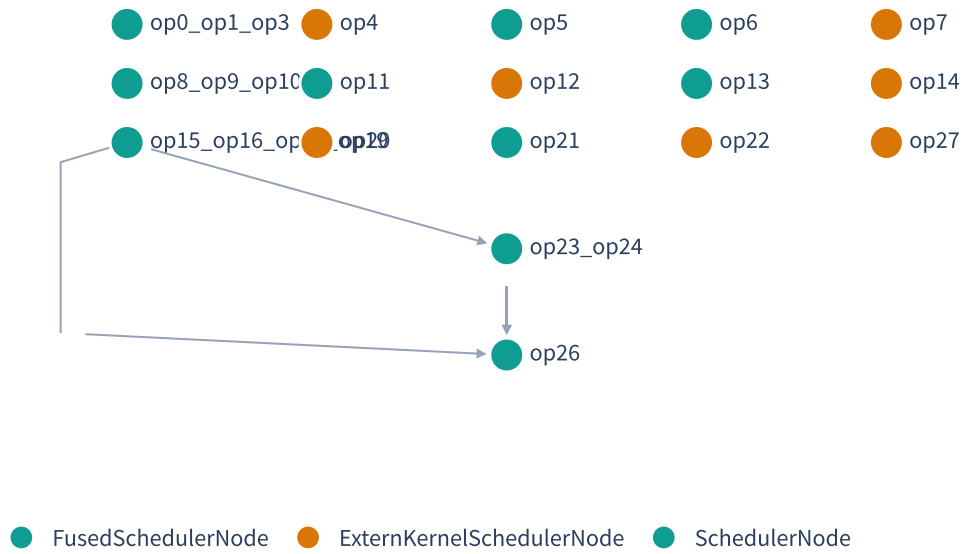
현재 capture · node 105-118/118 · 회색=이전 구간 입력



● boundary ● get_attr ● call_function ● output

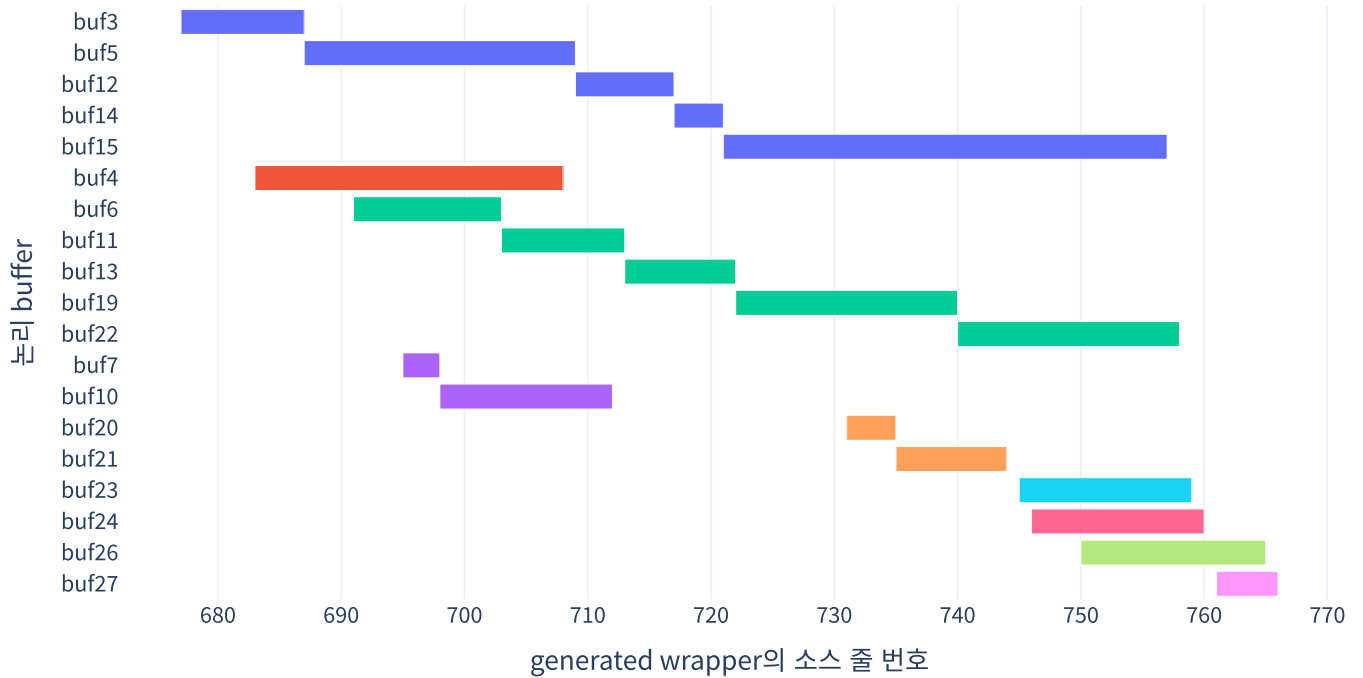
현재 한 block의 scheduled groups

현재 capture · 화살표=값의 의존성, 시간축 아님



Buffer lifetime / storage reuse

현재 wrapper · 정적 참조 구간이며 GPU wall time이 아님



storage buf3 storage buf4 storage buf6 storage buf7 storage buf20
storage buf23 storage buf24 storage buf26 storage buf27

이번 CUDA block의 실행 계획

output shape

[4, 1, 1024]

wrapper 호출	17
generated 호출	10
external 호출	7

11.3 SDPA와 training으로 바꾸면?

SDPA API는 backend 선택을 PyTorch에 맡깁니다. SDPA를 썼다는 사실만으로 FlashAttention이라고 부르지 않습니다. 실제 선택된 연산-kernel을 현재 artifact와 profiler에서 확인합니다.

Training 호출은 full logits와 loss를 계산하고 backward를 수행합니다. 아래 training 실험은 output뿐 아니라 parameter gradient5 eager와 비교합니다. Inference의 wrapper 개수를 training 전체의 kernel 수라고 읽지 않도록 각각의 capture를 표시합니다.

```
sdpa_ir = experiment('inductor', workload='block1', attention='sdpa', **BLOCK_SHAPE)
training_ir = experiment('inductor', workload='block1', attention='manual', training=True, **BLOCK_SHAPE)
viz.dag(sdpa_ir['graphs']['input'][0], 'SDPA block의 실제 ATen graph').show()
if len(training_ir['graphs']['input']) > 1:
    viz.dag(training_ir['graphs']['input'][1], 'Training: 두 번째 Inductor graph (backward capture 경로 확인)').show()
show_summary(training_ir['correctness'], 'Training output / gradient 비교')
show_summary({f'capture {i}': c['directory'] for i, c in enumerate(training_ir['captures'])},
             '현재 training capture의 forward/backward 디렉터리')
```

CUDA 실험 실행: inductor · inductor-3ac83887136a

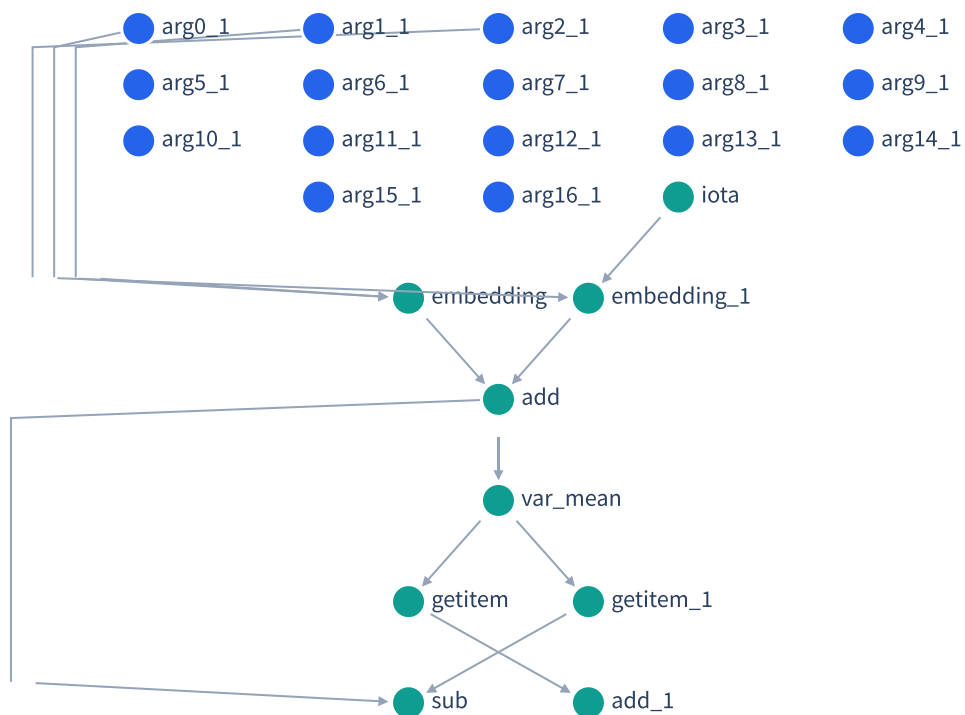
완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/inductor-3ac83887136a

CUDA 실험 실행: inductor · inductor-c47015e2ea19

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/inductor-c47015e2ea19

SDPA block의 실제 ATen graph

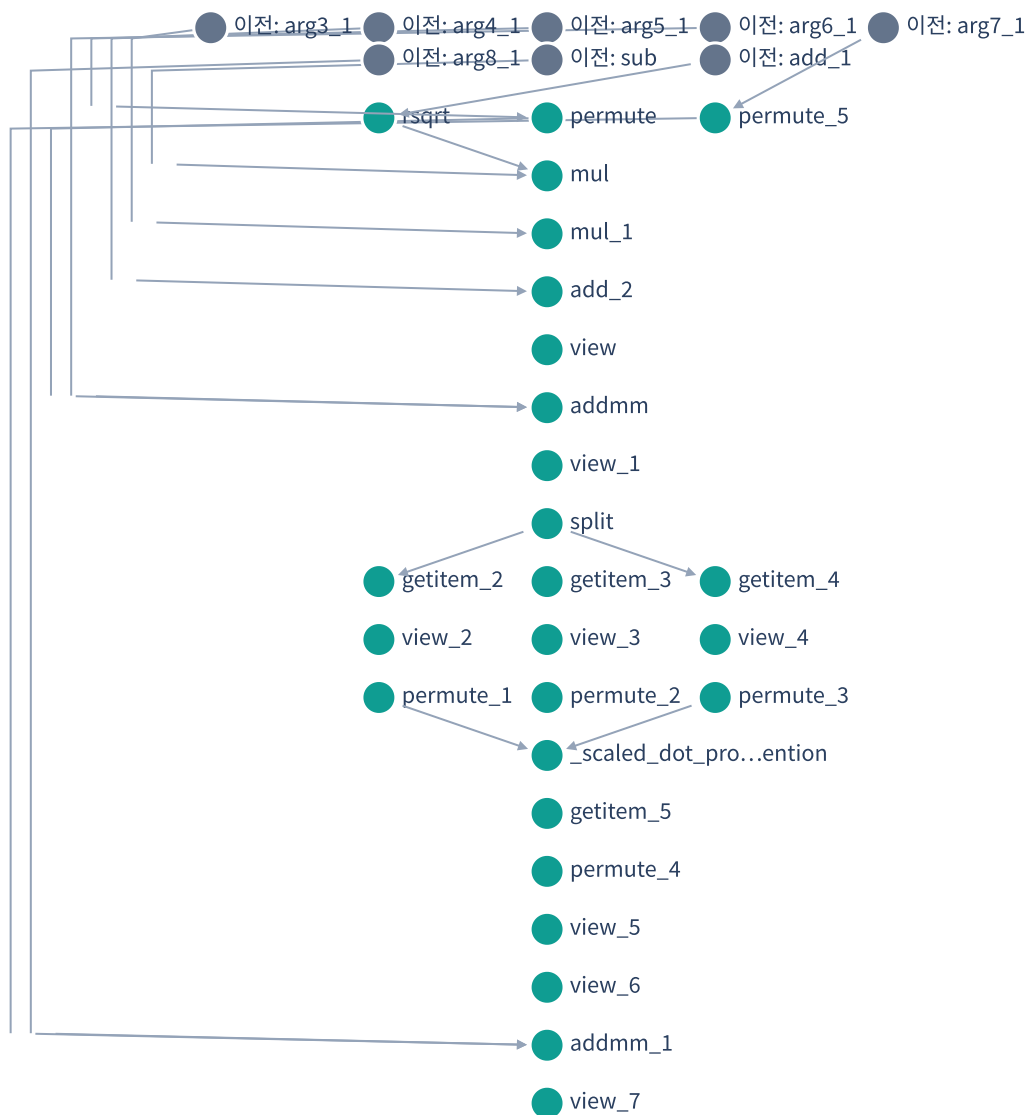
현재 capture · node 1-26/93 · 회색=이전 구간 입력



● placeholder ● call_function

SDPA block의 실제 ATen graph

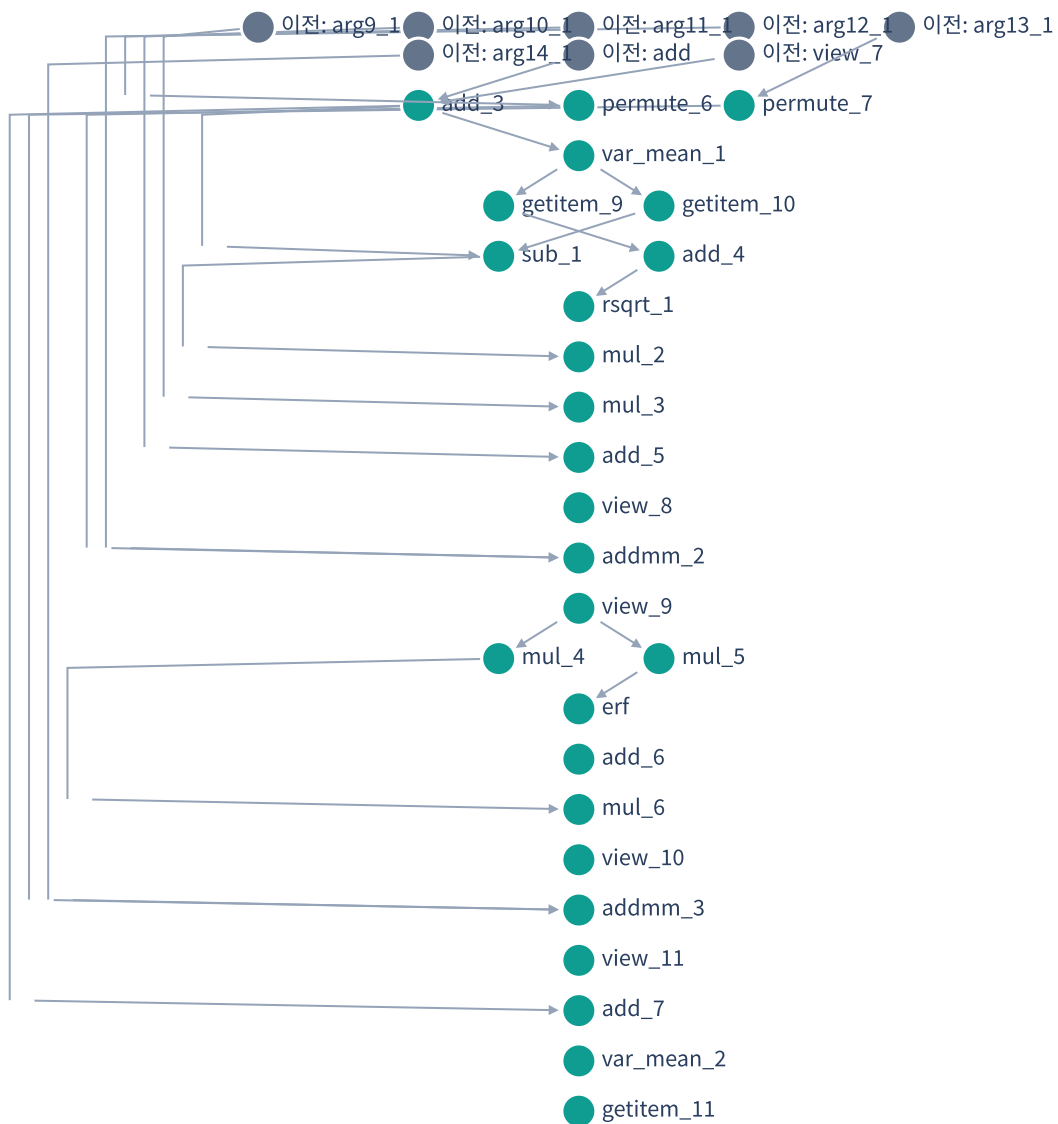
현재 capture · node 27-52/93 · 회색=이전 구간 입력



● boundary ● call_function

SDPA block의 실제 ATen graph

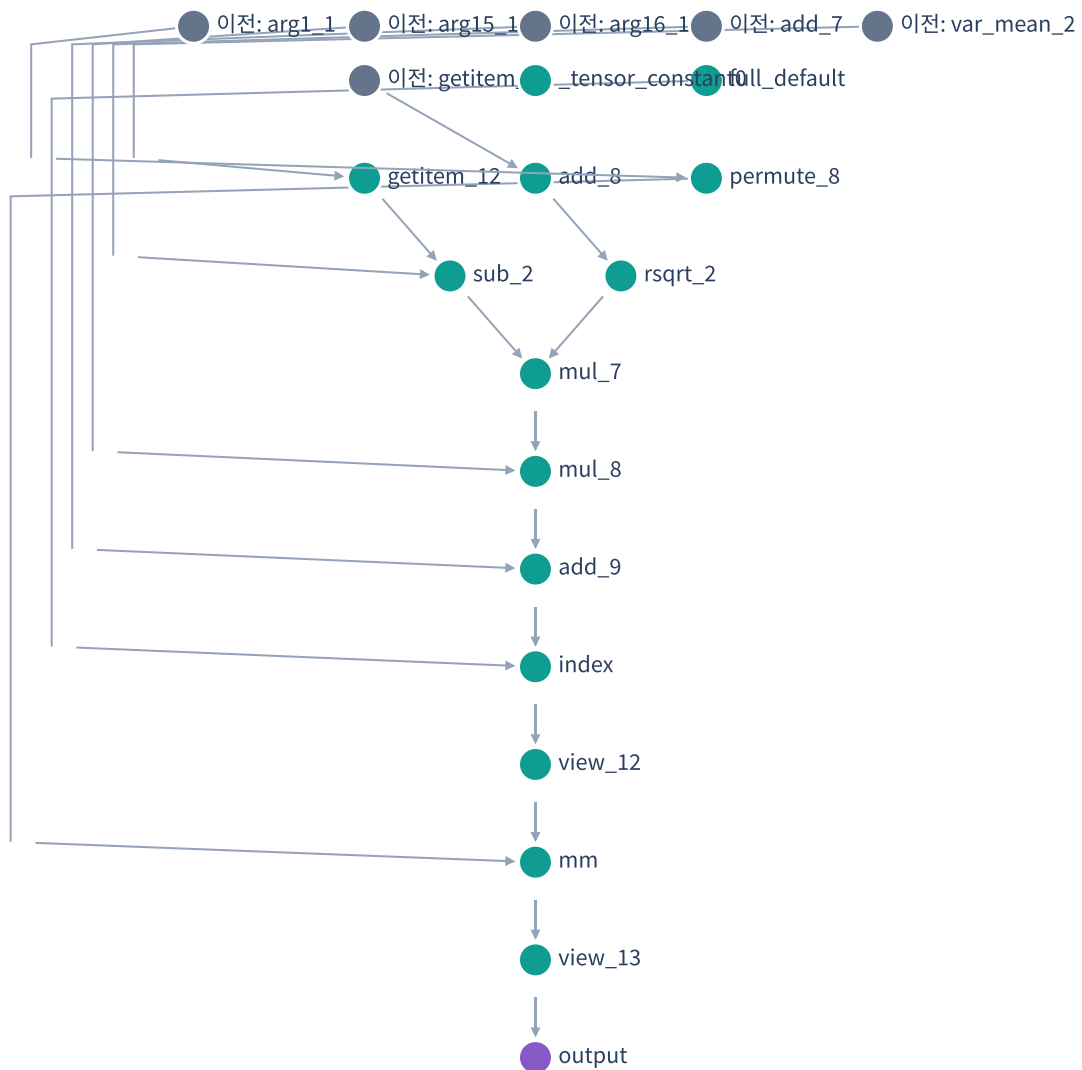
현재 capture · node 53-78/93 · 회색=이전 구간 입력



● boundary ● call_function

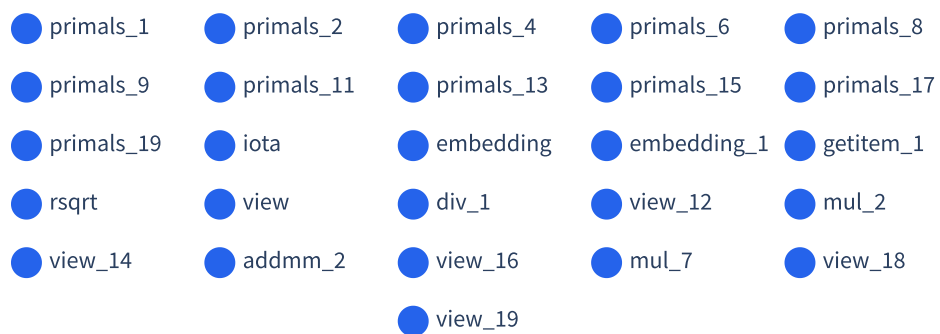
SDPA block의 실제 ATen graph

현재 capture · node 79-93/93 · 회색=이전 구간 입력



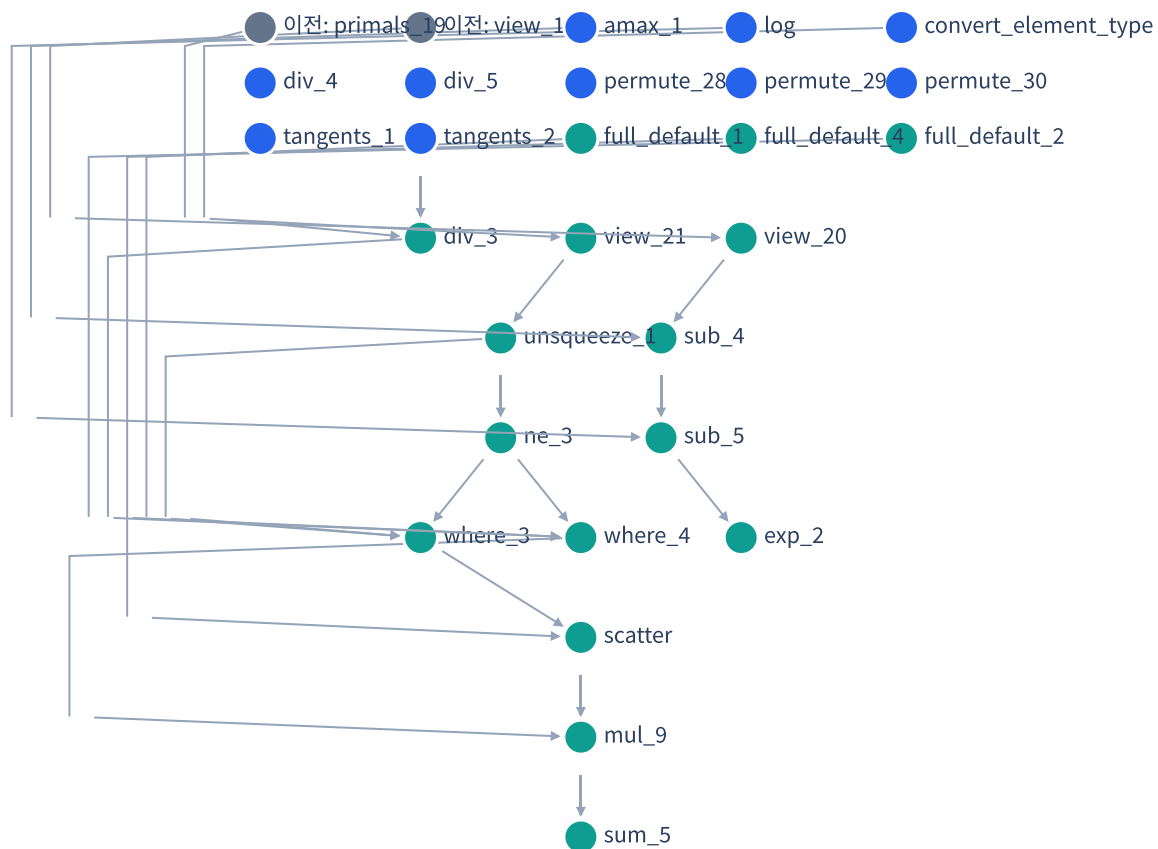
Training: 두 번째 Inductor graph (backward capture 경로 확인)

현재 capture · node 1-26/200 · 회색=이전 구간 입력



Training: 두 번째 Inductor graph (backward capture 경로 확인)

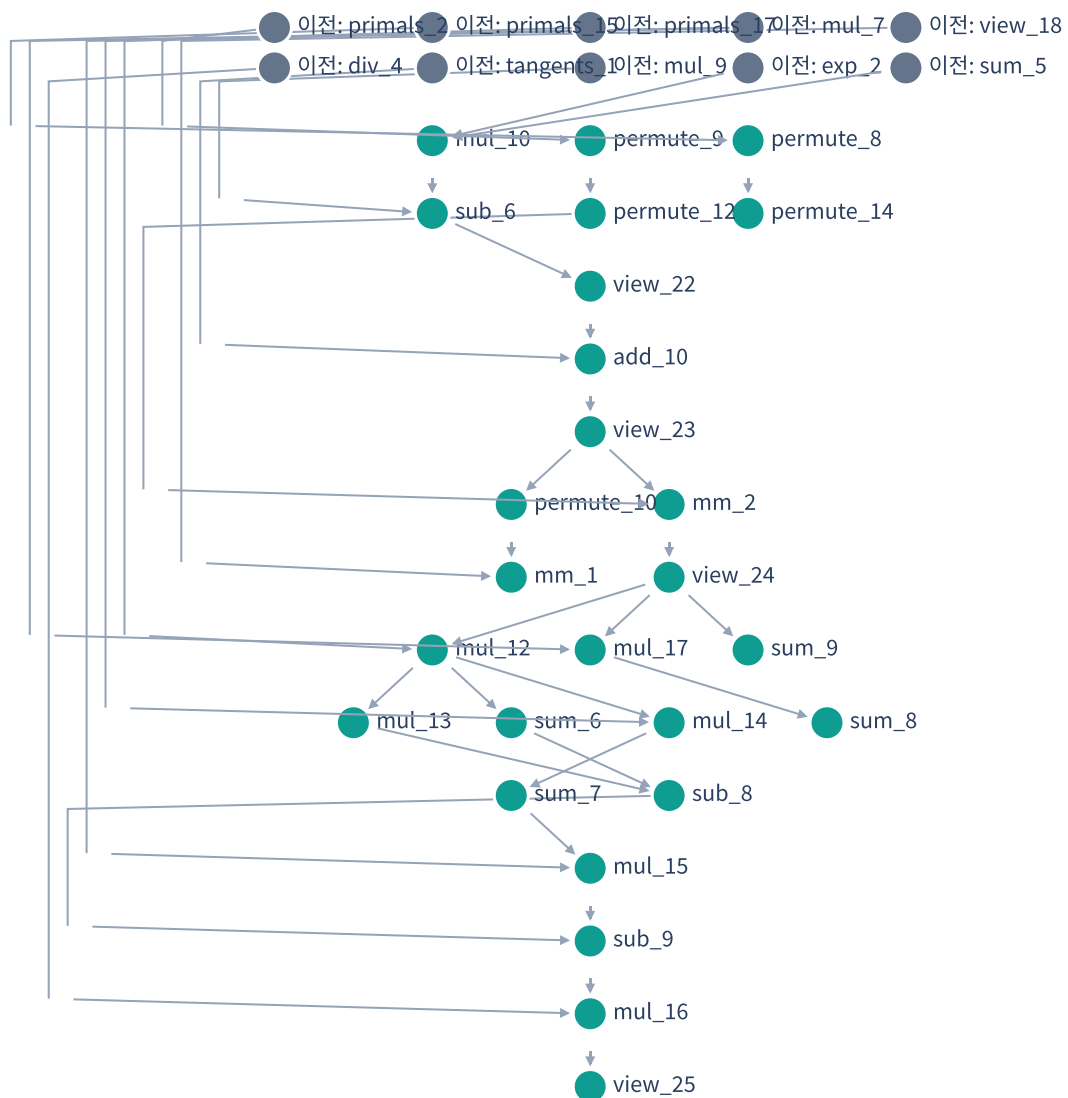
현재 capture · node 27-52/200 · 회색=이전 구간 입력



● boundary ● placeholder ● call_function

Training: 두 번째 Inductor graph (backward capture 경로 확인)

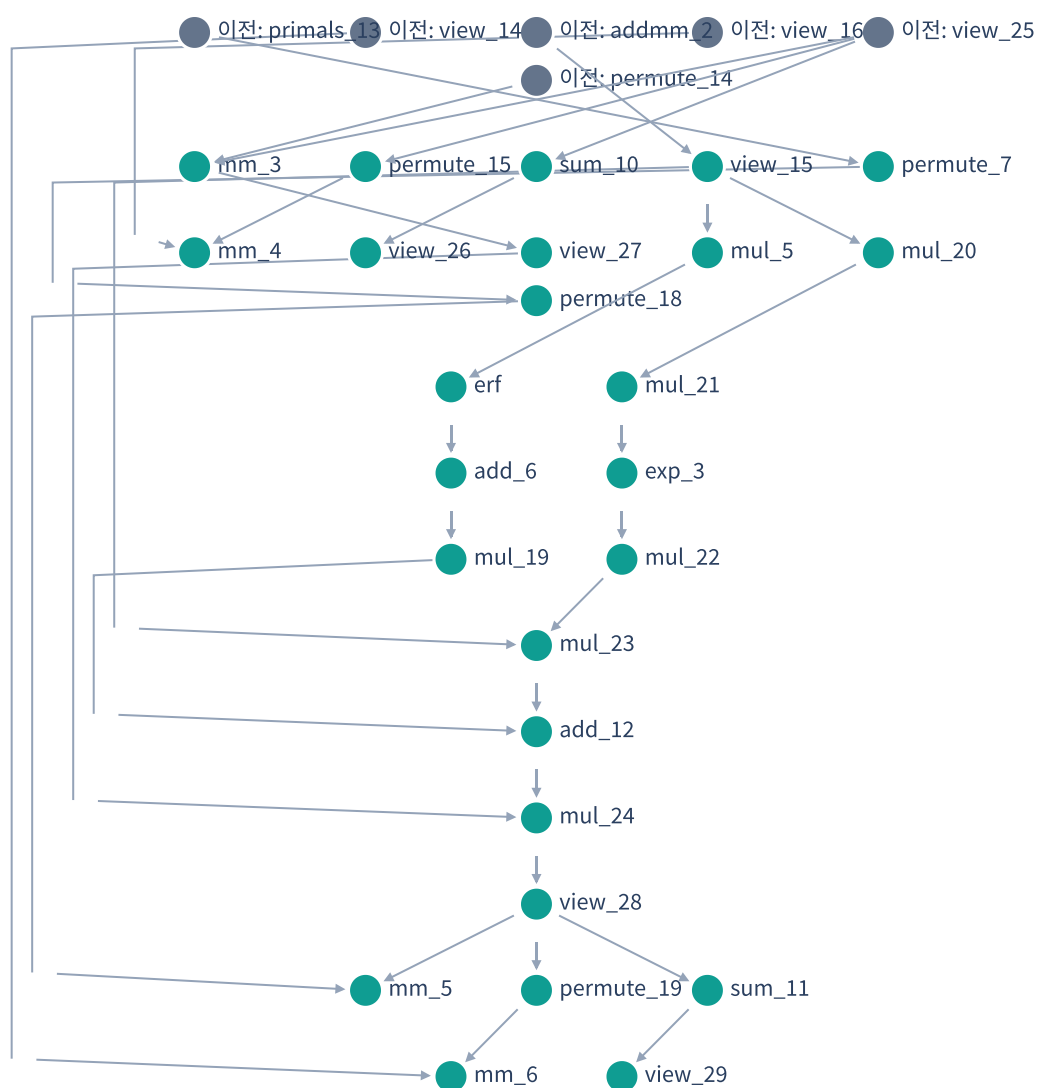
현재 capture · node 53-78/200 · 회색=이전 구간 입력



● boundary ● call_function

Training: 두 번째 Inductor graph (backward capture 경로 확인)

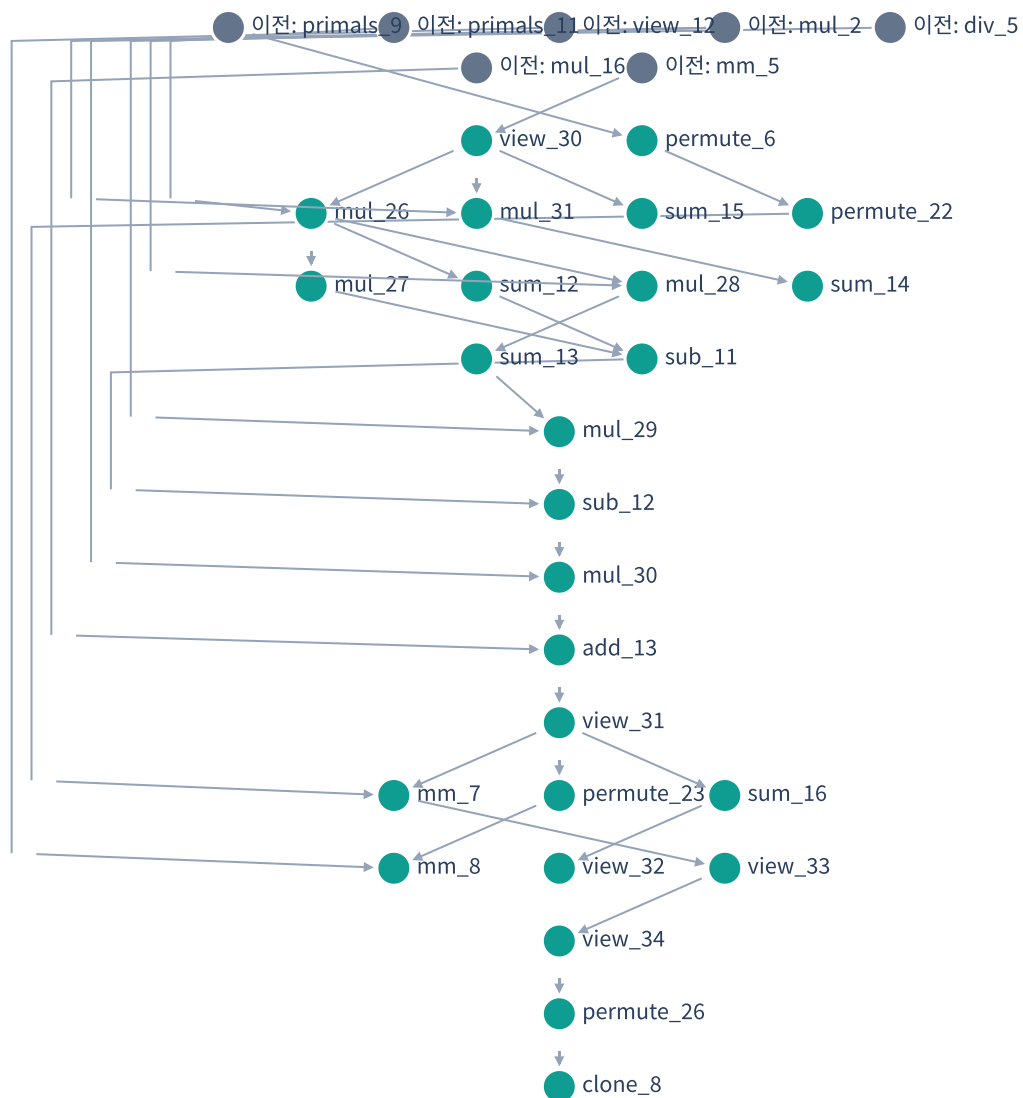
현재 capture · node 79-104/200 · 회색=이전 구간 입력



● boundary ● call_function

Training: 두 번째 Inductor graph (backward capture 경로 확인)

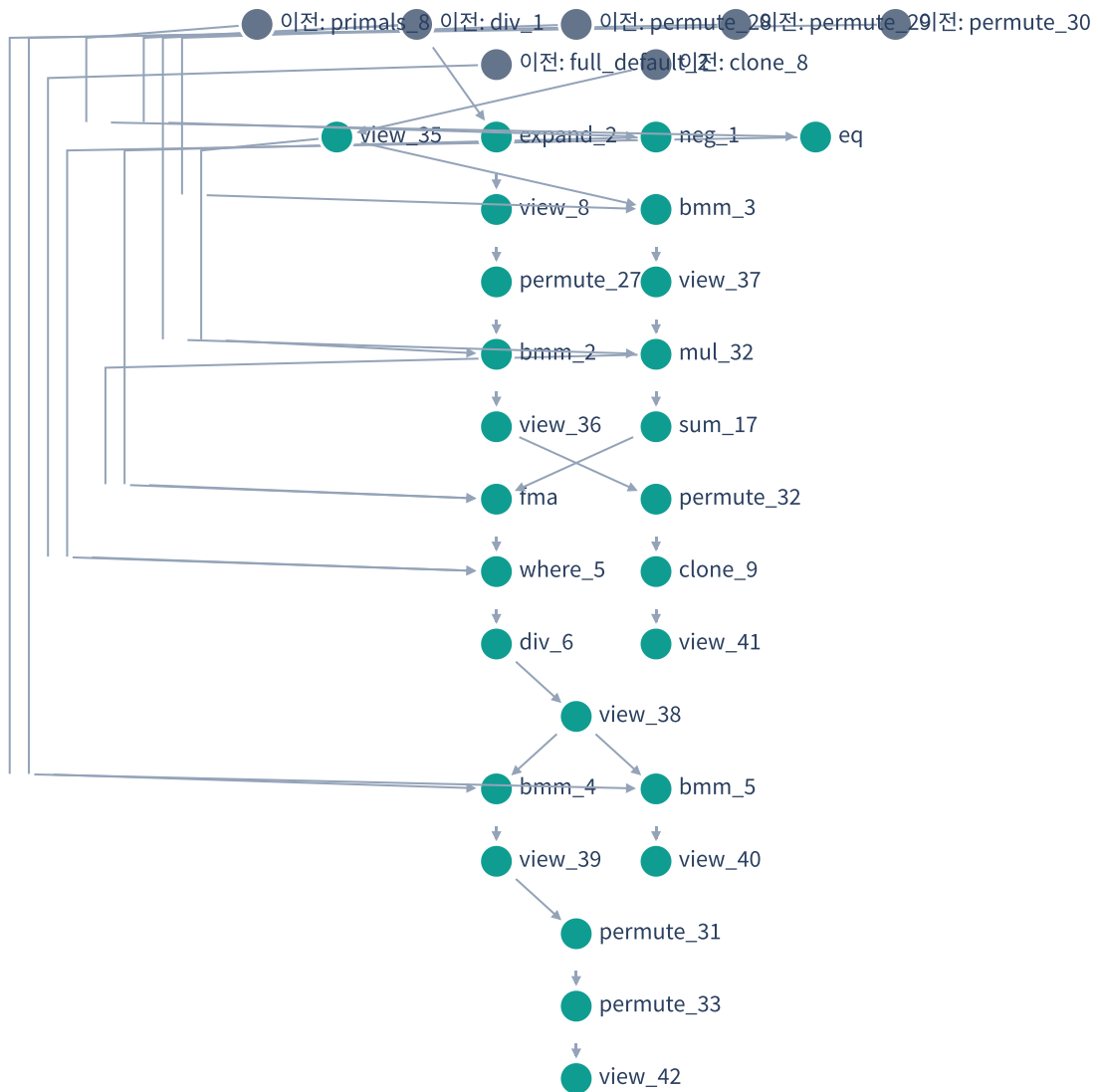
현재 capture · node 105-130/200 · 회색=이전 구간 입력



● boundary ● call_function

Training: 두 번째 Inductor graph (backward capture 경로 확인)

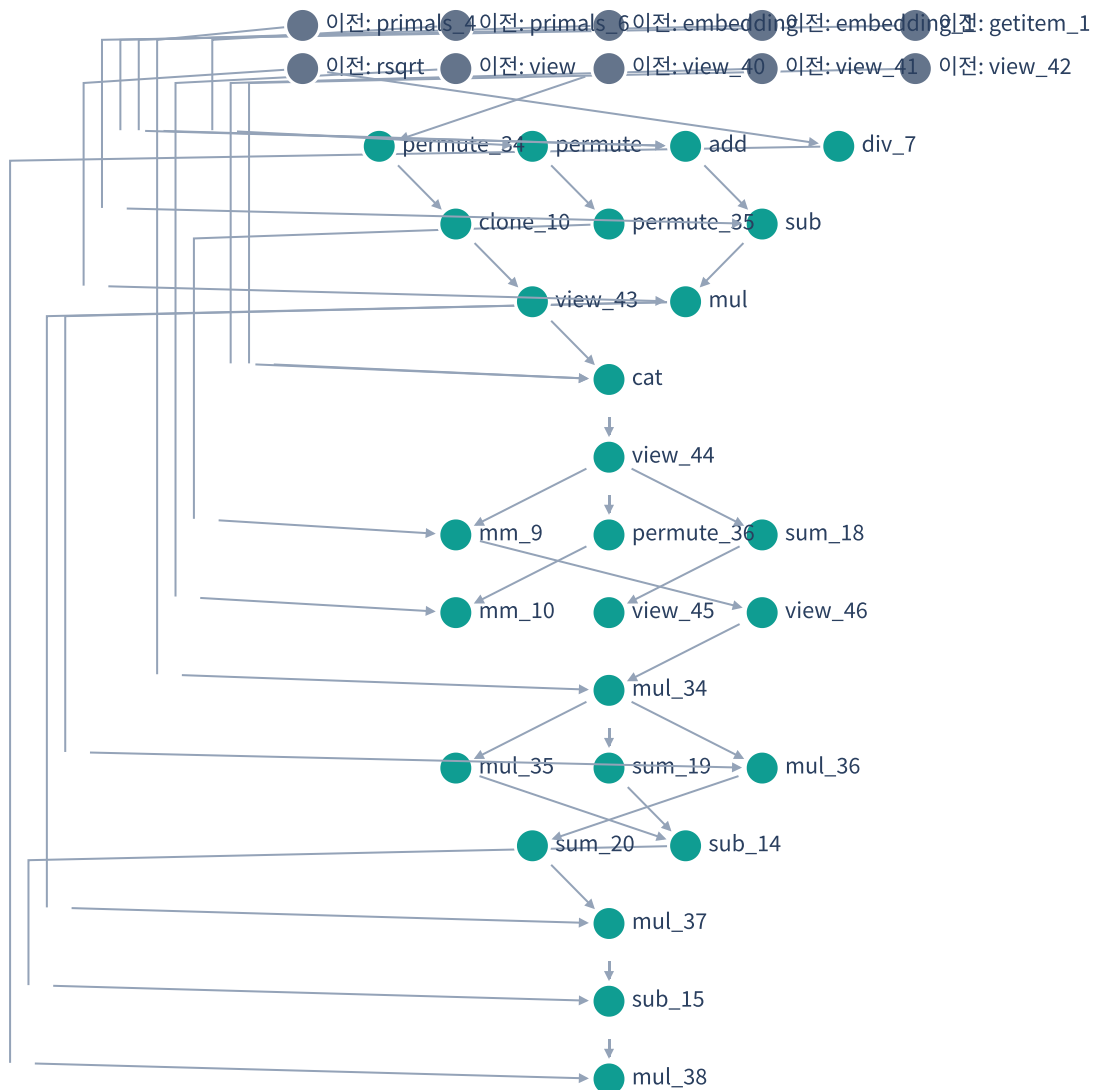
현재 capture · node 131-156/200 · 회색=이전 구간 입력



● boundary ● call_function

Training: 두 번째 Inductor graph (backward capture 경로 확인)

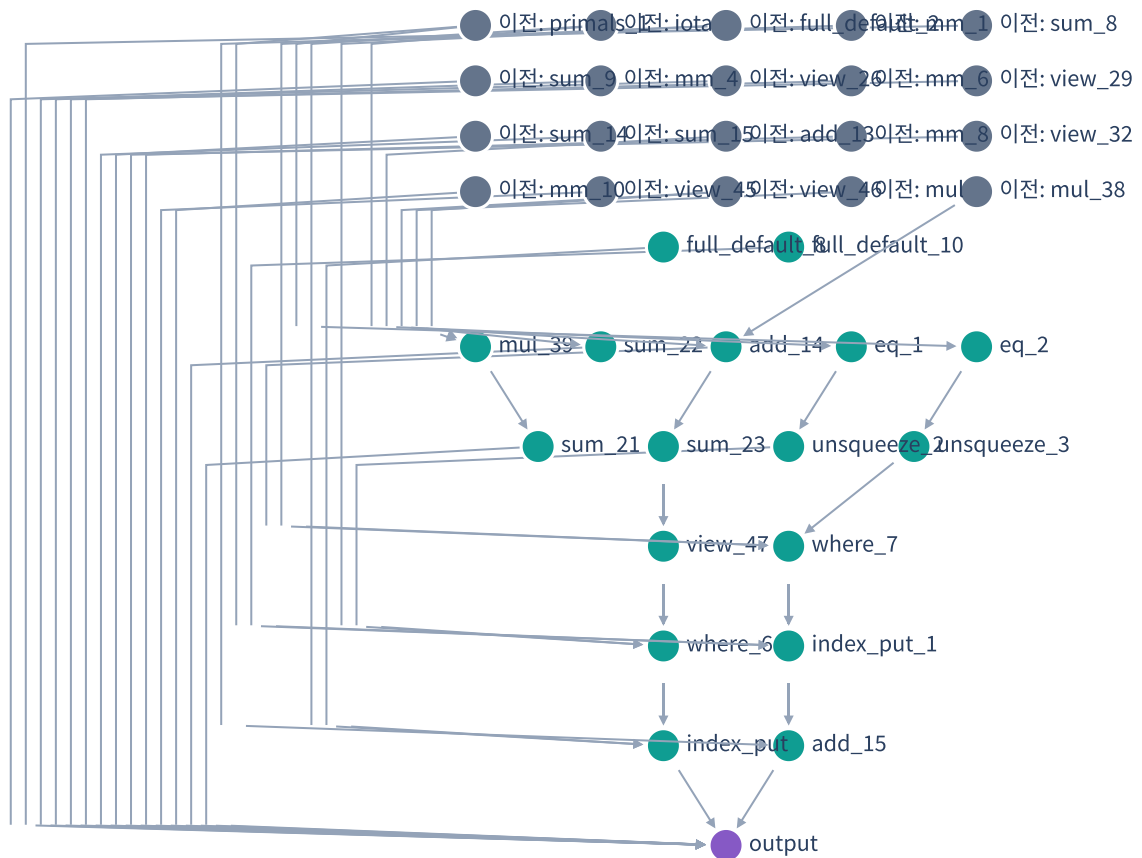
현재 capture · node 157~182/200 · 회색=이전 구간 입력



● boundary ● call_function

Training: 두 번째 Inductor graph (backward capture 경로 확인)

현재 capture · node 183-200/200 · 회색=이전 구간 입력



● boundary ● call_function ● output

Training output / gradient 비교

passed	True
rtol	0.0001
atol	1e-05
gradients_checked	16
output_shape	[4, 128, 1024]
max_abs_error	7.748603820800781e-07

현재 training capture의 forward/backward 디렉터리

capture 0	raw/debug/torch_compile_debug/run_2026_09_26_11_32_38_078593-pid_3413045/torchinductor/model__0_backward_3.1
capture 1	raw/debug/torch_compile_debug/run_2026_09_26_11_32_38_078593-pid_3413045/torchinductor/model__0_forward_1.0

확인해 보기 — Block이 한 FX graph에 들어가면 GPU kernel도 하나가 되나요?

▼ 해설 펼치기

아닙니다. 한 graph 안에서도 GEMM, reduction, layout 변환, 외부 attention 구현 등이 여러 실행 경계를 만듭니다. Scheduled IR wrapper 호출, 실제 CUDA trace를 차례로 연결해야 합니다.

12. Fusion과 성능: 실제 timeline으로 확인하기

원문: §14, §6.7, §11.8 · pp. 72–76 · 목차로

12.1 GPU에서 컴파일 효과를 직접 확인하기

먼저 작은 입력 → 중간 입력 → GPU 계산량이 충분한 큰 입력을 비교합니다. 같은 GELU라도 작은 입력은 Python/launch 비용의 비중이 크고, 큰 입력은 여러 kernel의 계산과 메모리 이동이 중요해집니다. 모든 점을 표시해 유리한 경우와 이득이 작은 경우를 함께 봅니다.

Eager/compiled는 각각 새 프로세스에서 같은 seed·shape·dtype·모델로 실행합니다. 입력은 이미 GPU에 있습니다. 데이터 로딩, host→device 전송, optimizer는 이 측정에 포함하지 않습니다. Inference에는 필요 없는 zero_grad/parameter 순회를 측정 loop에 넣지 않습니다.

- **CUDA event latency:** warm-up 후 100회 호출을 묶어 7번 측정한 중앙값. 오차 막대는 25–75% 구간(IQR)입니다. Stream의 대기 간격도 포함합니다.
- **입력 처리량:** 입력 원소 수 / event latency입니다. Block의 입력 원소는 token 위치이며 생성 token/s와 구별합니다.
- **CUDA launches:** 별도 profiler 구간에서 센 실제 GPU kernel 호출 수 / 반복 횟수입니다.
- **Kernel duration 합:** profiler의 실제 GPU kernel duration 합 / 반복 횟수입니다. Event latency나 host wall time과 같은 측정이 아닙니다.

이 GELU는 compiler가 연산의 결합을 보여주도록 수식을 펼친 ManualGELU입니다. 내장 F.gelu의 이미 최적화된 구현에서 같은 속도향상을 가정하지 않습니다. 뒤의 MLP와 GPT block에서도 실제 결과를 확인합니다.

```
GELU_CASES = [('작음', 1, 8, 16), ('중간', 1, 128, 128), ('큼', 4, 512, 1024)]
gelu_pairs = []
for label, batch, sequence, channels in GELU_CASES:
    pair = [experiment('profiling', workload='gelu', mode=mode,
                      batch=batch, sequence=sequence, channels=channels,
                      warmup=20, iterations=100, repeats=7)
            for mode in ['eager', 'compiled']]
    gelu_pairs.append(pair)
viz.performance_sweep(gelu_pairs).show()
gelu_eager, gelu_compiled = gelu_pairs[-1]
viz.performance([gelu_eager, gelu_compiled]).show()
viz.timeline(gelu_eager).show()
viz.timeline(gelu_compiled).show()
show_summary({
    '큰 입력 shape': gelu_eager['input_shape'],
    'CUDA event 속도비 (eager / compiled)': gelu_eager['steady_cuda_event_ms'] /
    gelu_compiled['steady_cuda_event_ms'],
    'GPU kernel duration 합 속도비': gelu_eager['cuda_kernel_duration_ms_per_call'] /
    gelu_compiled['cuda_kernel_duration_ms_per_call'],
    'launches/call (eager → compiled)': (gelu_eager['cuda_launches_per_call'],
    gelu_compiled['cuda_launches_per_call']),
    '정확성': [gelu_eager['correctness']['passed'], gelu_compiled['correctness']['passed']]})
```

CUDA 실험 실행: profiling · profiling-6dae3e4f697e

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-6dae3e4f697e

CUDA 실험 실행: profiling · profiling-8361ca7cb182

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-8361ca7cb182

CUDA 실험 실행: profiling · profiling-563b93b9626c

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-563b93b9626c

CUDA 실험 실행: profiling · profiling-9f956342d436

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-9f956342d436
CUDA 실험 실행: profiling · profiling-a36ae5b80ce9

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-a36ae5b80ce9
CUDA 실험 실행: profiling · profiling-665e87bec43f

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-665e87bec43f

그림 52

작은 입력부터 큰 입력까지: 유리한 경우와 이득이 작은 경우

현재 실측 · shape마다 별도 eager/compiled 프로세스 · 오차 막대=반복 측정 IQR

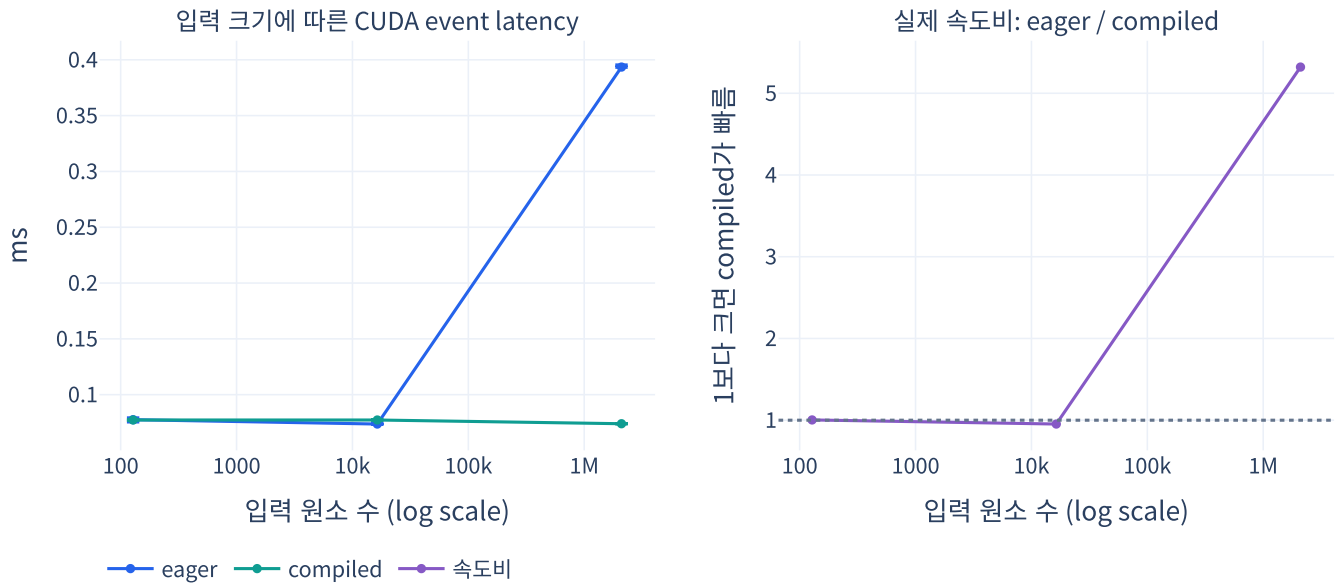


그림 53

gelu: GPU 시간 · 처리량 · launch 비교

현재 CUDA 실측 · event=반복 측정 중앙값/IQR · kernel duration은 별도 profiler 측정

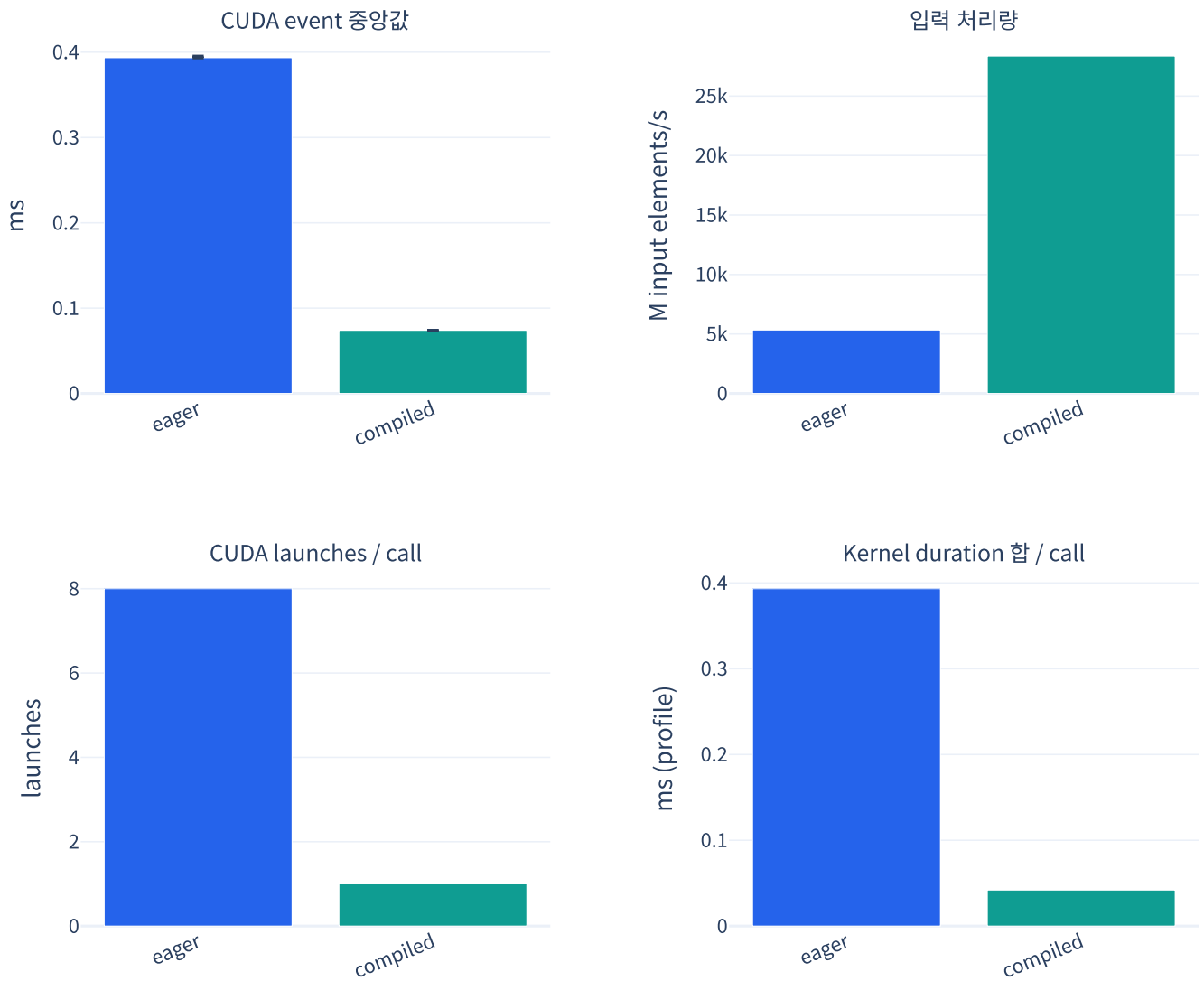
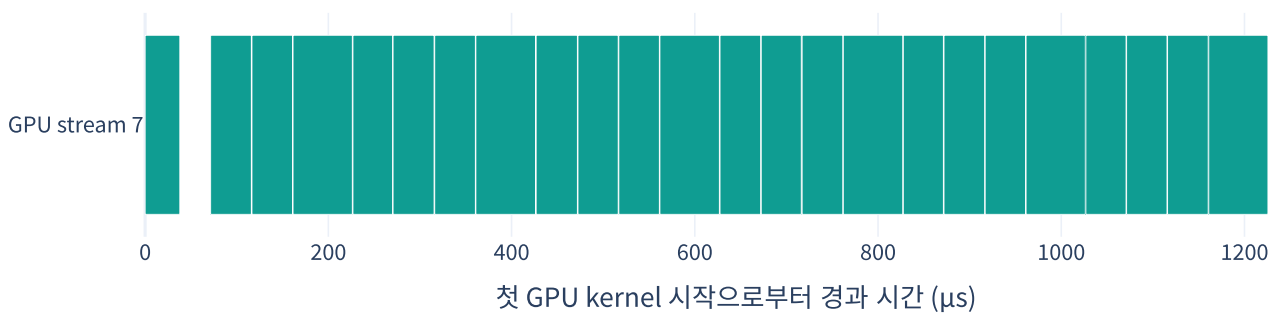


그림 54

gelu · eager · 실제 CUDA timeline

profiling-a36ae5b80ce9 · GPU kernel만 표시 · profiler 3회

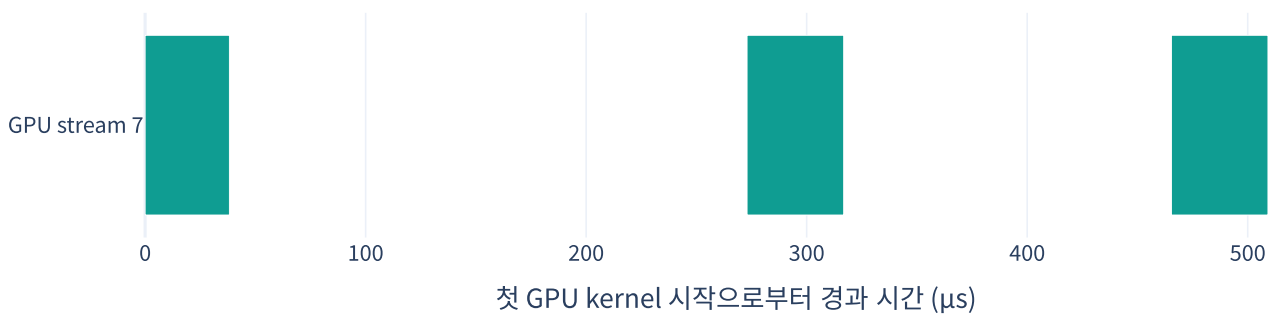


실제 kernel 이름	profile 구간 수	호출	duration 합계 (μs)
void at::native::vectorized_elementwise_kernel<4, at::native::AUnaryFuncutor<float, float, float, at::native::binary_internal::MulFuncutor<float> >, std::array<char*, 2ul> >(int, at::native::AUnaryFuncutor<float, float, float, at::native::binary_internal::MulFuncutor<float> >, std::array<char*, 2ul>)	9		391.327
void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::pow_tensor_scalar_kernel_impl<float, float>(at::TensorIteratorBase&, float)::lambda(float)#2, std::array<char*, 2ul> >(int, at::native::(anonymous namespace)::pow_tensor_scalar_kernel_impl<float, float>(at::TensorIteratorBase&, float)::lambda(float)#2, std::array<char*, 2ul>)	3		133.025
void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFuncutor_add<float>, std::array<char*, 3ul> >(int, at::native::CUDAFuncutor_add<float>, std::array<char*, 3ul>)	3		194.528
void at::native::vectorized_elementwise_kernel<4, at::native::tanh_kernel_cuda(at::TensorIteratorBase&::lambda()#2)::operator()() const::lambda(float)#1, std::array<char*, 2ul> >(int, at::native::tanh_kernel_cuda(at::TensorIteratorBase&::lambda()#2)::operator()() const::lambda(float)#1, std::array<char*, 2ul>)	3		132.768
void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFuncutorOnSelf_add<float>, std::array<char*, 2ul> >(int, at::native::CUDAFuncutorOnSelf_add<float>, std::array<char*, 2ul>)	3		133.344
void at::native::vectorized_elementwise_kernel<4, at::native::BinaryFuncutor<float, float, float, at::native::binary_internal::MulFuncutor<float> >, std::array<char*, 3ul> >(int, at::native::BinaryFuncutor<float, float, float, at::native::binary_internal::MulFuncutor<float> >, std::array<char*, 3ul>)	3		194.784

그림 55

gelu · compiled · 실제 CUDA timeline

profiling-665e87bec43f · GPU kernel만 표시 · profiler 3회



실제 kernel 이름	profile 구간 수	호출	duration 합계 (μs)
triton_poi_fused_add_mul_pow_tanh_0	3		125.825
확인한 결과			
큰 입력 shape	[4, 512, 1024]		
CUDA event 속도비 (eager / compiled)	5.3203822454340415		
GPU kernel duration 합 속도비	9.37632425988476		
launches/call (eager → compiled)	(8.0, 1.0)		
정확성	[True, True]		

같은 원리를 residual + LayerNorm에서도 확인

두 입력의 add와 normalization을 한 실행 단위로 결합할 기회를 봅니다. GPU 시간·처리량·launch를 먼저 읽고, 실제 peak memory도 별도로 비교합니다. 초반의 작은 IR 예제와 구조는 같고, 여기서는 B=4, T=512, C=1024로 GPU 작업량을 늘렸습니다.

```

residual_profiles = [experiment('profiling', workload='residual_layernorm', mode=mode,
                               batch=4, sequence=512, channels=1024,
                               warmup=20, iterations=100, repeats=7)
                     for mode in ['eager', 'compiled']]
viz.performance(residual_profiles).show()
show_summary({'CUDA event 속도비': residual_profiles[0]['steady_cuda_event_ms'] / residual_profiles[1]
             ['steady_cuda_event_ms'],
             'GPU kernel duration 합 속도비': residual_profiles[0]['cuda_kernel_duration_ms_per_call'] /
             residual_profiles[1]['cuda_kernel_duration_ms_per_call']})
fig = go.Figure(go.Bar(x=['eager', 'compiled'],
                       y=[r['memory']['peak_allocated_bytes']/2**20 for r in residual_profiles],
                       marker_color=[viz.BLUE, viz.TEAL]))
fig.update_yaxes(title='Peak allocated MiB')
viz.style(fig, '같은 실험의 실제 peak memory', '현재 CUDA 실행 · 모델과 기존 allocation 포함', 360).show()

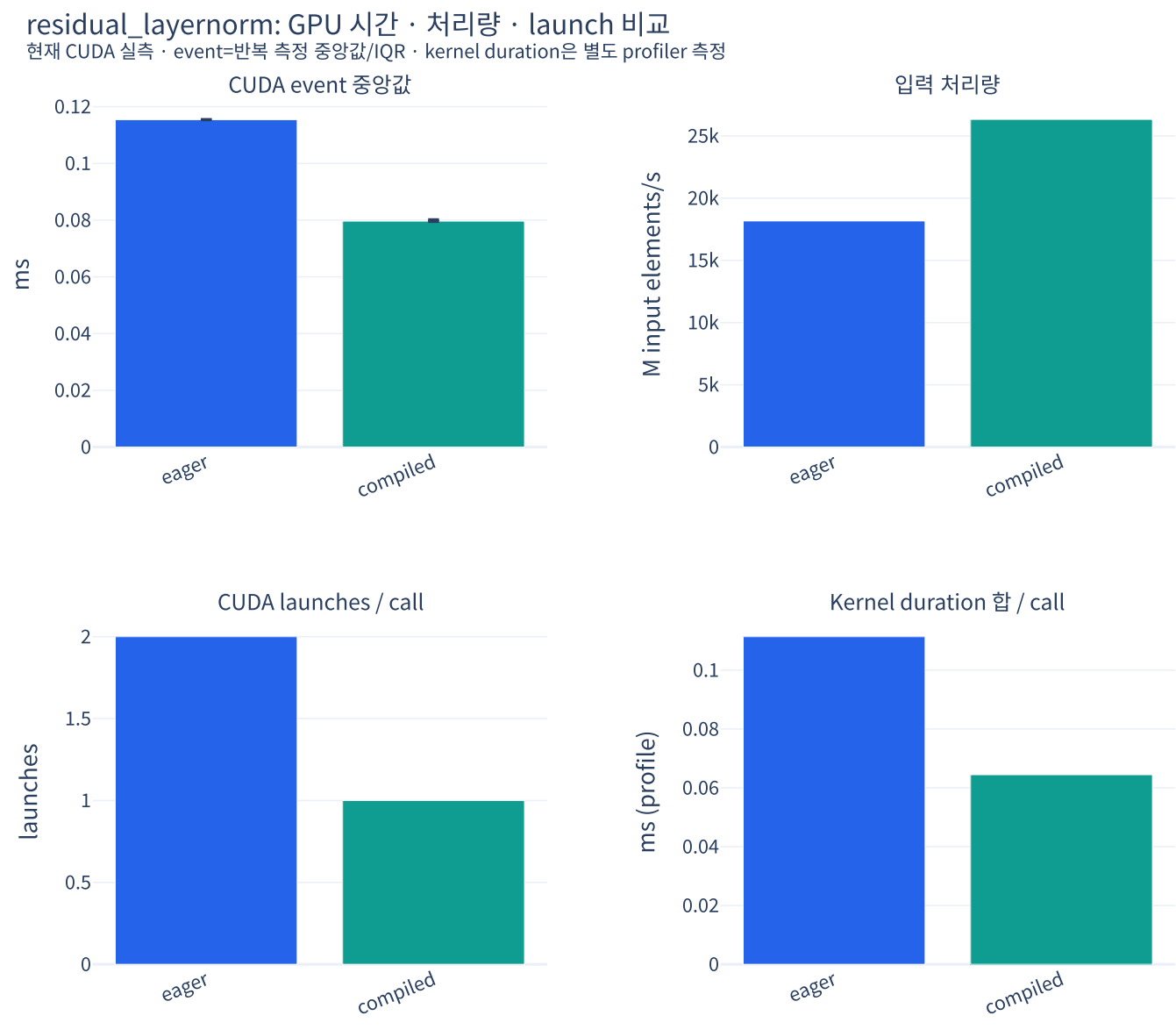
```

CUDA 실험 실행: profiling · profiling-ff53b8dba8f5

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-ff53b8dba8f5

CUDA 실험 실행: profiling · profiling-9ee5a3c32459

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-9ee5a3c32459

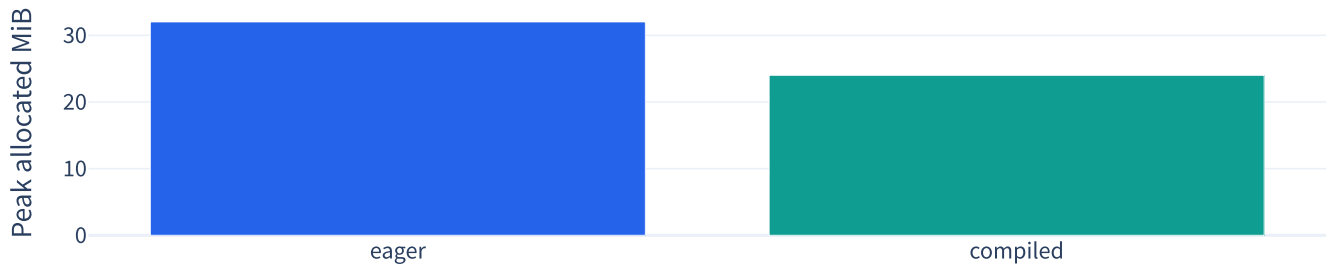


확인한 결과

CUDA event 속도비	1.4491123400207582
GPU kernel duration 합 속도비	1.7277701070965006

같은 실험의 실제 peak memory

현재 CUDA 실험 · 모델과 기존 allocation 포함



첫 컴파일 비용은 언제 회수할까?

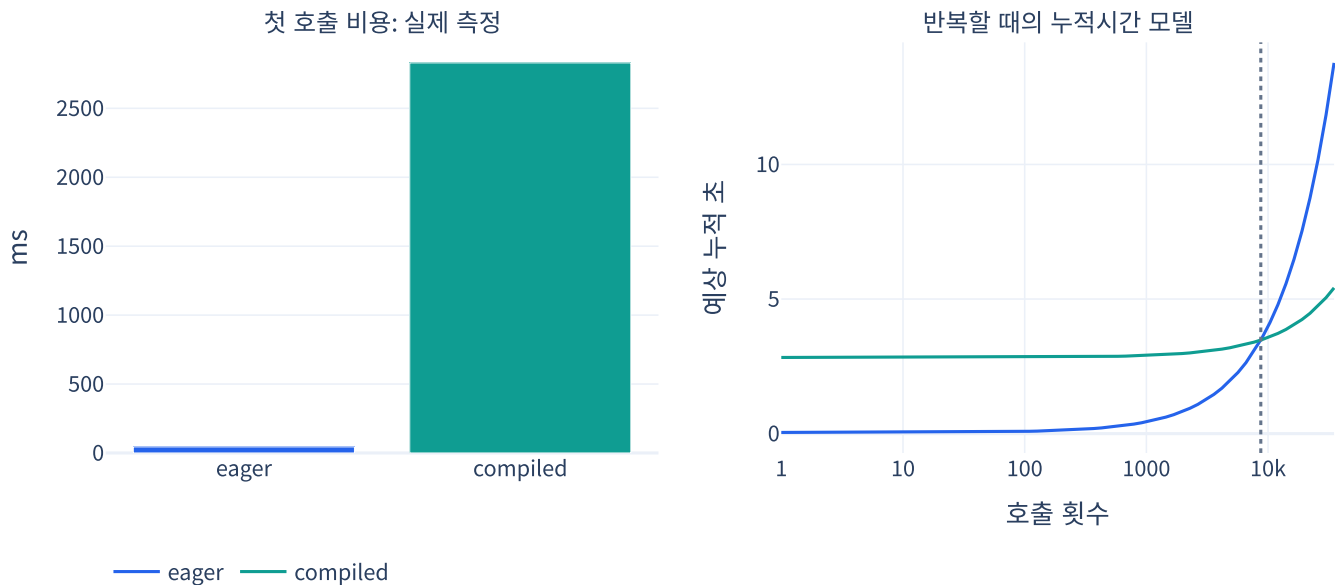
반복 실행이 빨라도 첫 compile에는 시간이 듭니다. 왼쪽은 첫 호출의 실제 wall time, 오른쪽은 $\text{first_call} + (N-1) \times \text{steady_wall}$ 로 계산한 누적시간 모델입니다. 오른쪽 곡선은 N번 모두 실행한 측정값이 아닙니다. 같은 guard가 유지되고 반복 latency가 안정적이라는 가정입니다.

```
viz.compilation_cost(gelu_eager, gelu_compiled).show()
viz.compilation_cost(residual_profiles[0], residual_profiles[1]).show()
```

그림 58

컴파일 비용 회수: 약 8717회

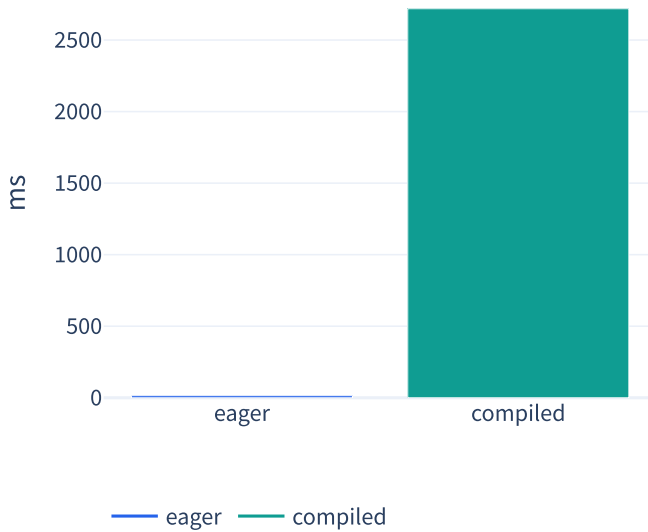
왼쪽=실측 · 오른쪽=first call + (N-1)×steady wall 중앙값으로 계산한 모델



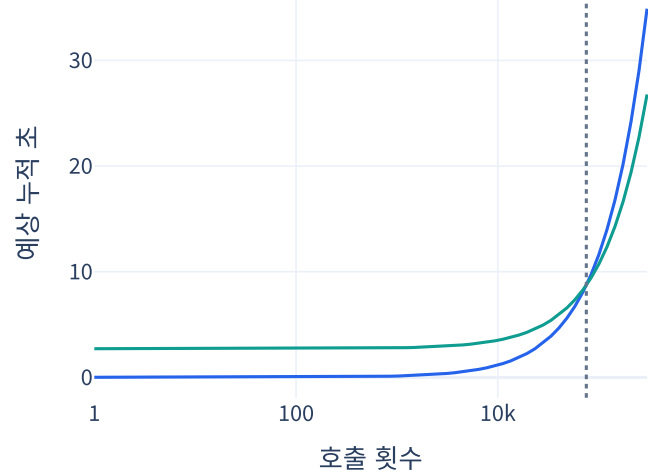
컴파일 비용 회수: 약 75356회

왼쪽=실측 · 오른쪽=first call + (N-1) × steady wall 중앙값으로 계산한 모델

첫 호출 비용: 실제 측정



반복할 때의 누적시간 모델



외부 GEMM 중심의 MLP는 같은 이득이 날까?

MLP의 Linear는 이미 최적화된 library/template 경계를 가질 수 있고 내장 GELU도 GPU 구현을 사용합니다. 아래 비교는 **이득이 작거나 느려지는 경우도 그대로 표시합니다**. 이 결과를 pointwise chain의 큰 개선과 연결하면 compiler가 어느 경계에서 비용을 줄이는지 이해할 수 있습니다.

```
mlp_profiles = [experiment('profiling', workload='mlp', mode=mode, batch=2, sequence=128,
                           channels=256, warmup=20, iterations=100, repeats=7)
                 for mode in ['eager', 'compiled']]
viz.performance(mlp_profiles).show()
show_summary({'MLP CUDA event 속도비': mlp_profiles[0]['steady_cuda_event_ms'] / mlp_profiles[1]
             ['steady_cuda_event_ms'],
             'launches/call (eager → compiled)': (mlp_profiles[0]['cuda_launches_per_call'],
             mlp_profiles[1]['cuda_launches_per_call'])})
```

CUDA 실험 실행: profiling · profiling-5bd3a8090517

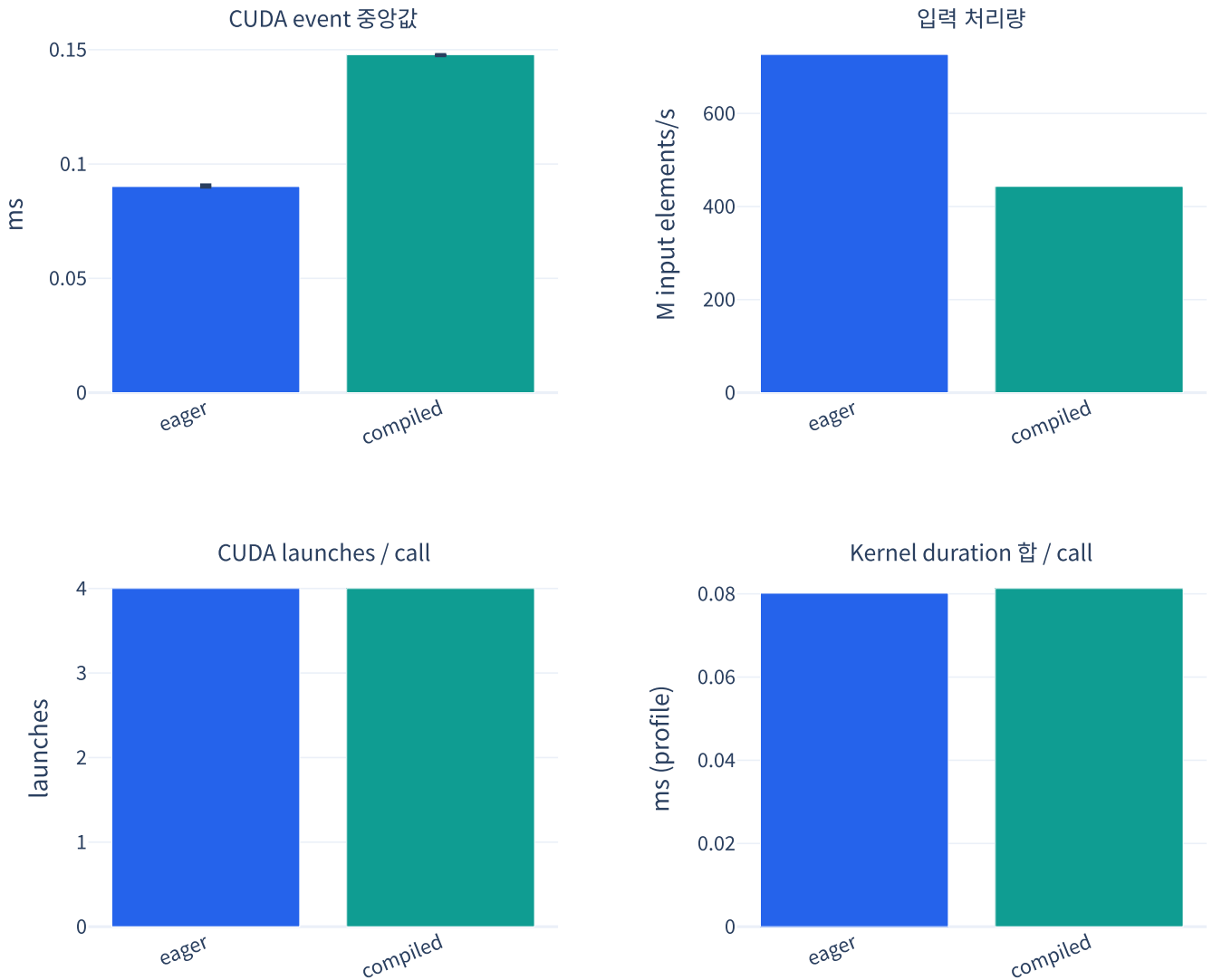
완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-5bd3a8090517

CUDA 실험 실행: profiling · profiling-c84da60e6f44

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-c84da60e6f44

mlp: GPU 시간 · 처리량 · launch 비교

현재 CUDA 실측 · event=반복 측정 중앙값/IQR · kernel duration은 별도 profiler 측정



확인한 결과

MLP CUDA event 속도비

0.6099727723818216

launches/call (eager → compiled)

(4.0, 4.0)

12.2 Fusion은 어느 비용을 없애는가?

Pointwise fusion은 중간 tensor의 global-memory write/read와 launch를 줄일 수 있습니다. Reduction+pointwise는 통계를 만드는 과정과 소비하는 과정을 함께 고려해야 합니다. GEMM의 **prologue**는 입력 쪽, **epilogue**는 output 쪽의 주변 계산입니다.

FlashAttention 같은 algorithmic fusion은 전체 attention score를 global memory에 저장하지 않도록 tile과 softmax 계산을 재구성합니다. 단순히 여러 함수를 한 이름으로 감싼다는 뜻이 아닙니다.

넓은 fusion에는 제약이 있습니다: iteration domain·layout·external library·alias/mutation·수치적 순서·register/shared-memory 자원입니다. 감소한 launch 수만 보지 말고 latency와 memory 결과를 함께 봅니다.

```
BLOCK_BENCH = dict(workload='block1', **BLOCK_SHAPE, warmup=20, iterations=100, repeats=7)
block_eager = experiment('profiling', attention='manual', mode='eager', **BLOCK_BENCH)
manual_profile = experiment('profiling', attention='manual', mode='compiled', **BLOCK_BENCH)
```

```

sdpa_profile = experiment('profiling', attention='sdpa', mode='compiled', **BLOCK_BENCH)
viz.performance([block_eager, manual_profile]).show()
show_summary({'같은 manual block의 CUDA event 속도비': block_eager['steady_cuda_event_ms'] /
manual_profile['steady_cuda_event_ms'],
              'launches/call (eager → compiled)': (block_eager['cuda_launches_per_call'],
manual_profile['cuda_launches_per_call'])})
viz.performance([manual_profile, sdpa_profile]).show()
viz.timeline(block_eager).show()
viz.timeline(manual_profile).show()
viz.timeline(sdpa_profile).show()
details('현재 manual CUDA kernel 이름', manual_profile['kernel_names'])
details('현재 SDPA CUDA kernel 이름 - Flash 여부를 여기서 확인', sdpa_profile['kernel_names'])

```

CUDA 실험 실행: profiling · profiling-955db03dae58

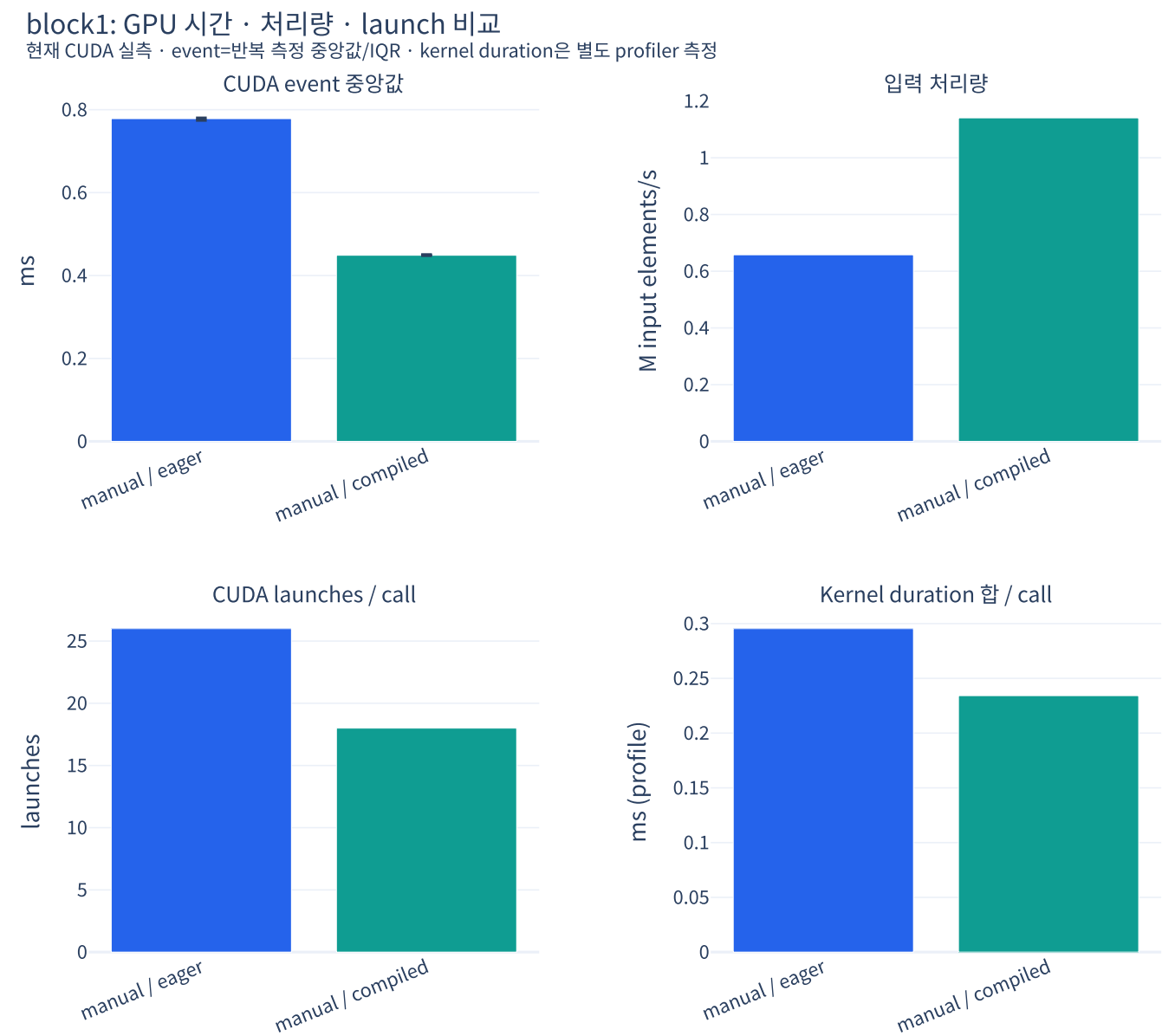
완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-955db03dae58

CUDA 실험 실행: profiling · profiling-4de74e194612

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-4de74e194612

CUDA 실험 실행: profiling · profiling-268c376308fb

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-268c376308fb



확인한 결과

같은 manual block의 CUDA event 속도비	1.7339394749016535
launches/call (eager → compiled)	(26.0, 18.0)

그림 62

block1: GPU 시간 · 처리량 · launch 비교

현재 CUDA 실행 · event=반복 측정 중앙값/IQR · kernel duration은 별도 profiler 측정

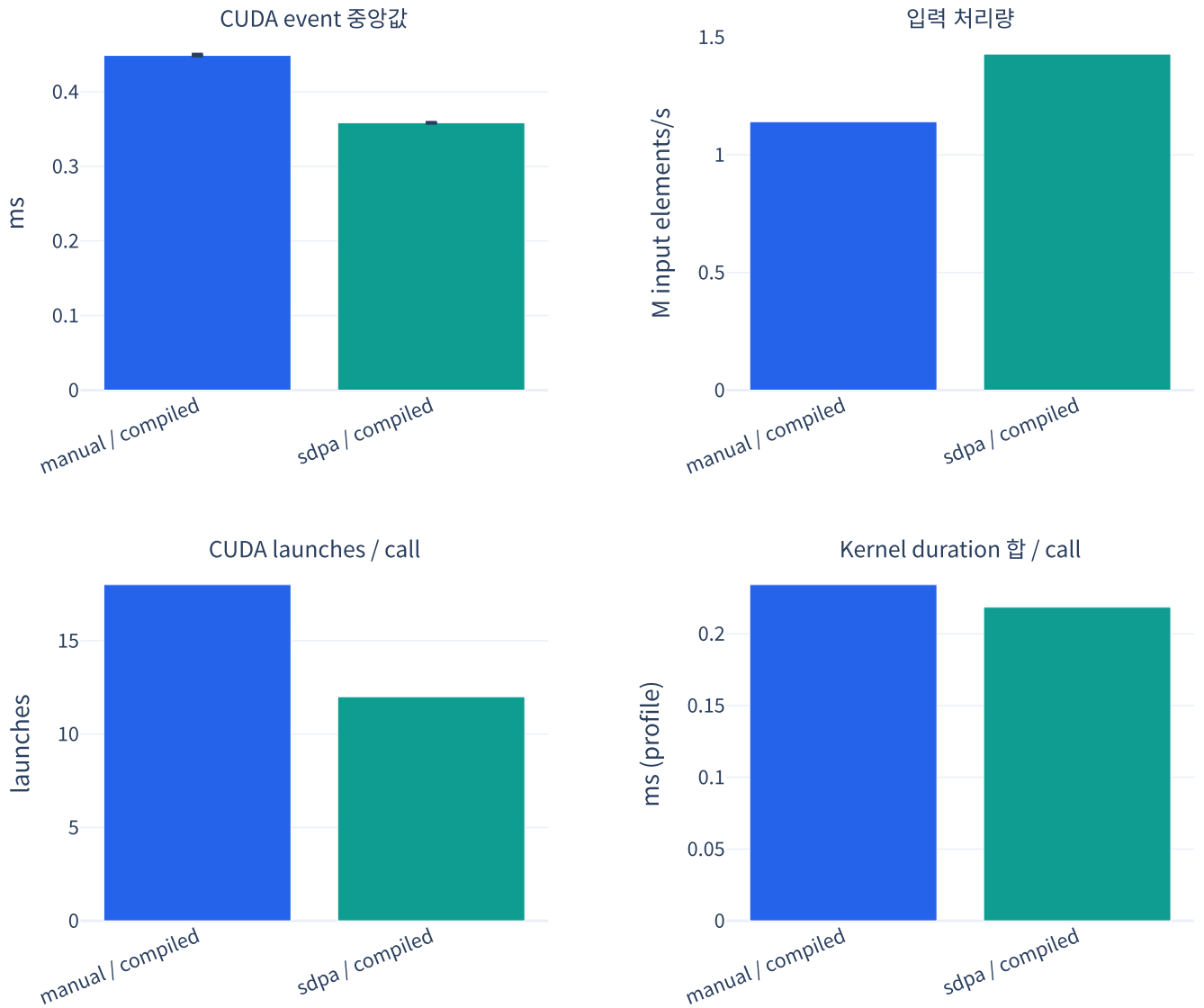
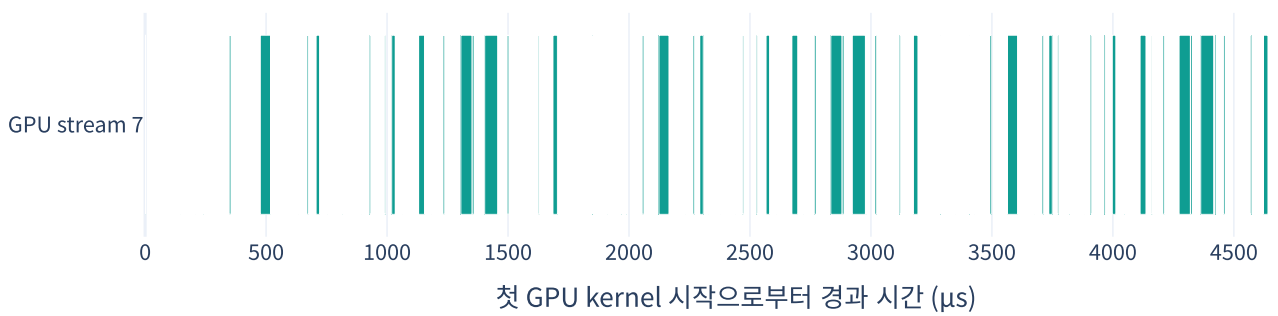


그림 63

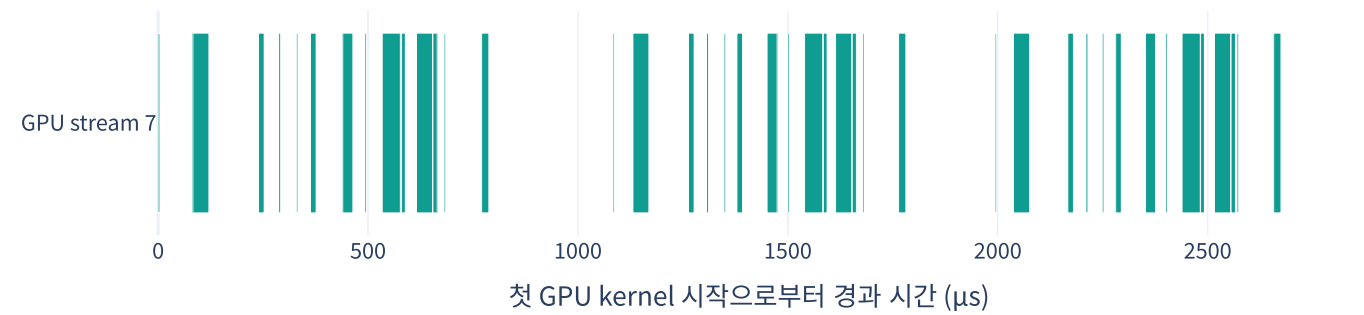
block1 · eager · 실제 CUDA timeline

profiling-955db03dae58 · GPU kernel만 표시 · profiler 3회



실제 kernel 이름	profile 구간 수	호출 수	duration 합계 (μs)
void (anonymous namespace)::elementwise_kernel_with_index<int, at::native::arange_cuda_out (c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&)::{lambda()}#1>::operator()() const::: {lambda()}#4>::operator()() const::: {lambda(long)#1}>(int, at::native::arange_cuda_out(c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&)::{lambda()}#1>::operator()() const::: {lambda()}#4>::operator()() const::: {lambda(long)#1}>::result_type*)	3		3.232
void at::native::vectorized_gather_kernel<16, long>(char*, char*, long*, int, long, long, long, long, bool)	6		12.734
void at::native::elementwise_kernel<128, 2, at::native::gpu_kernel_impl_nocast<at::native::CUDAFunction_add<float> >(at::TensorIteratorBase&, at::native::CUDAFunction_add<float> const&)::{lambda(int)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::CUDAFunction_add<float> >(at::TensorIteratorBase&, at::native::CUDAFunction_add<float> const&)::{lambda(int)#1})	3		9.088
void at::native::(anonymous namespace)::vectorized_layer_norm_kernel<float, float>(int, float, float const*, float const*, float const*, float*, float*, float*)	9		60.832
volta_sgemm_128x64_tn	9		427.329
void at::native::elementwise_kernel<128, 2, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const::: {lambda()}#7>::operator()() const::: {lambda(float)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const::: {lambda(float)#1}>::operator()() const::: {lambda(int)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const::: {lambda()}#7>::operator()() const::: {lambda(float)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const::: {lambda()}#7>::operator()() const::: {lambda(float)#1}>::operator()() const::: {lambda(int)#1})	12		52.734
volta_sgemm_64x64_nn	6		86.560
void at::native::vectorized_elementwise_kernel<4, at::native::BUnaryFunction<float, float, float, at::native::binary_internal::MulFunction<float> >, std::array<char*, 2ul> >(int, at::native::BUnaryFunction<float, float, float, at::native::binary_internal::MulFunction<float> >, std::array<char*, 2ul>)	3		12.609
void at::native::vectorized_elementwise_kernel<4, at::native::AUnaryFunction<float, float, bool, at::native::(anonymous namespace)::CompareEqFunction<float> >, std::array<char*, 2ul> >(int, at::native::AUnaryFunction<float, float, bool, at::native::(anonymous namespace)::CompareEqFunction<float> >, std::array<char*, 2ul>)	3		4.767
void at::native::elementwise_kernel<128, 2, at::native::gpu_kernel_impl_nocast<at::native::(anonymous namespace)::masked_fill_kernel(at::TensorIterator&, c10::Scalar const&)::{lambda()}#1>::operator()() const::: {lambda()}#7>::operator()() const::: {lambda(float, bool)#1}>(at::TensorIterator&, c10::Scalar const&)::{lambda()}#1>::operator()() const::: {lambda()}#7>::operator()() const::: {lambda(float, bool)#1}>::operator()() const::: {lambda(int)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::(anonymous namespace)::masked_fill_kernel(at::TensorIterator&, c10::Scalar const&)::{lambda()}#1>::operator()() const::: {lambda(float, bool)#1}>::operator()() const::: {lambda(float, bool)#1}>(at::TensorIteratorBase&, at::native::(anonymous namespace)::masked_fill_kernel(at::TensorIterator&, c10::Scalar const&)::{lambda()}#1>::operator()() const::: {lambda()}#7>::operator()() const::: {lambda(float, bool)#1}>::operator()() const::: {lambda(int)#1})	3		11.967
void (anonymous namespace)::softmax_warp_forward<float, float, float, 7, false, false>(float*, float const*, int, int, int, bool const*, int, bool)	3		14.721
volta_sgemm_32x128_tn	3		72.704
void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunction_add<float>, std::array<char*, 3ul> >(int, at::native::CUDAFunction_add<float>, std::array<char*, 3ul>)	6		24.608
void at::native::vectorized_elementwise_kernel<4, at::native::GeluCUDAKernelImpl(at::TensorIteratorBase&, at::native::GeluType)::{lambda()}#2>::operator()() const::: {lambda()}#2>::operator()() const::: {lambda(float)#1}, std::array<char*, 2ul> >(int, at::native::GeluCUDAKernelImpl(at::TensorIteratorBase&, at::native::GeluType)::{lambda()}#2>::operator()() const::: {lambda()}#2>::operator()() const::: {lambda(float)#1}, std::array<char*, 2ul>)	3		22.368
void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1}> const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1}> const&, bool)::{lambda(int)#1})	3		14.304
volta_sgemm_128x32_tn	3		55.616

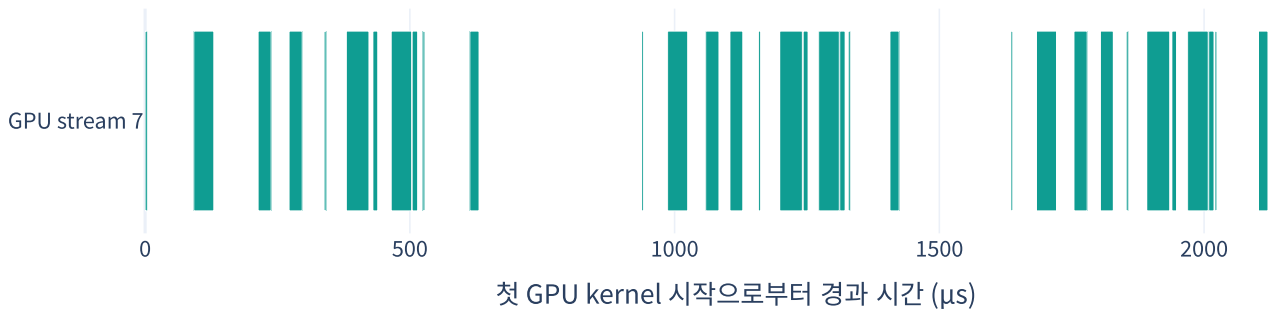
block1 · compiled · 실제 CUDA timeline
profiling-4de74e194612 · GPU kernel만 표시 · profiler 3회



실제 kernel 이름	profile 구간 수	호출 횟수	duration 합계 (μs)
triton_red_fused_add_arange_embedding_native_layer_norm_0	3		11.200
volta_sgemm_128x64_tn	6		244.638
triton_poi_fused_clone_1	3		6.334
triton_poi_fused_clone_2	3		6.210
volta_sgemm_64x64_nn	6		80.769
triton_per_fused__softmax_div_eq_exp_masked_fill_prepare_softmax_online_sub_3	3		14.080
triton_poi_fused_clone_4	3		10.176
triton_poi_fused_clone_5	3		4.864
volta_sgemm_32x32_sliced1x4_tn	3		71.967
triton_per_fused_add_arange_embedding_native_layer_norm_6	3		11.935
triton_poi_fused_gelu_7	3		27.136
volta_sgemm_128x128_tn	3		115.362
void cublasLt::splitKreduce_kernel<32, 16, int, float, float, float, float, false, float, float, float, true, false, false>(cublasLt::cublasSplitKParams<float>, float const*, float const*, float*, float*, float const*, float const*, float const*, float const*, float const*, float const*, void*, long, float*, int*, float*, float*, float const*, float const*, float const*, float const*, float const*, float const*)	3		30.560
triton_per_fused_add_native_layer_norm_8	3		11.168
triton_poi_fused_add_index_native_layer_norm_9	3		4.224
volta_sgemm_128x32_tn	3		51.904

block1 · compiled · 실제 CUDA timeline

profiling-268c376308fb · GPU kernel만 표시 · profiler 3회



실제 kernel 이름	profile 구간 수	duration 합계 (μs)
triton_red_fused_add_arange_embedding_native_layer_norm_0	3	11.137
volta_sgemm_128x64_tn	6	242.242
fmha_cutlassF_f32_aligned_64x64_rf_sm75(PyTorchMemEffAttention::AttentionKernel<float, cutlass::arch::Sm75, true, 64, 64, 64, true, true>::Params)	3	74.718
volta_sgemm_32x32_sliced1x4_tn	3	73.439
triton_per_fused_add_arange_embedding_native_layer_norm_1	3	11.455
triton_poi_fused_gelu_2	3	27.201
volta_sgemm_128x128_tn	3	116.577
void cublasLt::splitKreduce_kernel<32, 16, int, float, float, float, float, false, float, float, float, true, false, false>(cublasLt::cublasSplitKParams<float>, float const*, float const*, float const*, float const*, float const*, float const*, float const*, float const*, float const*, void*, long, float*, int*, float*, float*, float const*, float const*, float const*, float const*, float const*)	3	30.593
triton_per_fused_add_native_layer_norm_3	3	11.359
triton_poi_fused_add_index_native_layer_norm_4	3	4.161
volta_sgemm_128x32_tn	3	52.770

▼ 현재 manual CUDA kernel 이름

```
[ 'triton_per_fused__softmax_div_eq_exp_masked_fill_prepare_softmax_online_sub_3',
  'triton_per_fused_add_arange_embedding_native_layer_norm_6', 'triton_per_fused_add_native_layer_norm_8',
  'triton_poi_fused_add_index_native_layer_norm_9', 'triton_poi_fused_clone_1', 'triton_poi_fused_clone_2',
  'triton_poi_fused_clone_4', 'triton_poi_fused_clone_5', 'triton_poi_fused_gelu_7',
  'triton_red_fused_add_arange_embedding_native_layer_norm_0', 'void cublasLt::splitKreduce_kernel<32, 16, int,
float, float, float, float, false, float, float, float, true, false, false>
(cublasLt::cublasSplitKParams<float>, float const*, float const*, float*, float*, float const*, float const*,
float const*, float const*, float*, void*, long, float*, int*, float*, float*, float const*, float const*,
float const*, float const*, float const*)', 'volta_sgemm_128x128_tn', 'volta_sgemm_128x32_tn',
'volta_sgemm_128x64_tn', 'volta_sgemm_32x32_sliced1x4_tn', 'volta_sgemm_64x64_nn']
```

▼ 현재 SDPA CUDA kernel 이름 — Flash 여부를 여기서 확인

```
[ 'fmha_cutlassF_f32_aligned_64x64_rf_sm75(PyTorchMemEffAttention::AttentionKernel<float, cutlass::arch::Sm75,
true, 64, 64, 64, true, true>::Params)', 'triton_per_fused_add_arange_embedding_native_layer_norm_1',
'triton_per_fused_add_native_layer_norm_3', 'triton_poi_fused_add_index_native_layer_norm_4',
'triton_poi_fused_gelu_2', 'triton_red_fused_add_arange_embedding_native_layer_norm_0', 'void
cublasLt::splitKreduce_kernel<32, 16, int, float, float, float, float, false, float, float, float, true,
false, false>(cublasLt::cublasSplitKParams<float>, float const*, float const*, float*, float*, float const*,
float const*, float const*, float const*, float*, void*, long, float*, int*, float*, float*, float const*,
float const*, float const*, float const*, float const*)', 'volta_sgemm_128x128_tn', 'volta_sgemm_128x32_tn',
'volta_sgemm_128x64_tn', 'volta_sgemm_32x32_sliced1x4_tn']
```

확인해 보기 — CPU operator duration과 CUDA kernel duration을 모두 더하면 전체 시간이 되나요?

▼ 해설 펼치기

아닙니다. CPU/GPU 실행은 겹칠 수 있고 CPU profiler event는 중첩될 수 있습니다. 전체 latency, stream event 경과, 개별 kernel duration, CPU operator self/total 시간을 서로 다른 측정으로 읽어야 합니다.

13. Megakernel과 shape 실험

원문: §15, §5.15 · pp. 77–81, p. 25 · 목차로

13.1 Fusion 범위를 넓힌다는 뜻

Local pointwise fusion, reduction+pointwise, GEMM epilogue, attention 알고리즘 fusion, block 규모 megakernel은 결합 범위가 다릅니다.

Persistent kernel은 thread block이 오래 머물며 여러 작업을 처리하는 실행 방식입니다. **Megakernel**은 여러 논리 stage를 넓게 묶은 범위를 가리킵니다. 하나의 구현이 두 성질을 모두 가질 수 있습니다.

FlashAttention을 사용해도 QKV/output projection GEMM 등이 별도로 남을 수 있습니다. `torch.compile`을 호출했다고 block megakernel이 되는 것도 아닙니다. 바로 앞의 실제 block timeline에서 남아 있는 경계를 찾아보세요.

여러 stage를 합치면 global-memory 왕복과 launch를 줄일 기회가 생기지만, 서로 다른 최적 tile-reduction schedule을 조정해야 하고 live value가 늘어 register pressure/spill/occupancy 문제가 생길 수 있습니다. 구현되지 않은 megakernel의 속도는 숫자로 치우지 않습니다.

```
viz.concept(['pointwise chain', 'reduction + pointwise', 'GEMM 주변 prologue/epilogue',
            'attention 내부 algorithmic fusion', '여러 stage / block 규모 fusion'],
            '결합 범위를 넓힐 때 바뀌는 설계 문제').show()
show_summary({'현재 block wrapper의 남은 호출 수': len(block_capture['wrapper']['calls']),
            '현재 profiler의 kernel 수': manual_profile['cuda_kernel_count'],
            'profiler 실행 횟수': manual_profile['profile_iterations'],
            '비교 지표': 'latency, launch, device-memory traffic, registers, shared memory, occupancy,
            spill'})
```

결합 범위를 넓힐 때 바뀌는 설계 문제

개념도 · 실행 시 작성 · 실제 compiler capture와 구별



확인한 결과

현재 block wrapper의 남은 호출 수	17
현재 profiler의 kernel 수	54
profiler 실행 횟수	3
비교 지표	latency, launch, device-memory traffic, registers, shared memory, occupancy, spill

13.2 실제 resource metadata와 선택적 hardware counter

Triton metadata의 register/shared-memory 값은 현재 compile에서 얻었습니다. Occupancy와 DRAM traffic은 profiler의 hardware counter 측정이 필요합니다. RUN_NCU=True인 기본 설정에서는 ncu로 한 kernel을 별도 실행해 측정을 시도합니다. 권한·도구 문제는 그 실행의 진단을 표시합니다.

```

show_summary(triton_run['resources'], '현재 add_one kernel resource metadata')
counters = triton_run['hardware_counters']
if counters['status'] == 'measured':
    for metric in counters['metrics']:
        fig = go.Figure(go.Bar(x=[metric['name']], y=[metric['value']], marker_color=viz.TEAL))
        fig.update_yaxes(title=metric['unit'])
        viz.style(fig, '현재 Nsight hardware counter', '이번 실행의 별도 ncu 측정', 330).show()
else:
    show_summary({'상태': counters['status'], '사유': counters['reason']}, 'Hardware counter 측정 상태')
  
```

현재 add_one kernel resource metadata

registers_per_thread	12
shared_memory_bytes	0
num_warps	4

Hardware counter 측정 상태

상태 unavailable

사유
==PROF== Connected to process 3410681 (/home/riverfog7/.local/share/uv/python/cpython-3.9.25-linux-x86_64-gnu/bin/python3.9)
==PROF== Target process 3410787 terminated before first instrumented API call. ==PROF== Target process 3410788 terminated before first instrumented API call. ==PROF== Target process 3410791 terminated before first instrumented API call. ==PROF== Target process 3410792 terminated before first instrumented API call. ==PROF== Target process 3410794 terminated before first instrumented API call. ==PROF== Target process 3410795 terminated before first instrumented API call. ==PROF== Target process 3410797 terminated before first instrumented API call. ==PROF== Target process 3410796 terminated before first instrumented API call. ==PROF== Target process 3410793 terminated before first instrumented API call. ==PROF== Target process 3410846 terminated before first instrumented API call. ==PROF== Target process 3410847 terminated before first instrumented API call. ==PROF== Target process 3410850 terminated before first instrumented API call. ==PROF== Target process 3410854 terminated before first instrumented API call. ==PROF== Target process 3410855 terminated before first instrumented API call. ==PROF== Target process 3410856 terminated before first instrumented API call. ==PROF== Target process 3410859 terminated before first instrumented API call. ==PROF== Target process 3410880 terminated before first instrumented API call. ==PROF== Target process 3410882 terminated before first instrumented API call. ==PROF== Target process 3410881 terminated before first instrumented API call. ==ERROR== ERR_NVGPUCTRPERM - The user does not have permission to access NVIDIA GPU Performance Counters on the target device 0. For instructions on enabling permissions and to get more information see https://developer.nvidia.com/ERR_NVGPUCTRPERM
==PROF== Target process 3410858 terminated before first instrumented API call. ==PROF== Target process 3410911 terminated before first instrumented API call. triton_codegen: completed → /home/riverfog7/Workspace/mlsys_study/study02/artifacts/triton_codegen-48d95aaafde4/raw/nsight-child ==PROF== Disconnected from process 3410681 ==WARNING== No kernels were profiled.

13.3 Vocabulary 50,257과 50,304

GPT-2의 실제 vocabulary는 50,257입니다. 50,304는 64의 배수이며 47개의 물리적 row를 더 둔 값입니다. LM-head GEMM의 N 차원이 tile에 잘 맞도록 하는 선택이지만, 항상 빠르다고 단정할 수 없습니다.

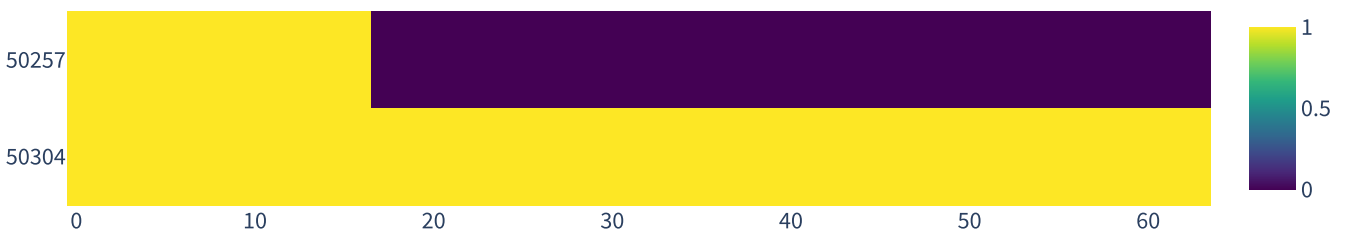
Weight tying 상태의 추가 parameter는 $47 \times C$ 개입니다. 아래는 마지막 tile의 유효 원소를 계산한 그림과 같은 작은 C, 같은 seed의 별도 CUDA 실행 비교입니다. 이는 kernel 선택/shape 효과를 보는 실습이며 큰 GPT-2 전체 성능 측정과는 아닙니다.

```
vocabulary_sizes = [50257, 50304]
last_tile = np.array([[int(i < (size % 64 or 64)) for i in range(64)] for size in vocabulary_sizes])
viz.heatmap(last_tile, '마지막 64-wide tile의 유효 column', y=[str(v) for v in vocabulary_sizes],
             source='개념도 · vocabulary dimension으로 계산').show()
show_summary({'추가 row': 50304 - 50257, 'C=32에서 추가 parameter (tied)': (50304-50257)*32})
if RUN_VOCAB_LAB:
    vocab_profiles = [experiment('profiling', workload='vocab_projection', vocab_size=size,
                               channels=32, sequence=8, mode='compiled', iterations=20)
                     for size in vocabulary_sizes]
    fig = go.Figure(go.Bar(x=[str(v) for v in vocabulary_sizes],
                           y=[r['steady_cuda_event_ms'] for r in vocab_profiles], marker_color=
[viz.BLUE,viz.TEAL]))
    fig.update_xaxes(title='physical vocabulary dimension'); fig.update_yaxes(title='CUDA event ms / call')
    viz.style(fig, '정렬의 효과는 실제로 측정한다', '현재 CUDA · B=1,T=8,C=32 · 각 shape의 eager/compiled 정확성 검증', 380).show()
else:
    show_summary({'vocabulary CUDA 비교': 'RUN_VOCAB_LAB=False: 이번 실행에서 측정하지 않음'})
```

그림 67

마지막 64-wide tile의 유효 column

개념도 · vocabulary dimension으로 계산



확인한 결과

추가 row

47

C=32에서 추가 parameter (tied)

1504

CUDA 실험 실행: profiling · profiling-d80f05ab9a3d

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-d80f05ab9a3d

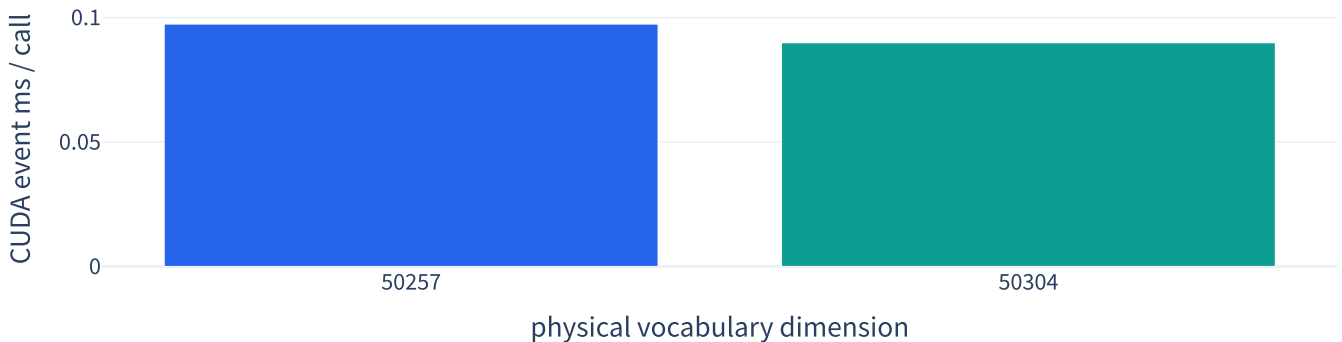
CUDA 실험 실행: profiling · profiling-c91016c6a7f2

완료 · NVIDIA GeForce RTX 2080 · 원본: study02/artifacts/profiling-c91016c6a7f2

그림 68

정렬의 효과는 실제로 측정한다

현재 CUDA · B=1,T=8,C=32 · 각 shape의 eager/compiled 정확성 검증



확인해 보기 — 같은 계산의 kernel 수가 줄었다는 사실만으로 megakernel이 더 낫다고 결론 낼 수 있나요?

▼ 해설 펼치기

없습니다. 같은 input/output 의미와 dtype에서 정확성을 먼저 검증하고, compile 비용과 반복 latency, memory 이동, register/shared-memory, occupancy/spill을 함께 비교해야 합니다. 학습 지원 범위와 backward 구현 여부도 같은 조건이어야 합니다.

14. 복습과 산출물 길잡이

원문: §16 · pp. 82–85 · 목차로

14.1 한 계산의 계보를 말로 설명해 보기

1. GELU의 Python 상수와 tensor 연산은 Dynamo graph에서 어떻게 달랐나요?
2. FakeTensor는 무엇을 알고 무엇을 모릅니까요?
3. 이번 AOT forward는 사용자 output 이외에 어떤 값을 반환했나요?
4. 이번 GELU의 pre/post IR이 같았다면 pointwise 결합은 어느 표현에서 이미 보였나요?
5. Residual+LayerNorm의 scheduler group과 실제 allocation은 어떻게 연결되나요?
6. MLP의 어느 호출은 generated이고 어느 호출은 external인가요?
7. Triton program의 BLOCK과 CUDA thread 개수는 같은 단위인가요?
8. Block의 source 연산 → FX node → scheduler group → wrapper → profiler를 어디까지 근거로 연결했나요?
9. 현재 GPU에서 SDPA는 어떤 kernel을 실제로 선택했나요?
10. 첫 compile을 포함한 시간과 warm-up 뒤 시간은 왜 따로 봐야 하나요?

정답은 고정된 node 수나 kernel 이름이 아닙니다. 이번 실행의 graph·코드·trace를 근거로 설명할 수 있는지가 목표입니다.

14.2 이번 실행의 파일

파일	역할
manifest.json	Run ID, 실행 인자, environment, 시작/종료, source/raw SHA-256
result.json	실제 결과, graph node/edge, 수치 검증, 시각화용 구조
raw/*.json, raw/*.py	현재 capture한 FX와 source
raw/debug/**/ir_pre_fusion.txt, ir_post_fusion.txt	실제 scheduler IR
raw/debug/**/output_code.py	CUDA wrapper와 generated Triton
raw/triton_dump/, raw/triton_cache/	현재 compilation의 backend stage
raw/add_one.*	직접 작성한 Triton 예제의 TTIR/TTGIR/LLVM IR/PTX/cubin/SASS
raw/profiler_trace.json	실제 CPU operator / CUDA kernel timeline

노트북 밖에서도 저장소 루트에서 `uv run --locked python -m study02.experiments.inductor --workload block1 --training`처럼 실행할 수 있습니다. CLI도 매번 새로운 출력 디렉터리를 만듭니다. 실패한 run의 `error.txt`, `stderr.log`는 진단용이며 다른 run의 결과로 대체되지 않습니다.

```
show_summary({f'{i+1:02d}. {r["run_id"]}' : r['_run_dir'] for i, r in enumerate(RUNS)}, '이 노트북 실행이 직접  
생성한 run 목록')  
assert all(r['device'] == 'cuda' for r in RUNS)  
assert len({r['run_id'] for r in RUNS}) == len(RUNS)  
show_summary({'CUDA run 수': len(RUNS), 'run ID 중복': False,  
             '실행 interpreter': sys.executable, 'core numerical checks': '실험 내부 assert_close 통과'})
```

이 노트북 실행이 직접 생성한 run 목록

01. python_runtime-4ae6f27dcec9	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/python_runtime-4ae6f27dcec9
02. dynamo-6b7857746e3d	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/dynamo-6b7857746e3d
03. aot_autograd-9e6fbfec602d	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/aot_autograd-9e6fbfec602d
04. inductor-41b12ff94e55	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-41b12ff94e55
05. inductor-8c13408a2fbb	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-8c13408a2fbb
06. inductor-8dcfade14a8d	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-8dcfade14a8d
07. triton_codegen-48d95aaafde4	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/triton_codegen-48d95aaafde4
08. inductor-3f58222f26a	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-3f58222f26a
09. inductor-63f0066fbf2d	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-63f0066fbf2d
10. inductor-3ac83887136a	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-3ac83887136a
11. inductor-c47015e2ea19	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/inductor-c47015e2ea19
12. profiling-6dae3e4f697e	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-6dae3e4f697e
13. profiling-8361ca7cb182	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-8361ca7cb182
14. profiling-563b93b9626c	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-563b93b9626c
15. profiling-9f956342d436	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-9f956342d436
16. profiling-a36ae5b80ce9	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-a36ae5b80ce9
17. profiling-665e87bec43f	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-665e87bec43f
18. profiling-ff53b8dba8f5	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-ff53b8dba8f5
19. profiling-9ee5a3c32459	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-9ee5a3c32459

20. profiling-5bd3a8090517	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-5bd3a8090517
21. profiling-c84da60e6f44	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-c84da60e6f44
22. profiling-955db03dae58	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-955db03dae58
23. profiling-4de74e194612	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-4de74e194612
24. profiling-268c376308fb	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-268c376308fb
25. profiling-d80f05ab9a3d	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-d80f05ab9a3d
26. profiling-c91016c6a7f2	/home/riverfog7/Workspace/mlsys_study/study02/artifacts/profiling-c91016c6a7f2

확인한 결과

CUDA run 수	26
run ID 중복	False
실행 interpreter	/home/riverfog7/Workspace/mlsys_study/.venv/bin/python3
core numerical checks	실험 내부 assert_close 통과

14.3 조건을 바꿔 보는 다음 실습

아래 예제는 실행 가능한 형태의 선택 실습입니다. 기본 Run All에는 추가 compile을 넣지 않습니다. Shape를 하나씩 바꾸고 graph/group/latency 변화를 설명해 보세요.

```
# Reduction 축을 늘리기
changed = experiment('inductor', workload='residual_layernorm', channels=128)
viz.fusion_groups(changed['captures'][0]).show()

# 같은 FP16 조건의 manual/SDPA 비교 (지원 backend는 실제 trace 확인)
# DTYPE_NAME을 'float16'으로 바꾸기 전에 새 커널에서 전체 설정을 일치시키세요.

# Nsight 권한이 있는 환경에서 현재 kernel의 counter 생성
measured = experiment('triton_codegen', nsight=True)
show_summary(measured['hardware_counters'])
```

14.4 참고와 구현 위치

이 노트북은 제공된 85쪽 강의자료의 §1-16을 학습 순서로 다시 구성했습니다. 새로 생성하는 작은 nanoGPT의 출처와 라이선스는 README.md, LICENSE.nanogpt에 있습니다. 모델 소스는 model.py, 작은 수식은 examples.py, 독립 실험은 experiments/, 그림 함수는 visuals.py에 있습니다.

설치 버전의 동작을 더 확인할 때 읽을 공식 문서:

- PyTorch 2.8 Dynamo overview: frame hook, guards, custom backend의 경계.
- PyTorch 2.8 compile troubleshooting: graph break-recompilation 진단.
- PyTorch 2.8 compiler getting started: compile 실행 흐름.
- Triton vector addition tutorial: program, offset, mask를 읽는 입문 예제.
- Plotly network graphs: node/edge를 interactive figure로 표현하는 방식.

진행 중의 출력이 길면 접힌 원본을 필요할 때만 열고, 먼저 그림의 source·축·범례를 확인하세요. 산출물은 study02/artifacts/에 남으며 Git에서 제외됩니다. 루트 tmp/의 별도 확인용 복사본은 실험 입력으로 사용하지 않습니다.

```
# Reduction 축을 늘리기
changed = experiment('inductor', workload='residual_layernorm', channels=128)
viz.fusion_groups(changed['captures'][0]).show()

# 같은 FP16 조건의 manual/SDPA 비교 (지원 backend는 실제 trace 확인)
# DTYPE_NAME을 'float16'으로 바꾸기 전에 새 커널에서 전체 설정을 일치시키세요.

# Nsight 권한이 있는 환경에서 현재 kernel의 counter 생성
```

```
measured = experiment('triton_codegen', nsight=True)
show_summary(measured['hardware_counters'])
```